

Question - 1

SQL: Detecting Potential Payment Fraud in an Online Marketplace

The company needs a report identifying users who have failed transactions using different payment methods. Failed transactions have "Failed" in the *status* field.

The result should have the following columns: *user_id* | *failed_transactions* | *distinct_payment_methods*.

- user_id* – User attempting multiple failed transactions.
- failed_transactions* – Total number of failed transactions.
- distinct_payment_methods* – Total number of unique payment methods used.

Note:

- Only users who have made more than 5 failed transactions in the entire dataset should be included in the report.
- Row order does not matter.

▼ Schema

transactions

Name	Type	Constraint	Description
id	INT	PRIMARY KEY	Unique identifier for a transaction
user_id	INT		User attempting the payment
payment_method	VARCHAR(255)		Payment method used
amount	DECIMAL(10,2)		Transaction amount
transaction_date	DATE		Date of the transaction
status	VARCHAR(255)		Status of transaction

▼ Sample Data Tables

transactions

id	user_id	payment_method	amount	transaction_date	status
101	202	Credit Card	200.43	2025-02-16	Completed
102	203	Netbanking	3233.10	2025-03-11	Failed
103	203	Netbanking	1195.35	2025-02-24	Failed
104	203	Debit Card	376.11	2025-03-10	Failed
105	203	Netbanking	112.01	2025-04-04	Failed
106	203	Credit Card	111.1	2025-09-12	Failed
107	203	Debit Card	2344.5	2025-10-03	Failed

Sample Output

```
user_id  failed_transactions  distinct_payment_methods
203      6                    3
```

Explanation

The user with *user_id* 203 attempted 6 distinct transactions, which have the status as "Failed", thus it is a potential case of fraud.

Question - 2
SQL: Average Response Time

A customer support team wants to analyze response times for resolving tickets to identify performance metrics and improve service quality. The goal is to generate a report calculating the average response time for successfully resolved customer support tickets. Resolved tickets have a value in the *resolved_at* field.

The result should have the following columns: *average_response_time*.

- average_response_time* - The average time between *created_at* and *resolved_at*, calculated in hours and set to two decimal places, including trailing zeros if necessary (e.g., 5.00).

▼ Schema

support_tickets			
Name	Type	Constraint	Description
id	INT	PRIMARY KEY	Unique identifier for each support ticket
customer_id	INT		Reference to the customer who created the ticket
created_at	VARCHAR(19)		Date and time when the ticket was created
resolved_at	VARCHAR(19)		Date and time when the ticket was resolved

▼ Sample Data Tables

support_tickets			
id	customer_id	created_at	resolved_at
1	1	2023-12-21 05:42:00	2024-01-01 05:42:00
2	2	2023-07-08 14:22:00	NULL
3	3	2023-05-22 08:54:00	2023-06-17 08:54:00

Sample Output

```
average_response_time
444.00
```

Explanation

The sample output shows the average response time for resolved tickets to be 444.00, excluding Ticket *id* 2 since it has not been resolved.

Question - 3

SQL: Highest-Spending Customers per City

A retail company wants to identify the highest-spending customer in each city to target them for personalized marketing campaigns and loyalty programs. The goal is to generate a report highlighting the top customer by total spending in each city location. Row order does not matter.

The result should have the following columns: *customer_id* / *name* / *city* / *total_spending*.

- customer_id* - Unique identifier for the customer.
- name* - Name of the customer.
- city* - The city where the customer is located.
- total_spending* - The total spending of the customer, calculated by summing all order amounts for each customer, and should be converted to an integer by rounding down using an appropriate function, e.g., 1.99 rounds to 1.

Note:

- Only include the highest-spending customer(s) from each city based on their total spending. That is, if the maximum highest amount spent in a city is 100, include all customers in that city that spent 100.

▼ Schema

customers

Name	Type	Constraint	Description
id	INT	PRIMARY KEY	Unique identifier for a customer
name	VARCHAR(255)		Name of the customer
city	VARCHAR(255)		City where the customer is located

orders

Name	Type	Constraint	Description
id	INT	PRIMARY KEY	Unique identifier for an order
customer_id	INT	FOREIGN KEY(customer_id => customers.id)	Reference to the customer
amount	DECIMAL(10,2)		Total amount of the order

▼ Sample Data Tables

customers

id	name	city
1	Customer 1	Los Angeles
2	Customer 2	Chicago
3	Customer 3	Chicago

orders

id	customer_id	amount
1	1	150.75
2	2	230.50
3	3	345.25

Sample Output

customer_id	name	city	total_spending
1	Customer 1	Los Angeles	150
3	Customer 3	Chicago	345

Explanation

The sample output shows the highest-spending customers in each city with their customer ID, name, city, and total spending. For "Los Angeles", "Customer 1" has the highest spending of 150. In "Chicago", "Customer 2" has a total spending of 230, but "Chicago" top spender is "Customer 3", with a total of 345.

Question - 4

SQL: E-commerce Product Request Report

An e-commerce platform maintains a database to track its products and customer requests for these products. The task is to generate a report that lists each available product's name and the total number of requests received for it.

The result should have the following columns: *product_name* / *total_requests*.

- product_name* - the name of the available product.
- total_requests* - the total number of requests received for the product.

The result should be sorted in descending order based on *total_requests*, and in case of a tie, by *product_name* in ascending order.

Note:

- Only products that are currently available should be included in the report.
- The *is_available* field in the products table indicates whether a product is available (1 for available, 0 for not available).

▼ Schema

products			
name	type	constraints	description
id	INT	NOT NULL PRIMARY KEY	The identifier of the product
name	VARCHAR(255)		The name of the product
category	VARCHAR(255)		The category of the product
is_available	SMALLINT		The flag indicating if the product is available

requests			
name	type	constraints	description
product_id	INT	FOREIGN KEY REFERENCES products(id)	The reference to the product
client_email	VARCHAR(255)		The email address of the client

products			
id	name	category	is_available
1	PromoPro	beauty products	1
2	AdVantage	outdoor gear	1
3	MarketMagnet	sports equipment	1
5	AdBlitz	beauty products	0

requests	
product_id	client_email
1	salgate1@fc2.com
1	lwycliff6@list-manage.com
1	ekimbleyf@scientificamerican.com
2	bgooro@spotify.com
2	vsamwayest@bbb.org
3	apappin0@yellowbook.com
3	ringreyb@businessinsider.com
3	mrysonm@istockphoto.com
5	ayushin1c@opera.com
5	bcoulston1q@hubpages.com

Sample Output

```
MarketMagnet 3
PromoPro      3
AdVantage     2
```

Explanation

The sample output lists the available products with their total request counts. 'PromoPro' and 'MarketMagnet' both have 3 requests, but 'MarketMagnet' appears first due to alphabetical ordering. 'AdVantage' has 2 requests and is listed after the others.

Question - 5
SQL: Active Campaign Engagement Report

A marketing company maintains a database to track its advertising campaigns and engagements. The task is to generate a report that includes the name of each active campaign, the total number of engagements, and the sum of views and clicks for those engagements.

The result should have the following columns: *campaign_name* / *total_engagements* / *total_views_and_clicks*.

- *campaign_name* - the name of the active campaign
- *total_engagements* - the number of engagements
- *total_views_and_clicks* - the combined number of views and clicks

The result should be sorted in ascending order by *campaign_name*.

Note:

- Only active campaigns should be included in the report.
- The *is_active* field in the campaigns table indicates whether a campaign is active (1 for active, 0 for inactive).

▼ Schema

campaigns			
name	type	constraints	description
id	INT	NOT NULL PRIMARY KEY	The identifier of the campaign
name	VARCHAR(255)		The name of the campaign
is_active	SMALLINT		The flag indicates if the campaign is active

engagements			
name	type	constraints	description
campaign_id	INT	FOREIGN KEY REFERENCES campaigns(id)	The reference to the campaign
views	INT		The total number of views of the engagement
clicks	INT		The total number of clicks of the engagement

▼ Sample Data Tables

campaigns		
id	name	is_active
1	SummerSavings	1
2	FallFrenzy	1
3	WinterWonderland	0

engagements		
campaign_id	views	clicks
1	100	10
1	150	20
2	200	30
2	250	40
3	300	50
1	120	15
2	180	25
3	220	35
1	130	18
2	210	28

Sample Output

```
FallFrenzy      3 963
SummerSavings  4 443
```

Explanation

The output includes only the active campaigns. For each active campaign, the total number of engagements is calculated by counting the number of records in the engagements table for that campaign. The number of views and clicks is calculated by summing (views + clicks) for each engagement related

to the campaign. The campaigns are then sorted by their names in ascending order.

Question - 6

SQL: Tax Report Summary

An online tax reporting application needs a summary report of account activity. It should list each account's email and the sum of reported amounts during 2023.

The result should have the following columns: *email* / *total_report_amount*.

- email* - the email address of the account.
- total_report_amount* - the sum of reported amounts submitted in 2023, rounded to two decimal places.

The results should be sorted in ascending order by *email*.

Note:

- Only reports submitted in the year 2023 should be included in the report.
- Ensure all decimal values are formatted to include trailing zeros if necessary (e.g., 5.00).

▼ Schema

accounts			
name	type	constraints	description
id	INT	NOT NULL PRIMARY KEY	The identifier of the account
email	VARCHAR(255)		The email address of the account

reports			
name	type	constraints	description
account_id	INT	FOREIGN KEY REFERENCES accounts(id)	The reference to the account
dt	VARCHAR(19)		The date and time of report
amount	DECIMAL(6, 2)		The reported amount

▼ Sample Data Tables

accounts	
id	email
1	hratke0@disqus.com
2	lcaiger1@si.edu
3	gburkett2@vinaora.com

reports		
account_id	dt	amount
1	2023-05-27 01:46:19	830.45
2	2023-01-15 09:23:21	2518.18
3	2023-05-08 01:44:41	4637.39
1	2023-06-30 15:02:03	3953.69

2	2023-12-05 04:39:31	3357.99
3	2023-02-03 09:41:00	1907.38
1	2022-12-30 04:05:57	1217.29
2	2024-01-24 14:18:07	2441.66
3	2024-01-05 23:19:31	3055.2
1	2023-05-26 01:54:24	2077.36

Sample Output

```
gburkett2@vinaora.com 6544.77
hratke0@disqus.com 6861.50
lcaiger1@si.edu 5876.17
```

Explanation

The sample output shows the total amount of reports for each account email in 2023. The amounts are summed and rounded to two decimal places. The output is sorted by the account email in ascending order.

Question - 7

SQL: Antivirus Device Scan Report

An antivirus software company maintains a database to track devices and the files scanned on each device. The task is to generate a report that lists each device's MAC address along with the total number of files scanned and the total number of infected files for that device.

The result should have the following columns: *mac_address* / *total_files_scanned* / *total_infected_files*.

- mac_address* - the MAC address of the device.
- total_files_scanned* - the total number of files scanned on the device.
- total_infected_files* - the total number of infected files on the device.

The result should be sorted in ascending order by *mac_address*.

- Note:**
- The *is_infected* field in the scanned files table indicates whether a file is infected (1 for infected, 0 for not infected).

▼ Schema

devices

name	type	constraints	description
id	INT	NOT NULL PRIMARY KEY	The identifier of the device
mac_address	VARCHAR(255)		The MAC address of the device

scanned_files

name	type	constraints	description
device_id	INT	FOREIGN KEY REFERENCES devices(id)	The reference to the device
filename	VARCHAR(255)		The name of the file
is_infected	SMALLINT		The flag indicating if the file is infected

▼ Sample Data Tables

devices

id	mac_address
1	66-0F-84-41-B8-8E
2	A6-1A-2F-3A-7B-83
3	76-CD-24-48-F0-DD

scanned_files		
device_id	filename	is_infected
1	File1.mp3	0
1	File2.xls	1
2	File3.doc	0
2	File4.ppt	1
2	File5.mp3	1
3	File6.xls	0
3	File7.doc	1
3	File8.ppt	0
3	File9.mp3	1
3	File10.xls	0

Sample Output

```
66-0F-84-41-B8-8E 2 1
A6-1A-2F-3A-7B-83 3 2
76-CD-24-48-F0-DD 5 2
```

Explanation

The report shows the MAC address of each device along with the total number of files scanned and the total number of infected files. For example, the device with MAC address '66-0F-84-41-B8-8E' has 2 files scanned, out of which 1 is infected.

Question - 8

SQL: Cryptocurrency Transactions Report

In the cryptocurrency market, a database engineer is tasked with generating a report for all cryptocurrency coins and their associated transactions. The report should include the name of each coin, the total amount of transactions, and the total number of transactions for each coin in the year 2023.

The result should have the following columns: *coin_name* / *total_transaction_amount* / *total_transactions*.

- coin_name* - the name of the cryptocurrency coin.
- total_transaction_amount* - the sum of transaction amounts in 2023, rounded to two decimal places.
- total_transactions* - the total number of transactions in 2023

The result should be sorted in ascending order by *coin_name*.

- Note:**
- Only transactions that occurred in 2023 should be included in the report.
 - Ensure decimal values are formatted to include trailing zeros if necessary, e.g., 5.00.

▼ Schema

coins			
name	type	constraints	description
id	INT	NOT NULL PRIMARY KEY	The identifier of the cryptocurrency coin
name	VARCHAR(255)		The name of the cryptocurrency coin

transactions			
name	type	constraints	description
coin_id	INT	FOREIGN KEY REFERENCES coins(id)	The reference to the cryptocurrency coin
dt	VARCHAR(19)		The date and time of the transaction
amount	DECIMAL(5, 2)		The amount of the transaction

▼ Sample Data Tables

coins	
id	name
1	BitCash
2	Etherium
3	Litecoin

transactions		
coin_id	dt	amount
1	2023-07-03 12:16:53	34.32
1	2023-12-08 12:14:58	47.59
2	2022-12-16 20:42:10	45.54
2	2023-11-05 09:27:11	53.3
3	2023-12-05 06:45:23	71.51
3	2023-01-19 01:43:25	97.18
3	2024-01-24 13:34:00	86.68
1	2023-05-07 05:30:06	25.6
2	2023-03-08 08:07:20	40.11
3	2023-08-13 10:44:54	87.54

Sample Output

```
BitCash  107.51 3
Etherium  93.41 2
Litecoin 256.23 3
```

Explanation

The sample output shows the total transaction amount and count for each coin in the year 2023. For 'BitCash', there are three transactions totaling 107.51. 'Etherium' has two transactions totaling 93.41. 'Litecoin' has three transactions totaling 256.23. All amounts are rounded to two decimal places.

Question - 9

SQL: Customer Domain Ownership Report

A domain hosting company maintains a database to manage its customers and the domains they own. The task is to generate a report that lists each customer's email address along with the total number of domains they own.

The result should have the following columns: *email* / *total_domains*.

- email* - the email address of the customer.
- total_domains* - the total number of domains owned by the customer.

The result should be sorted in ascending order by *email*.

▼ Schema

customers			
name	type	constraints	description
id	INT	NOT NULL PRIMARY KEY	The identifier of the customer
email	VARCHAR(255)		The email address of the customer

domains			
name	type	constraints	description
customer_id	INT	FOREIGN KEY REFERENCES customers(id)	The reference to the customer
name	VARCHAR(255)		The name of the domain

▼ Sample Data Tables

customers	
id	email
1	ebayldon0@washingtonpost.com
2	agammade1@comcast.net
3	goloshkin2@reference.com
4	cantonescu3@earthlink.net
5	fparzis4@ow.ly
6	cpetroulis5@shutterfly.com
7	tbeels6@bbb.org
8	zmacturlough7@4shared.com
9	eshury8@skype.com
10	jfehners9@github.io

domains	
customer_id	name
1	bfilipa.net

1	gsparsholti.net
1	jhughsr.org
2	scopas8.net
2	cglison1u.org
3	tginiz.com
3	arubinowitsch2l.net
3	clockyear2m.org
4	sfinnigand.com
4	vborreлт.net

Sample Output

```
agammade1@comcast.net 2
cantonescu3@earthlink.net 2
ebayldon0@washingtonpost.com 3
goloshkin2@reference.com 3
```

Explanation

The output lists the email addresses of customers along with the total number of domains they own. For instance, the customer with the email 'ebayldon0@washingtonpost.com' owns 3 domains, while 'agammade1@comcast.net' owns 2 domains. The result is sorted by email addresses in ascending order.

Question - 10

SQL: E-commerce Wishlist Report

Generate a report from an e-commerce database that lists the product names and prices, along with the total number of times each is on a wishlist.

The result should have the following columns: *product_name* / *price* / *total_wishlist_count*.

- product_name* - the name of the product.
- price* - its price
- total_wishlist_count* - the total number of times it appears in wishlists

The result should be sorted in ascending order by *product_name*.

Note:

- Only include products that are currently in stock.
- Ensure all decimal values are formatted to include trailing zeros if necessary, e.g., 5.00.

▼ Schema

products			
name	type	constraints	description
id	INT	NOT NULL PRIMARY KEY	The identifier of the product
name	VARCHAR(255)		The name of the product
price	DECIMAL(6, 2)		The price of the product
in_stock	SMALLINT		1 indicates 'in stock', 0 indicates 'out of stock'

wishlists

name	type	constraints	description
product_id	INT	FOREIGN KEY REFERENCES products(id)	The reference to the product
customer_email	VARCHAR(255)		The email address of the customer

▼ Sample Data Tables

products

id	name	price	in_stock
1	TechGadget Pro X	324.24	1
2	LuxuryHome Decor Set	884.9	1
3	FitnessTracker Elite	698.59	0

wishlists

product_id	customer_email
1	user1@example.com
1	user2@example.com
2	user3@example.com
2	user4@example.com
2	user5@example.com
3	user6@example.com
1	user7@example.com
2	user8@example.com
1	user9@example.com
3	user10@example.com

Sample Output

LuxuryHome Decor Set	884.9	3
TechGadget Pro X	324.24	4

Explanation

The report includes only products that are in stock. 'TechGadget Pro X' is in stock and appears 4 times in wishlists, while 'LuxuryHome Decor Set' is also in stock and appears 3 times. 'FitnessTracker Elite' is not included in the report as it is not in stock.

Question - 11

SQL: Email Campaign Report

An email campaign tracking platform maintains data on various campaigns and their email statistics. The task is to generate a report that lists each campaign's name along with the total number of emails sent, emails opened, and emails not opened.

The result should have the following columns: *campaign_name* / *total_emails_sent* / *total_emails_opened* / *total_emails_not_opened*.

- campaign_name* - the name of the email campaign.
- total_emails_sent* - the total number of emails sent in the campaign.
- total_emails_opened* - the total number of emails opened in the campaign.

- *total_emails_not_opened* - the total number of emails not opened.

The result should be sorted in ascending order by *campaign_name*.

Note:

- The number of emails not opened is calculated as the difference between the total emails sent and the emails opened.

▼ Schema

campaigns			
name	type	constraints	description
id	INT	NOT NULL PRIMARY KEY	The identifier of the email campaign
name	VARCHAR(255)		The name of the email campaign

email_stats			
name	type	constraints	description
campaign_id	INT	FOREIGN KEY REFERENCES campaigns(id)	The reference to the email campaign
emails_sent	INT		The number of emails sent in the email campaign
emails_opened	INT		The number of emails opened in the email campaign

▼ Sample Data Tables

campaigns	
id	name
1	SummerSale2021
2	FallPromo
3	WinterWonderland

email_stats		
campaign_id	emails_sent	emails_opened
1	1000	800
2	1500	1200
3	2000	1800
1	500	300
2	700	500
3	800	600
1	300	200
2	400	300
3	600	500
3	400	300

Sample Output

FallPromo	2600	2000	600
SummerSale2021	1800	1300	500
WinterWonderland	3800	3200	600

Explanation

The report shows each campaign's name along with the total emails sent, opened, and not opened. For example, the 'FallPromo' campaign had a total of 2600 emails sent, 2000 emails opened, and 600 emails not opened, calculated as the difference between sent and opened.

Question - 12

SQL: Auction Lot Offers Report

In an e-commerce auction platform, the task is to generate a report that provides insights into the bidding activities on various lots. The report should list each lot's name, the highest offer made for that lot, and the total number of offers received.

The result should have the following columns: *lot_name* / *highest_offer* / *total_offers*.

- lot_name* - the name of the lot.
- highest_offer* - the highest offer made for the lot.
- total_offers* - the total number of offers received for the lot.

The result should be sorted in ascending order by *lot_name*.

- Note:
- Ensure all decimal values are formatted to include trailing zeros if necessary (e.g., 5.00).

▼ Schema

lots			
name	type	constraints	description
id	INT	NOT NULL PRIMARY KEY	The identifier of the lot
name	VARCHAR(255)		The name of the lot

offers			
name	type	constraints	description
lot_id	INT	FOREIGN KEY REFERENCES lots(id)	The identifier of the lot for which the offer is made
amount	DECIMAL(6, 2)		The amount of the offer

▼ Sample Data Tables

lots	
id	name
1	Acacia parramattensis Tindale
2	Poa arctica R. Br. ssp. aperta (Scribn. & Merr.) Soreng
3	Calophyllum inophyllum L.

offers	
lot_id	amount
1	260.91
1	802.83

1	986.78
2	814.57
2	999.06
2	414.67
3	200.41
3	593.07
3	701.88
3	972.87

Sample Output

Acacia parramattensis Tindale	986.78	3
Calophyllum inophyllum L.	972.87	4
Poa arctica R. Br. ssp. aperta (Scribn. & Merr.) Soreng	999.06	3

Explanation

The sample output shows the highest offer and the total number of offers for each lot. For example, the lot 'Acacia parramattensis Tindale' received a maximum offer of 986.78 and a total of 3 offers. The decimal values are rounded to two decimal places.

Question - 13

SQL: Online Banking Transactions Report

A financial services company needs a report of account transactions for September 2022.

For each account, the report should have the following columns: *IBAN* / *min_transaction* / *max_transaction* / *avg_transaction* / *total_transactions*.

- IBAN* - the account's International Bank Account Number
- min_transaction* - the minimum transaction amount
- max_transaction* - the maximum transaction amount
- avg_transaction* - the average transaction amount
- total_transactions* - the total number of transactions

The results should be sorted in ascending order by *IBAN* to facilitate easy reference.

Note:

- Only transactions from September 2022 should be included in the report.
- Ensure all decimal values are formatted to include trailing zeros if necessary, e.g., 5.00.

▼ Schema

accounts			
name	type	constraints	description
id	INT	NOT NULL PRIMARY KEY	The identifier of the account
iban	VARCHAR(255)		The IBAN of the account

transactions			
name	type	constraints	description
account_id	INT	FOREIGN KEY REFERENCES accounts(id)	The identifier of the account related to the transaction

dt	CHAR(19)		The date and time of the transaction
amount	DECIMAL(5, 2)		The amount of the transaction

▼ Sample Data Tables

accounts	
id	iban
1	BG40 RFFX 4898 53DD CZD6 KQ
2	PT42 5267 0592 8451 8001 2180 3
3	FR96 8758 8909 81LR DJ71 ERKN D56

transactions		
account_id	dt	amount
1	2022-09-02 06:33:39	33.31
1	2022-09-20 08:14:39	31.77
1	2022-09-25 06:41:45	72.84
2	2022-09-04 22:28:12	35.26
2	2022-09-17 07:57:29	33.27
2	2022-09-27 22:30:36	70.78
3	2022-09-16 21:54:12	75.04
3	2022-09-19 18:27:39	71.19
3	2022-09-28 01:38:56	14.34
3	2022-08-30 01:35:31	69.19

Sample Output

BG40 RFFX 4898 53DD CZD6 KQ	31.77	72.84	45.97	3
PT42 5267 0592 8451 8001 2180 3	33.27	70.78	46.44	3
FR96 8758 8909 81LR DJ71 ERKN D56	14.34	75.04	53.52	3

Explanation

The sample output shows the IBAN for each account along with the minimum, maximum, and average transaction amounts for September 2022, rounded to two decimal places. It also includes the total number of transactions for each account during that month.

Question - 14

SQL: Advertising Campaign Engagement Summary

In digital advertising, gauging campaign engagement levels is crucial for refining marketing strategies and enhancing client satisfaction. A development team is creating a query for the customer panel dashboard to generate a report showing campaign engagement metrics.

The result should have the following columns: *name* / *total_views* / *total_clicks* / *engagement_rate*.

- name* - the name of the advertising campaign
- total_views* - the total number of views the campaign received
- total_clicks* - the total number of clicks the campaign generated

- *engagement_rate* - the engagement rate of the campaign, calculated as $(total_clicks / total_views) * 100$, rounded to two decimal places, including trailing zeros if necessary, e.g., 500.00

The results should be sorted in descending order by *engagement_rate* and then in ascending order by *name*.

Note:

- Only campaigns that are active should be included.

▼ Schema

campaigns

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the campaign
name	VARCHAR(255)		The name of the campaign
is_active	BOOLEAN		The flag indicating if the campaign is active

engagements

Name	Type	Constraints	Description
campaign_id	INT	FOREIGN KEY(campaign_id => campaigns.id)	The reference to the campaign
views	INT		The total number of views of the engagement
clicks	INT		The total number of clicks of the engagement

▼ Sample Data Tables

campaigns

id	name	is_active
1	SummerSavings	1
2	FallFrenzy	0
3	WinterWonderland	1

engagements

campaign_id	views	clicks
1	130	105
1	182	122
1	395	368
1	408	159
1	587	537
1	805	704
1	814	65
1	846	446
1	932	462

1	943	491
1	944	896
2	60	59
2	413	245
2	784	187
3	68	53
3	210	35
3	582	217
3	862	841
3	938	799
3	956	255

Sample Output

name	total_views	total_clicks	engagement_rate
SummerSavings	6986	4355	62.34
WinterWonderland	3616	2200	60.84

Question - 15

SQL: Most Requested Advertising Products Summary

In the advertising industry, staying attuned to client demands and preferences is key to optimizing ad inventory and crafting effective sales strategies. A development team is creating a query to generate a detailed report identifying all the requested advertising products. This comprehensive report aims to spotlight products currently available for advertising campaigns, offering insights into client interests and guiding inventory management decisions.

The result should have the following columns: *name* / *category* / *total_requests*.

- name* - the name of the advertising product
- category* - the category of the advertising product
- total_requests* - the total number of requests made by clients for the advertising product

The results should be sorted in descending order by *total_requests* to clearly identify the most popular advertising products, then in ascending order by *name*, and then by *category*.

Note:

- Only advertising products that are available should be included.

▼ Schema

products			
Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the product
name	VARCHAR(255)		The name of the product

category	VARCHAR(255)		The category of the product
is_available	BOOLEAN		The flag indicating if the product is available

requests

Name	Type	Constraints	Description
product_id	INT	FOREIGN KEY(product_id => products.id)	The reference to the product
client_email	VARCHAR(255)		The email address of the client

▼ Sample Data Tables

products

id	name	category	is_available
1	PromoPro	home goods	1
2	AdVantage	beauty products	1
3	MarketMagnet	apparel	0

requests

product_id	client_email
1	bsyder0@cnbc.com
1	cvangiffenf@prweb.com
1	eknaggesa@fc2.com
1	jchecchi3@geocities.com
1	jfacher7@amazonaws.com
1	lbrettor8@washington.edu
1	merickson4@amazon.de
1	nmacaloreni@skype.com
2	arasell2@ox.ac.uk
2	cbinestead6@icio.us
2	fmiddasd@ovh.net
2	ggowlerc@bbc.co.uk
2	lwestoll9@opensource.org
2	vyushkinb@amazonaws.com
3	aleipelte@linkedin.com
3	creynardh@phpbb.com
3	fsimondson1@noaa.gov
3	kfarraj@wordpress.org
3	mgoodg@ucoz.ru

Sample Output

name	category	total_requests	
PromoPro	home goods	8	
AdVantage	beauty products	6	

Question - 16

SQL: Top Wishlist Products Summary

In the competitive e-commerce landscape, understanding customer preferences and product popularity is crucial for inventory management and marketing strategies. A development team is creating a report that identifies the top 3 products in customers' wishlists. This report will highlight in-stock products, allowing immediate action to capitalize on customer interest.

The result should have the following columns: *name* / *price* / *total_wishes*.

- name* - the name of the product
- price* - the price of the product
- total_wishes* - the total number of times the product appears in customers' wishlists

The results should be sorted in descending order by *total_wishes* to easily identify the most desired products, then in ascending order by *name*.

The result should be limited to the top 3 products.

Note:

- Only products that are in stock should be included.

▼ Schema

products			
Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the product
name	VARCHAR(255)		The name of the product
price	DECIMAL(6,2)		The price of the product
in_stock	SMALLINT		The flag indicating if the product in stock

wishlists			
Name	Type	Constraints	Description
product_id	INT	FOREIGN KEY(product_id => products.id)	The reference to the product
customer_email	VARCHAR(255)		The email address of the customer

▼ Sample Data Tables

products			
id	name	price	in_stock

1	TechGadget Pro X	274.80	1
2	LuxuryHome Decor Set	262.84	1
3	FitnessTracker Elite	637.92	0
4	GourmetCookware Set	535.34	1
5	Fashionista Wardrobe Collection	525.44	1

wishlists	
product_id	customer_email
1	crabbec@redcross.org
1	efindlow2@tinypic.com
1	jmachoste5@issuu.com
1	nselle@simplemachines.org
2	aonn1@ebay.co.uk
2	bbolton0@google.cn
2	ebockett3@storify.com
2	fdunguyg@symantec.com
2	slowried@cbsnews.com
3	jgately7@goo.ne.jp
3	ospearettj@bandcamp.com
3	rpanonsb@paypal.com
3	ydevauxh@toplist.cz
3	zbabbage9@imageshack.us
4	dpauleya@cnbc.com
4	jletterick4@dailymotion.com
4	khunnisett6@princeton.edu
4	rkernellf@uiuc.edu
5	blodin8@wikimedia.org
5	lyusupovi@nps.gov

Sample Output

```
+-----+-----+-----+
|name           |price |total_wishes|
+-----+-----+-----+
|LuxuryHome Decor Set|262.84|5           |
|GourmetCookware Set |535.34|4           |
|TechGadget Pro X    |274.80|4           |
+-----+-----+-----+
```

Question - 17

SQL: E-commerce Customer Purchases Report

In e-commerce, understanding customer purchasing behavior is essential for tailoring marketing strategies and improving customer service. A development team is creating a query for a report that details customer activity in March 2024. This report will provide insights into the total number and sum of purchases made by each customer, facilitating targeted engagement and promotional efforts.

The result should have the following columns: *email* / *total_purchases* / *total_purchase_amount*.

- email* - the email address of the customer
- total_purchases* - the total number of purchases made by the customer
- total_purchase_amount* - the total sum of the purchases made by the customer, with two decimal places, including trailing zeros if necessary, e.g., 500.00

The results should be sorted in ascending order by *email*.

Note:

- Only purchases made in March 2024 should be included.

▼ Schema

customers

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the customer
email	VARCHAR(255)		The email address of the customer

purchases

Name	Type	Constraints	Description
customer_id	INT	FOREIGN KEY(customer_id => customers.id)	The reference to the customer
dt	VARCHAR(19)		The date and time of the purchase
amount	DECIMAL(6,2)		The amount of the purchase

▼ Sample Data Tables

customers

id	email
1	floggie0@newsvine.com
2	sgillbe1@ca.gov
3	jgohn2@elegantthemes.com

purchases

customer_id	dt	amount
2	2024-02-21 02:56:12	228.58
2	2024-02-23 09:32:47	972.41
1	2024-03-14 15:50:13	109.16

1	2024-03-17 00:31:44	11.49
1	2024-03-17 04:15:42	692.64
2	2024-03-01 04:35:09	589.74
2	2024-03-13 14:42:23	508.75
2	2024-03-17 07:57:36	933.91
2	2024-03-19 08:24:38	488.26
2	2024-03-31 23:30:54	55.07
3	2024-03-03 11:34:30	816.67
3	2024-03-08 23:46:07	672.93
3	2024-03-15 18:09:56	260.66
3	2024-03-20 15:18:11	321.07
3	2024-03-20 17:40:35	29.06
3	2024-03-20 23:41:39	314.85
3	2024-03-25 11:41:07	67.12
1	2024-04-05 03:05:10	417.78
2	2024-04-09 08:16:17	697.53
3	2024-04-02 07:56:48	156.27

Sample Output

email	total_purchases	total_purchase_amount
floggie0@newsvine.com	3	813.29
jgohn2@elegantthemes.com	7	2482.36
sgillbel@ca.gov	5	2575.73

Question - 18

SQL: Report on Applicants Pending Consular Service

In managing consular services, addressing delays caused by system issues is critical. A development team is creating a query for the system's dashboard to generate a report that identifies applicants who have not received service due to delays, emphasizing the extent of these delays. This report is essential for prioritizing these applicants and efficiently managing the backlog.

The result should have the following columns: *email* / *scheduled_appointment* / *days_of_delay*.

- email* - the email address of the applicant
- scheduled_appointment* - the date when the appointment was scheduled
- days_of_delay* - the number of full days between the scheduled appointment date and the current date, April 10, 2024

The result should be sorted in ascending order by *scheduled_appointment*, then in ascending order by *email*.

Note:

- Only applicants with an appointment prior to April 10, 2024, who have not yet received service should be included in the result.

▼ Schema

applicants

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the applicant
email	VARCHAR(255)		The email address of the applicant

appointments

Name	Type	Constraints	Description
applicant_id	INT	FOREIGN KEY(applicant_id => applicants.id)	The reference to the applicant
dt	VARCHAR(19)		The date of the scheduled appointment
is_received	BOOLEAN		The flag indicating if the service was received

▼ Sample Data Tables

applicants

id	email
1	nkienzle0@spiegel.de
2	alaste1@bbc.co.uk
3	jjochanany2@ow.ly
4	bsenn3@salon.com
5	bwhittall4@nhs.uk

appointments

applicant_id	dt	is_received
1	2024-04-27	0
2	2024-04-01	0
3	2024-04-15	0
4	2024-03-27	0
5	2024-03-26	1

Sample Output

```
+-----+-----+-----+
| email                | scheduled_appointment | days_of_delay |
+-----+-----+-----+
| bsenn3@salon.com     | 2024-03-27            | 14 |
| alaste1@bbc.co.uk    | 2024-04-01            | 9  |
+-----+-----+-----+
```

Question - 19

SQL: Weekend Appointments for Consular Services Rescheduling

In consular services, efficiently managing appointment schedules is key to effective service delivery and resource utilization. A development team is creating a query for the system's dashboard to list all appointments scheduled on weekends (Saturday or Sunday). This initiative aims to identify appointments that need to be rescheduled to a working day.

The result should have the following columns: *email* / *scheduled_appointment*.

- *email* - the email address of the user who made the appointment
- *scheduled_appointment* - the name of the day when the appointment is scheduled

The results should be sorted in ascending order by *email*.

Note:

- Only appointments that are scheduled on weekends (Saturday or Sunday) should be included.

▼ Schema

applicants

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the applicant
email	VARCHAR(255)		The email address of the applicant

appointments

Name	Type	Constraints	Description
applicant_id	INT	FOREIGN KEY(applicant_id => applicants.id)	The reference to the applicant
dt	VARCHAR(19)		The date and time of the scheduled appointment

▼ Sample Data Tables

applicants

id	email
1	rastlatt0@instagram.com
2	gcarmony1@stanford.edu
3	mgreenset2@state.tx.us

appointments

applicant_id	dt
1	2024-05-26 01:36:43
2	2024-05-27 16:30:28
3	2024-05-18 19:28:52

Sample Output

```
+-----+-----+
|email           |scheduled_appointment|
+-----+-----+
|mgreenset2@state.tx.us |Saturday              |
|rastlatt0@instagram.com|Sunday                |
+-----+-----+
```

Question - 20

SQL: Active Domains Registration by Country with Totals

Generate a report listing countries with active domain registrations and the total number of domains per country for strategic planning and market analysis.

The report should include:

- country_name* - The name of the country from where the domain was registered
- total_domains* - The total number of active domains registered from that country

Results should be sorted in ascending order by *country_name*.

Note:

- Only include active domains in the result.

▼ Schema

countries

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the country
name	VARCHAR(255)		The name of the country

domains

Name	Type	Constraints	Description
country_id	INT	FOREIGN KEY(country_id => countries.id)	The reference to the country
name	VARCHAR(255)		The name of the domain
is_active	BOOLEAN	0 (false) or 1 (true)	The status of the domain

▼ Sample Data Tables

countries

id	name
1	Azerbaijan
2	Colombia
3	China

domains

country_id	name	is_active
1	angelfire.com	1
1	free.fr	1
1	google.cn	1
1	nationalgeographic.com	1
1	ovh.net	1

1	surveymonkey.com	1
1	twitpic.com	1
2	ameblo.jp	1
2	berkeley.edu	1
2	multiply.com	1
2	redcross.org	1
2	sourceforge.net	1
3	hc360.com	1
3	liveinternet.ru	1
3	squidoo.com	1
3	technorati.com	1
3	webnode.com	1
3	yahoo.co.jp	1
1	1und1.de	0
1	qq.com	0

Sample Output

```
+-----+-----+
|country_name|total_domains|
+-----+-----+
|Azerbaijan  |7           |
|China       |6           |
|Colombia    |5           |
+-----+-----+
```

Question - 21

SQL: Domain Renewal Overview

Generate a report of all domains with their renewal dates to help domain owners plan timely renewals and maintain their online presence. The report should include:

- *name* - The name of the domain
- *today_date* - The current date, April 10, 2024, in the format YYYY-MM-DD
- *next_renewal_date* - The date when the domain is scheduled for its next renewal
- *days_until_renewal* - The number of full days from *today_date* (April 10, 2024) until the domain's next renewal date

Results should be sorted in ascending order by *days_until_renewal*, then in ascending order by *name*.

▼ Schema

domains			
Name	Type	Constraints	Description
name	VARCHAR(255)		The name of the domain

next_renewal_date	VARCHAR(19)		The date when the domain is scheduled for next renewal
-------------------	-------------	--	--

▼ Sample Data Tables

domains

name	next_renewal_date
wired.com	2024-06-14 00:10:12
blogger.com	2024-07-18 05:54:57
com.com	2024-07-21 02:57:25

Sample Output

name	today_date	next_renewal_date	days_until_renewal
wired.com	2024-04-10	2024-06-14	65
blogger.com	2024-04-10	2024-07-18	99
com.com	2024-04-10	2024-07-21	102

Question - 22

SQL: User Transaction Details

Create a comprehensive user transaction report for March to tailor engagement strategies and improve user experience.

The report should include:

- email* - The email address of the user
- total_transactions* - The total number of transactions made by the user during March 2024
- min_amount* - The minimum transaction amount by the user during March 2024, with two decimal places including trailing zeros when needed (e.g., 500.00)
- max_amount* - The maximum transaction amount by the user during March 2024, with two decimal places including trailing zeros when needed (e.g., 500.00)
- total_amount* - The total amount of all transactions made by the user during March 2024, with two decimal places including trailing zeros when needed (e.g., 500.00)

Results should be sorted in ascending order by *email*.

Note:

- Only include transactions that occurred in March.

▼ Schema

users

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the user
email	VARCHAR(255)		The email address of the user

transactions

Name	Type	Constraints	Description
------	------	-------------	-------------

user_id	INT	FOREIGN KEY(user_id => users.id)	The reference to the user
dt	VARCHAR(19)		The date and time of transaction
amount	DECIMAL(5,2)		The amount of transaction

▼ Sample Data Tables

users

id	email
1	bblaszczynski0@devhub.com
2	dwookey1@chronoengine.com
3	flerway2@wikipedia.org

transactions

user_id	dt	amount
1	2024-02-23 19:30:03	942.50
1	2024-03-07 09:01:15	855.22
1	2024-04-01 04:18:41	253.35
1	2024-04-07 02:40:58	886.88
2	2024-02-25 05:11:39	957.77
2	2024-03-06 03:00:40	413.39
2	2024-03-07 14:41:03	906.16
2	2024-03-10 00:58:13	116.59
2	2024-03-13 23:38:29	550.31
2	2024-03-22 03:07:46	196.23
2	2024-03-24 00:23:14	399.76
2	2024-03-25 12:28:18	398.07
2	2024-03-27 09:11:15	212.33
2	2024-04-09 06:33:26	97.85
3	2024-03-01 17:24:48	323.11
3	2024-03-05 10:16:06	673.23
3	2024-03-08 14:19:46	236.74
3	2024-03-23 15:37:47	234.87
3	2024-04-05 20:55:45	989.35
3	2024-04-07 05:26:35	369.20

Sample Output

email	total_transactions	min_amount	max_amount	total_amount
bblaszczyński0@devhub.com	1	855.22	855.22	855.22
dwookey1@chronoengine.com	8	116.59	906.16	3192.84
flerway2@wikipedia.org	4	234.87	673.23	1467.95

Question - 23

SQL: Total Transactions and Sum for Each User

Generate a report detailing the total number and sum of transactions for each user to identify highly active users and those requiring additional engagement.

The report should include:

- email* - The email address of the user
- total_transactions* - The number of transactions made by the user
- total_amount* - The sum of transactions made by the user, with two decimal places including trailing zeros when needed (e.g., 500.00)

Results should be sorted in ascending order by *email*.

Note:

- Only include transactions that occurred in 2023.

▼ Schema

users			
Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the user
email	VARCHAR(255)		The email address of the user

transactions			
Name	Type	Constraints	Description
user_id	INT	FOREIGN KEY(user_id => users.id)	The reference to the user
dt	VARCHAR(19)		The date and time of transaction
amount	DECIMAL(5,2)		The amount of transaction

▼ Sample Data Tables

users	
id	email
1	lvasilevich0@google.co.uk
2	hscholey1@sina.com.cn
3	mmcjury2@hibu.com

transactions		
user_id	dt	amount

3	2022-12-05 00:16:56	162.11
1	2023-05-20 03:20:58	81.58
1	2023-06-08 19:24:02	52.46
1	2023-06-27 21:16:07	447.59
1	2023-07-20 08:19:32	136.68
1	2023-12-11 17:08:05	852.55
1	2023-12-15 04:45:54	77.11
1	2023-12-22 00:46:34	670.71
1	2023-12-29 12:43:23	948.46
2	2023-01-04 00:51:46	793.50
2	2023-04-07 16:29:14	762.52
2	2023-06-17 17:42:50	527.18
2	2023-10-10 11:16:51	733.47
2	2023-10-18 23:32:00	920.14
3	2023-03-27 18:31:41	408.13
3	2023-04-08 09:57:55	817.88
3	2023-05-18 09:47:14	916.98
3	2023-09-14 14:00:54	53.30
3	2023-09-30 01:34:01	589.37
3	2024-01-27 15:13:58	666.37

Sample Output

email	total_transactions	total_amount
hscholey1@sina.com.cn	5	3736.81
lvasilevich0@google.co.uk	8	3267.14
mmcjury2@hibu.com	5	2785.66

Question - 24

SQL: Top Cryptocurrencies by Average Transaction Amount

Analyze cryptocurrencies based on average transaction amounts in 2023. Higher average transaction values often indicate increased investor confidence or significant institutional involvement. We want to know the least attractive coin types, i.e., those with the smallest average transaction amounts.

The report should include:

- name* - The name of the cryptocurrency
- avg_transaction_amount* - The average transaction amount for the cryptocurrency during 2023, rounded to two decimal places with trailing zeros when needed (e.g., 5.00)

Results should be sorted in ascending order by *avg_transaction_amount* and limited to the first 3 rows.

- Note:
- Only include transactions that occurred in 2023.

▼ Schema

coins			
Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of cryptocurrency
name	VARCHAR(255)		The name of the cryptocurrency

transactions			
Name	Type	Constraints	Description
coin_id	INT	FOREIGN KEY(coin_id => coins.id)	The reference to the cryptocurrency
dt	VARCHAR(19)		The date and time of transaction
amount	DECIMAL(5,2)		The amount of transaction

▼ Sample Data Tables

coins	
id	name
1	BitCash
2	Etherium
3	Litecoin
4	Ripple
5	Dogecoin

transactions		
coin_id	dt	amount
1	2022-12-09 19:40:17	60.91
1	2023-01-02 09:35:37	76.35
1	2023-03-21 09:34:39	23.11
1	2023-08-11 03:43:27	80.20
1	2023-10-21 19:42:46	29.59
2	2023-07-08 19:47:20	69.49
2	2023-09-22 14:23:40	23.13
3	2023-01-08 10:22:10	72.45
3	2023-01-28 00:54:51	98.72
3	2023-02-24 00:13:32	70.36

Name	Type	Constraints	Description
coin_id	INT	FOREIGN KEY(coin_id => coins.id)	The reference to the cryptocurrency coin
dt	VARCHAR(19)		The date and time of transaction
amount	DECIMAL(5,2)		The amount of transaction

▼ Sample Data Tables

coins

id	name
1	BitCash
2	Etherium
3	Litecoin

transactions

coin_id	dt	amount
1	2024-02-24 14:56:31	69.38
2	2024-02-24 17:23:54	46.79
1	2024-03-07 15:07:57	73.45
1	2024-03-13 00:47:18	2.10
1	2024-03-13 06:21:06	5.68
1	2024-03-14 15:06:59	25.32
1	2024-03-20 02:10:37	72.85
2	2024-03-09 12:06:47	67.79
2	2024-03-16 06:17:14	82.87
2	2024-03-24 11:11:23	5.96
2	2024-03-26 14:36:34	21.80
3	2024-03-20 08:28:56	5.07
3	2024-03-23 06:45:13	11.85
3	2024-03-27 02:40:23	34.25
3	2024-03-28 07:37:50	81.12
3	2024-03-29 07:34:32	19.06
1	2024-04-02 08:49:17	13.04
2	2024-04-04 17:29:13	4.74
2	2024-04-04 20:17:36	94.42
2	2024-04-07 01:47:46	64.76

Sample Output

name	total_transactions	min_amount	max_amount	avg_amount
BitCash	5	2.10	73.45	35.88
Litecoin	5	5.07	81.12	30.27
Etherium	4	5.96	82.87	44.61

Question - 26

SQL: Antivirus Suspicious File Extensions Report

Create a report showing the top 5 suspicious file extensions in March for an antivirus dashboard. It aims to give clients insights into common threats.

The result should include these columns:

- extension* - the extension of the suspicious file
- total_suspicious_files* - the number of times files with this extension were flagged as suspicious

The results should be sorted in descending order by *total_suspicious_files*.

Note:

- Only include files that were scanned and flagged as suspicious in March.
- Limit the results to the top 5 extensions.

▼ Schema

suspicious_files

Name	Type	Constraints	Description
filename	VARCHAR(255)		The name of the file
extension	VARCHAR(255)		The extension of the file
scan_dt	VARCHAR(19)		The date and time when the file was scanned
is_suspicious	BOOLEAN		The flag indicating whether the file was flagged as suspicious

▼ Sample Data Tables

suspicious_files

filename	extension	scan_dt	is_suspicious
Mauris.pdf	.pdf	2024-04-05 23:55:27	1
Augue.xls	.xls	2024-02-28 18:11:28	1
Sapien.avi	.avi	2024-03-30 12:24:10	1
Pulvinar.doc	.doc	2024-03-08 22:00:41	1
TemporConvallisNulla.gif	.gif	2024-03-29 21:32:41	1
InFaucibus.mp3	.mp3	2024-03-20 14:18:32	1
EleifendPedelLibero.ppt	.ppt	2024-03-05 04:47:56	1
VestibulumAntelpsum.ppt	.ppt	2024-03-05 17:34:34	1

IntegerPede.ppt	.ppt	2024-03-12 17:11:28	1
VenenatisNon.tiff	.tiff	2024-03-20 18:04:47	1
IaculisDiam.xls	.xls	2024-03-01 05:18:03	1
QuisqueArcuLibero.xls	.xls	2024-03-09 09:00:32	1
MaurisSit.png	.png	2024-04-03 23:20:03	0
SitAmetSem.mp3	.mp3	2024-02-23 22:06:43	0
Nisi.mp3	.mp3	2024-02-29 09:40:45	0
Magna.tiff	.tiff	2024-02-27 00:25:16	0
EratVestibulum.gif	.gif	2024-03-30 04:19:52	0
Neque.jpeg	.jpeg	2024-03-07 07:11:26	0
VolutpatQuam.ppt	.ppt	2024-03-23 04:33:43	0
NonQuam.xls	.xls	2024-03-10 19:12:29	0

Sample Output

```
+-----+-----+
|extension|total_suspicious_files|
+-----+-----+
|.ppt      |3                        |
|.xls      |2                        |
|.avi      |1                        |
|.doc      |1                        |
|.gif      |1                        |
+-----+-----+
```

Question - 27

SQL: Antivirus Scanned Devices Report

Create a report detailing the total number of devices scanned by antivirus software for an antivirus dashboard.

The result should include these columns:

- email* - the email address of the client
- total_scanned_devices* - the total number of devices scanned for each client

The results should be sorted in ascending order by *email*.

Note:

- Only include devices that were scanned in March.

▼ Schema

clients			
Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the client
email	VARCHAR(255)		The email address of the client

devices

Name	Type	Constraints	Description
client_id	INT	FOREIGN KEY(client_id => clients.id)	The reference to the client
mac_address	VARCHAR(255)		The MAC address of the device
is_scanned	BOOLEAN		The flag indicating whether the device was scanned
scheduled_scan_dt	VARCHAR(19)		The date and time the device is scheduled to be scanned

▼ Sample Data Tables

clients

id	email
1	cbracegirdle0@irs.gov
2	gwickardt1@msu.edu
3	mpaulon2@edublogs.org

devices

client_id	mac_address	is_scanned	scheduled_scan_dt
1	87-EF-C7-BD-DF-A2	1	2024-02-28 08:13:55
1	37-FE-45-2B-9D-2A	1	2024-03-07 00:00:05
1	13-82-F2-48-88-FD	1	2024-03-11 06:53:47
1	0D-56-2A-B2-33-EF	1	2024-03-30 19:41:31
1	17-C3-E3-2E-37-7E	1	2024-04-06 13:14:06
1	93-25-74-C5-07-32	0	2024-02-21 16:23:31
2	64-E1-5B-12-AC-F9	0	2024-02-22 10:40:23
2	0F-66-56-E2-B0-3A	0	2024-02-23 11:03:58
2	40-F4-40-12-C8-A5	0	2024-02-24 18:01:50
2	B0-2B-99-84-68-7C	0	2024-02-25 21:31:06
2	07-1F-BD-16-AC-23	0	2024-02-29 21:45:22
2	CA-79-F4-B4-9E-69	0	2024-03-29 00:05:10
3	71-EB-63-A2-3C-AF	1	2024-03-03 07:50:20
3	0B-40-DF-14-53-0F	1	2024-03-21 11:10:52
3	0A-77-ED-ED-50-28	1	2024-04-03 04:11:25
3	A4-79-0C-6D-B8-4C	1	2024-04-04 13:50:43
3	44-A5-56-27-C8-70	0	2024-03-28 21:15:26
3	93-64-42-51-62-6F	0	2024-03-31 20:26:01
3	87-5E-B3-51-38-2D	0	2024-04-05 10:51:00

3	DE-F2-F6-AD-76-4A	0	2024-04-08 23:15:03
---	-------------------	---	---------------------

Sample Output

+-----+-----+	
email	total_scanned_devices
+-----+-----+	
cbracegirdle0@irs.gov 3	
mpaulon2@edublogs.org 2	
+-----+-----+	

Question - 28

SQL: Resource Usage Report for Online Hosting Panel

Create a report showing customers whose websites use more than 50% of any resource (CPU, memory, or disk) for a web hosting provider. It provides a detailed overview of their average resource consumption.

The result should include these columns:

- email* - the email address of the customer
- average_cpu_usage* - the average CPU usage across all sites for each customer, rounded to two decimal places, including trailing zeros if necessary, e.g., 500.00
- average_memory_usage* - the average memory usage across all sites for each customer, rounded to two decimal places, including trailing zeros if necessary, e.g., 500.00
- average_disk_usage* - the average disk usage across all sites for each customer, rounded to two decimal places, including trailing zeros if necessary, e.g., 500.00

The results should be sorted in ascending order by email.

Note:

- Only include customers with at least one average value of a metric (CPU, memory, or disk usage) greater than 50%.

▼ Schema

customers

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the customer
email	VARCHAR(255)		The email address of the customer

site_metrics

Name	Type	Constraints	Description
customer_id	INT	FOREIGN KEY(customer_id => customers.id)	The reference to the customer
cpu_usage	DECIMAL(5,2)		The CPU usage percentage
memory_usage	DECIMAL(5,2)		The memory usage percentage
disk_usage	DECIMAL(5,2)		The disk usage percentage

▼ Sample Data Tables

customers

id	email
1	lrathke0@usa.gov

2	epearsall1@fema.gov
3	sivasechko2@cisco.com

site_metrics			
customer_id	cpu_usage	memory_usage	disk_usage
1	31.53	80.84	1.51
1	12.54	26.47	47.74
1	12.34	46.24	34.43
1	26.64	84.98	17.56
2	80.45	50.05	10.63
2	40.14	86.67	15.98
2	30.14	34.38	17.67
2	1.11	83.44	2.95
3	30.60	18.60	28.02
3	41.64	33.64	5.20
3	31.88	7.37	91.14
3	43.20	9.56	40.40
3	2.33	34.29	18.65
3	11.50	32.89	71.39
3	39.57	4.49	48.05
3	25.06	23.77	33.00
3	32.81	1.59	25.85
3	48.38	79.21	8.31
3	11.62	26.75	71.71
3	54.43	6.48	4.86

Sample Output

email	average_cpu_usage	average_memory_usage	average_disk_usage
epearsall1@fema.gov	37.96	63.64	11.81
lrathke0@usa.gov	20.76	59.63	25.31

Question - 29
SQL: Dashboard Report for Online Hosting Customers Panel

Generate a dashboard report of all customers and the number of active websites they own for an online hosting customers panel.

The result should include these columns:

- *email* - the email address of the customer
- *total_active_sites* - the total number of active websites for each customer

The results should be sorted in ascending order by *email*.

Note:

- Only include customers with active websites.

▼ Schema

customers			
Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the customer
email	VARCHAR(255)		The email address of the customer

sites			
Name	Type	Constraints	Description
customer_id	INT	FOREIGN KEY(customer_id => customers.id)	The reference to the customer
url	VARCHAR(255)		The URL of the website
is_active	BOOLEAN		The status of the website

▼ Sample Data Tables

customers	
id	email
1	dcristofol0@slashdot.org
2	mbillanie1@japanpost.jp
3	hmainz2@utexas.edu

sites		
customer_id	url	is_active
1	https://trellian.com	1
1	https://www.google.de	1
1	https://merriam-webster.com	1
1	https://wordpress.com	1
1	https://nsw.gov.au	1
1	https://www.barnesandnoble.com	1
1	https://www.yahoo.com	1
2	https://cloudflare.com	0
2	https://www.is.gd	1

2	https://www.unesco.org	1
3	https://www.sina.com.cn	0
3	https://xinhuanet.com	1
3	https://cyberchimps.com	1
3	https://ask.com	1
3	https://businessinsider.com	1
3	https://www.dailymail.co.uk	1
3	https://www.guardian.co.uk	1
3	https://www.microsoft.com	1
3	https://www.gizmodo.com	1
3	https://www.163.com	1

Sample Output

```
+-----+-----+
|email          |total_active_sites|
+-----+-----+
|dcristofol0@slashdot.org|7                |
|hmainz2@utexas.edu      |9                |
|mbillanie1@japanpost.jp |2                |
+-----+-----+
```

Question - 30

SQL: Average Income Report in Online Tax Application

Generate a report showing the accounts with the highest average incomes based on income received within a specific timeframe for an online tax application.

The result should include these columns:

- iban* - the IBAN of the account
- average_income* - the average income for each account during the specified period, rounded to two decimal places, including trailing zeros if necessary, e.g., 500.00
- total_income* - the total amount of income for each account during the specified period, with two decimal places, including trailing zeros if necessary, e.g., 500.00

The results should first be sorted in descending order by *average_income*.

Note:

- Only include income recorded in the first quarter of 2024 (from January 1, 2024, to March 31, 2024).
- Limit the results to the top 3 accounts by average income.

▼ Schema

accounts			
Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the account
iban	VARCHAR(255)		The IBAN of the account

Name	Type	Constraints	Description
account_id	INT	FOREIGN KEY(account_id => accounts.id)	The reference to the account
dt	VARCHAR(19)		The date and time of income
amount	DECIMAL(6,2)		The income amount

▼ Sample Data Tables

accounts

id	iban
1	SK39 8924 2092 2997 1101 4161
2	PL28 9141 8610 8442 2367 7521 0000
3	CH93 8418 0F7G KQK4 NEHF Q
4	GT41 TBM8 DPFH MTNS BVW5 D4CX VIRR
5	IT27 Y015 0159 036T W7E5 I6ZD EQZ

income

account_id	dt	amount
1	2024-01-17 16:43:20	4061.53
1	2024-02-28 05:30:15	4488.11
1	2024-04-07 05:41:27	4001.91
2	2023-12-21 07:38:45	4313.69
2	2024-01-08 04:48:45	3640.82
2	2024-01-20 17:31:20	3385.15
3	2024-01-06 23:18:30	2347.15
3	2024-03-08 12:53:20	3814.86
3	2024-04-01 21:18:16	2764.27
4	2024-01-02 23:52:06	3526.08
4	2024-02-04 12:32:28	2221.91
4	2024-02-11 19:44:53	4197.07
4	2024-03-06 06:28:34	1357.44
4	2024-03-16 16:13:49	1854.52
5	2023-12-31 22:08:57	2819.54
5	2024-01-14 18:03:47	2641.20
5	2024-01-23 07:50:22	3692.56
5	2024-02-28 23:43:28	1999.09

5	2024-03-20 10:29:44	1670.18
5	2024-03-27 11:12:04	1193.15

Sample Output

+-----+-----+-----+							+-----+-----+-----+						
iban							average_income total_income						
+-----+-----+-----+							+-----+-----+-----+						
SK39 8924 2092 2997 1101 4161							4274.82 8549.64						
PL28 9141 8610 8442 2367 7521 0000							3512.99 7025.97						
CH93 8418 0F7G KQK4 NEHF Q							3081.01 6162.01						
+-----+-----+-----+							+-----+-----+-----+						

Question - 31

SQL: Tax Calculation for Online Tax Application

Calculate a fixed 20% tax on the total income for each account in an online tax application system.

The result should include these columns:

- iban* - the IBAN of the account
- total_income* - the total income for the account, with two decimal places, including trailing zeros if necessary, e.g., 500.00
- tax_rate* - the tax rate applied, which is a fixed text 20%
- calculated_tax* - the tax calculated, rounded to two decimal places, including trailing zeros if necessary, e.g., 500.00

The results should be sorted in ascending order by *iban*.

Note:

- Only include income in 2023.

▼ Schema

accounts			
Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the account
iban	VARCHAR(255)		The IBAN of the account

income			
Name	Type	Constraints	Description
account_id	INT	FOREIGN KEY(account_id => accounts.id)	The reference to the account
dt	VARCHAR(19)		The date and time of income
amount	DECIMAL(6,2)		The income amount

▼ Sample Data Tables

accounts	
id	iban
1	FR55 4477 6154 73ND TN3F HMOU T36
2	DK46 1272 1831 2573 01

3	RS53 5237 5794 6016 5411 43
---	-----------------------------

income		
account_id	dt	amount
1	2022-12-31 10:03:42	2779.19
1	2023-02-04 08:50:14	1777.68
1	2023-02-13 04:22:07	1954.81
1	2023-03-04 14:46:04	1547.79
1	2023-05-23 15:42:13	1208.49
1	2023-05-24 23:24:07	1521.72
1	2023-07-28 11:01:46	1792.75
1	2023-12-07 14:19:09	2374.25
1	2024-01-27 05:55:36	2803.39
2	2022-12-03 18:04:34	1826.65
2	2023-02-17 00:59:57	3074.11
2	2023-03-01 08:17:15	1007.30
2	2023-08-19 09:16:41	4515.04
2	2024-01-08 04:14:22	3321.78
2	2024-01-10 15:16:28	2033.87
3	2023-05-09 07:28:27	3158.66
3	2023-05-22 04:39:34	3851.20
3	2023-07-21 19:51:14	4152.29
3	2023-10-05 05:42:49	4722.20
3	2023-11-11 02:42:59	1592.16

Sample Output

iban							total_income	tax_rate	calculated_tax
DK46 1272 1831 2573 01							8596.45	20%	1719.29
FR55 4477 6154 73ND TN3F HMOU T36							12177.49	20%	2435.50
RS53 5237 5794 6016 5411 43							17476.51	20%	3495.30

Question - 32

SQL: Monthly Budget Report for Online Budgeting Application

Generate a report for each customer that provides insights into their total expenses and total income for an online budgeting application. This helps customers better manage their finances.

The result should include these columns:

- *email* - the email address of the customer
- *total_expenses* - the sum of all expenses recorded for the customer, showing two decimal places, for example, 500.00
- *total_income* - the sum of income recorded for the customer, showing two decimal places, for example, 500.00

The results should be sorted in ascending order by *email*.

Note:

- Only include expenses and incomes recorded in March.

▼ Schema

customers			
Name	Type	Constraint	Description
id	INT	PRIMARY KEY	The identifier of the customer
email	VARCHAR(255)		The email address of the customer

expenses			
Name	Type	Constraint	Description
customer_id	INT	FOREIGN KEY(customer_id => customers.id)	The reference to the customer
dt	VARCHAR(19)		The date and time of expense
amount	DECIMAL(6,2)		The expense amount

income			
Name	Type	Constraint	Description
customer_id	INT	FOREIGN KEY(customer_id => customers.id)	The reference to the customer
dt	VARCHAR(19)		The date and time of income
amount	DECIMAL(6,2)		The income amount

▼ Sample Data Tables

customers	
id	email
1	otoohey0@elpais.com
2	egrebbin1@state.gov
3	arides2@sohu.com

expenses		
customer_id	dt	amount
1	2024-02-21 22:12:12	90.41
1	2024-02-27 06:48:37	792.88
1	2024-03-10 05:19:43	442.01
1	2024-03-11 19:48:25	327.35

1	2024-03-24 22:03:06	639.62
1	2024-03-29 00:37:46	150.12
1	2024-04-02 03:36:50	257.67
2	2024-02-21 06:11:26	400.22
2	2024-03-11 15:34:19	298.41
2	2024-03-25 04:36:27	376.87
2	2024-03-29 19:05:51	530.07
2	2024-03-30 07:07:28	287.84
2	2024-04-02 15:44:22	868.03
3	2024-03-01 16:02:47	33.30
3	2024-03-06 11:53:42	838.51
3	2024-03-20 23:34:48	968.08
3	2024-03-21 21:18:08	35.36
3	2024-03-30 06:51:13	956.12
3	2024-03-31 10:11:56	896.32
3	2024-03-31 22:36:57	740.94

income		
customer_id	dt	amount
1	2024-02-20 21:00:55	366.66
1	2024-03-11 03:25:04	769.38
1	2024-03-15 00:49:53	84.10
1	2024-03-21 18:32:51	839.48
1	2024-03-29 15:34:13	333.97
1	2024-04-01 00:34:24	253.13
1	2024-04-02 11:13:49	263.56
2	2024-02-20 15:03:26	822.75
2	2024-02-26 14:57:39	277.23
2	2024-03-19 09:24:47	24.08
2	2024-03-20 15:54:24	988.34
2	2024-04-02 08:28:38	990.54
3	2024-02-21 10:23:33	430.82
3	2024-02-29 08:25:32	482.85
3	2024-03-01 05:10:42	962.60

3	2024-03-04 08:27:34	30.21
3	2024-03-19 12:12:01	80.00
3	2024-03-21 00:32:10	674.76
3	2024-03-23 14:14:32	863.79
3	2024-04-09 13:37:07	51.42

Sample Output

```

+-----+-----+-----+
|email          |total_expenses|total_income|
+-----+-----+-----+
|arides2@sohu.com  |4468.63      |2611.36     |
|egrebbin1@state.gov|1493.19      |1012.42     |
|otoohey0@elpais.com|1559.10      |2026.93     |
+-----+-----+-----+

```

Question - 33

SQL: Balance Report for Online Budgeting Application

Create a balance report showing the difference between each customer's total income and total expenses for an online budgeting application. This provides insights into financial health.

The result should include these columns:

- email* - the email address of the customer
- balance* - the difference between the total income and total expenses for the customer, showing two decimal places, for example, 500.00

The results should be sorted in ascending order by *email*.

Note:

- Only include customers who have a negative balance.

▼ Schema

customers			
Name	Type	Constraint	Description
id	INT	PRIMARY KEY	The identifier of the customer
email	VARCHAR(255)		The email address of the customer

expenses			
Name	Type	Constraint	Description
customer_id	INT	FOREIGN KEY(customer_id => customers.id)	The reference to the customer
amount	DECIMAL(6,2)		The expense amount

income			
Name	Type	Constraint	Description
customer_id	INT	FOREIGN KEY(customer_id => customers.id)	The reference to the customer
amount	DECIMAL(6,2)		The income amount

customers

id	email
1	dtollmache0@typepad.com
2	eclutterbuck1@baidu.com
3	mdensun2@ustream.tv

expenses

customer_id	amount
1	136.18
1	323.28
1	383.37
1	505.41
1	841.21
2	5.23
2	408.33
2	489.45
2	545.40
2	591.43
2	706.13
2	716.82
2	761.75
2	796.30
3	152.26
3	211.30
3	447.57
3	685.03
3	966.89
3	967.30

income

customer_id	amount
1	39.44
1	49.49

1	292.19
1	419.36
1	529.26
1	695.43
1	763.72
1	797.92
1	833.34
2	139.42
2	422.18
2	506.59
2	566.00
2	697.92
2	938.51
3	304.66
3	345.03
3	371.86
3	371.88
3	552.08

Sample Output

```
+-----+-----+
|email          |balance |
+-----+-----+
|eclutterbuck1@baidu.com|-1750.22|
|mdensun2@ustream.tv   |-1484.84|
+-----+-----+
```

Question - 34

SQL: Monthly Sales Report

Generate a monthly sales report for the first quarter of the year to track the performance of each product for a sales department. The goal is to identify the top-selling products based on the total sales amount.

The result should include these columns:

- name* - the name of the product
- month* - the month name of the sale, e.g., "January", "February", "March"
- total_sales* - the total sales amount for the product in a specific month, rounded to two decimal places, including trailing zeros if necessary (e.g., 5.00)

The result should be sorted first by *month* in ascending calendar order, then by *total_sales* in descending order within each month.

Note:

- Only consider transactions that occurred in the first quarter of 2024.

products

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the product
name	VARCHAR(255)		The name of the product

sales

Name	Type	Constraints	Description
product_id	INT	FOREIGN KEY(product_id => products.id)	The reference to the product
dt	VARCHAR(19)		The date and time of sale
amount	DECIMAL(7,2)		The amount of the sale

products

id	name
1	Luxury Gold Watch
2	Smartphone Holder Stand
3	Stainless Steel Water Bottle

sales

product_id	dt	amount
1	2024-01-13 17:12:22	7008.16
1	2024-01-03 03:15:27	6191.64
1	2024-01-22 18:29:09	4527.86
1	2024-01-26 19:38:53	7828.36
1	2024-02-17 09:27:13	5273.16
1	2024-02-11 09:51:24	3364.73
1	2024-02-22 23:53:15	8584.33
2	2024-01-28 11:33:58	3710.06
2	2024-01-25 14:47:25	5221.02
2	2024-01-21 07:58:53	2525.72
2	2024-03-15 14:16:18	8158.08
2	2024-03-12 17:02:01	6760.77
3	2024-01-13 19:27:51	1942.79
3	2024-02-15 08:04:40	9186.38
3	2024-03-06 08:02:37	5821.97

3	2024-03-03 15:39:18	8676.24
2	2024-04-08 09:53:01	6327.20
1	2023-12-26 05:48:22	8360.43
1	2023-12-15 14:52:51	9101.30
3	2023-12-21 15:57:50	3857.98

Sample Output

name	month	total_sales
Luxury Gold Watch	January	25556.02
Smartphone Holder Stand	January	11456.80
Stainless Steel Water Bottle	January	1942.79
Luxury Gold Watch	February	17222.22
Stainless Steel Water Bottle	February	9186.38
Smartphone Holder Stand	March	14918.85
Stainless Steel Water Bottle	March	14498.21

Question - 35

SQL: IT Project Resource Analysis

Analyze the allocation of resources across various projects for an IT company. The goal is to understand the distribution of employees among projects, focusing on projects that require more resources due to their complexity or scale.

The result should include these columns:

- project_name* - the name of the project
- employee_count* - the total number of employees assigned to the project
- avg_experience_years* - the average years of experience per employee on the project, rounded up to the nearest integer (the ceiling)
- is_understaffed* - the derived column that shows "Yes" if the employee count is less than 5, otherwise "No"

The result should be sorted in descending order by *employee_count*.

Note:

- The same employee can be assigned to two or more projects.
- The same employee can have two or more assignments to the same project, so the number of employees on a project should be counted as the number of assignments on that project.
- Only include projects with more than 2 years of average experience.

▼ Schema

projects

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the project
name	VARCHAR(255)		The name of the project

employees

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the employee

ein	VARCHAR(255)		The employee identification number
experience_years	INT		Years of experience

projects_employees

Name	Type	Constraints	Description
project_id	INT	FOREIGN KEY(project_id => projects.id)	The reference to the project
employee_id	INT	FOREIGN KEY(employee_id => employees.id)	The reference to the employee

▼ Sample Data Tables

projects

id	name
1	Project X
2	Sunshine Project
3	Blue Sky Initiative

employees

id	ein	experience_years
1	62-0524667	4
2	62-1435366	1
3	29-3144922	1
4	80-9606443	1
5	63-6630813	1

projects_employees

project_id	employee_id
1	1
1	1
1	2
1	3
1	5
2	1
2	1
2	2
2	5
3	1

3	1
3	2
3	3
3	3
3	4
3	4
3	5
3	5
3	5
3	5

Sample Output

project_name	employee_count	avg_experience_years	is_understaffed
Project X	5	3	No
Sunshine Project	4	3	Yes

Question - 36

SQL: Ethereum Market Dashboard Analysis

Analyze Ethereum transactions on a cryptocurrency trading platform. The analysis focuses on understanding the buying and selling activities of Ethereum for each wallet address.

The result should include these columns:

- wallet* - the unique identifier of a wallet on the platform
- total_transactions* - the total number of transactions for the specific wallet address
- total_bought* - the total amount of Ethereum bought by the wallet, rounded to two decimal places, including trailing zeros if necessary (e.g., 5.00)
- total_sold* - the total amount of Ethereum sold by the wallet, rounded to two decimal places, including trailing zeros if necessary (e.g., 5.00)

The result should be sorted in ascending order by *wallet*.

- Note:
- Positive transaction values represent purchases, negative values represent sales.
 - Only include transactions that occurred in February 2024.

▼ Schema

transactions			
Name	Type	Constraints	Description
dt	VARCHAR(19)		The date and time when the transaction occurred
wallet	VARCHAR(255)		The wallet address involved in the transaction
amount	DECIMAL(4,2)		The amount of Ethereum transacted

transactions		
dt	wallet	amount
2024-01-31 13:42:19	0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c	-5.78
2024-01-24 06:07:14	0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c	-2.79
2024-01-25 06:39:19	0x9B8aDc2eFf4cC3DdEe5f6a7B8dE9aC1F	-2.73
2024-01-29 04:37:45	0x9B8aDc2eFf4cC3DdEe5f6a7B8dE9aC1F	4.68
2024-02-28 06:20:04	0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c	-7.36
2024-02-12 07:45:28	0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c	-3.71
2024-02-25 10:49:54	0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c	-3.53
2024-02-03 19:43:00	0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c	4.01
2024-02-14 08:55:30	0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c	8.20
2024-02-16 04:31:26	0x3a4FbC5Df2E1bBfDdE5c4fA7bF8dE7aC1F	-8.96
2024-02-06 23:45:31	0x3a4FbC5Df2E1bBfDdE5c4fA7bF8dE7aC1F	-7.88
2024-02-11 01:00:35	0x3a4FbC5Df2E1bBfDdE5c4fA7bF8dE7aC1F	-7.66
2024-02-25 09:39:01	0x3a4FbC5Df2E1bBfDdE5c4fA7bF8dE7aC1F	-7.45
2024-02-14 04:04:15	0x3a4FbC5Df2E1bBfDdE5c4fA7bF8dE7aC1F	4.17
2024-02-15 11:47:23	0x3a4FbC5Df2E1bBfDdE5c4fA7bF8dE7aC1F	7.56
2024-02-24 14:58:54	0x9B8aDc2eFf4cC3DdEe5f6a7B8dE9aC1F	-1.45
2024-02-18 21:17:24	0x9B8aDc2eFf4cC3DdEe5f6a7B8dE9aC1F	1.05
2024-02-19 11:12:32	0x9B8aDc2eFf4cC3DdEe5f6a7B8dE9aC1F	3.67
2024-03-09 16:52:14	0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c	1.07
2024-03-07 10:29:46	0x9B8aDc2eFf4cC3DdEe5f6a7B8dE9aC1F	7.26

Sample Output

wallet	total_transactions	total_bought	total_sold
0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c	5	12.21	14.60
0x3a4FbC5Df2E1bBfDdE5c4fA7bF8dE7aC1F	6	11.73	31.95
0x9B8aDc2eFf4cC3DdEe5f6a7B8dE9aC1F	3	4.72	1.45

Question - 37

SQL: Employee Leave Tracker

Track the number of leave days taken by employees in a specific year for a human resources department. The goal is to identify employees who have exceeded their annual leave allowance.

The result should include these columns:

- *email* - the email address of the employee
- *leave_days_taken* - the total number of leave days taken by the employee
- *leave_status* - a derived column that shows Within Limit if the total number of leave days taken by the employee is less than or equal to 20, otherwise Exceeded

The result should be sorted in ascending order by *email*.

Note:

- Only consider leave records created in 2023.

▼ Schema

employees

Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the employee
email	VARCHAR(255)		The email address of the employee

leave_records

Name	Type	Constraints	Description
employee_id	INT	FOREIGN KEY(employee_id => employees.id)	The reference to the employee
leave_dt	VARCHAR(19)		The date and time when the leave record was created
days_taken	INT		The number of leave days taken

▼ Sample Data Tables

employees

id	email
1	jquartly0@macromedia.com
2	cchastand1@stanford.edu
3	lpuckrin2@creativecommons.org

leave_records

employee_id	leave_dt	days_taken
1	2022-11-10 11:52:14	4
1	2022-09-07 23:22:46	1
2	2022-11-11 01:47:50	7
3	2022-11-06 23:12:27	7
3	2022-11-17 07:43:18	7
1	2023-05-19 04:40:25	2
1	2023-12-25 16:29:51	7
1	2023-03-12 18:54:29	1
1	2023-08-23 12:33:56	6

2	2023-04-20 04:19:10	5
2	2023-04-28 00:41:50	7
3	2023-06-11 18:49:25	2
3	2023-12-23 15:53:10	7
3	2023-03-13 13:46:16	2
3	2023-10-08 11:57:43	2
3	2023-04-12 07:49:02	4
3	2023-01-17 06:05:35	6
1	2024-02-05 16:01:59	1
1	2024-01-05 22:15:30	7
2	2024-02-21 00:50:11	4

Sample Output

email	leave_days_taken	leave_status
cchastand1@stanford.edu	12	Within Limit
ljquartly0@macromedia.com	16	Within Limit
lpuckrin2@creativecommons.org	23	Exceeded

Question - 38

SQL: Email Platform Engagement Stats

Analyze user engagement for an email marketing platform by generating a report showing the total number of emails sent, opened, and the open rate for each campaign.

The result should include these columns:

- name* - the name of the email campaign
- total_emails_sent* - the total number of emails sent in the campaign
- total_emails_opened* - the total number of emails that were opened in the campaign
- open_rate* - percentage of emails opened out of the total emails sent, calculated as (emails_opened / emails_sent) * 100, rounded to two decimal places, including trailing zeros if necessary (e.g., 5.00)

The result should be sorted in descending order by open_rate, and then alphabetically by *name*.

Note:

- Only include email campaigns with an open rate greater than 50%.

▼ Schema

campaigns			
Name	Type	Constraints	Description
id	INT	PRIMARY KEY	The identifier of the email campaign
name	VARCHAR(255)		The name of the email campaign

Name	Type	Constraints	Description
campaign_id	INT	FOREIGN KEY(campaign_id => campaigns.id)	The reference to the email campaign
emails_sent	INT		The number of emails sent in the email campaign
emails_opened	INT		The number of emails opened in the email campaign

▼ Sample Data Tables

campaigns

id	name
1	SummerSale2021
2	FallPromo
3	WinterWonderland

email_stats

campaign_id	emails_sent	emails_opened
1	1749	775
1	641	423
1	976	598
1	756	121
1	975	107
1	752	367
1	1068	809
1	1046	589
1	1212	939
1	567	214
2	1084	283
2	992	478
2	1505	604
3	899	315
3	742	554
3	1744	917
3	1163	423
3	1501	948
3	736	451
3	537	434

Sample Output

name	total_emails_sent	total_emails_opened	open_rate
WinterWonderland	7322	4042	55.20
SummerSale2021	9742	4942	50.73

Question - 39

SQL: Bond Maturity Analysis

Assess the maturity dates associated with mortgages within mortgage-backed bonds for a team of financial analysts.

The result should include these columns:

- name* - name of the bond
- maturity_dates* - total number of maturity dates for a specific bond
- earliest_maturity* - earliest maturity date for a specific bond
- latest_maturity* - latest maturity date for a specific bond
- avg_days_to_maturity* - average days to maturity for a specific bond, rounded up to the nearest whole number, e.g., 4.01 rounds up to 5

The result should be sorted in ascending order by name.

Note:

- Only include bonds with an average days to maturity greater than 365 days.
- Today is September 13, 2023.
- Use actual days rather than 30-day months.

▼ Schema

bonds			
name	type	constraint	description
id	INT	PRIMARY KEY	Bond ID
name	VARCHAR(255)		Name of the bond

maturities			
name	type	constraint	description
bond_id	INT	FOREIGN KEY(bond_id => bonds.id)	Reference to the bond
maturity	DATE		Maturity date of the bond

▼ Sample Data Tables

bonds

id	name
1	Alpha Mortgage Bond
2	Beta Mortgage Bond
3	Gamma Mortgage Bond

maturities	
bond_id	maturity
1	2024-01-26
1	2024-02-22
1	2024-03-26
1	2024-05-13
1	2024-07-06
1	2024-08-23
1	2024-09-06
1	2024-11-30
1	2024-12-30
1	2025-04-30
1	2025-05-03
2	2024-07-25
2	2024-12-07
3	2023-12-16
3	2024-01-25
3	2024-01-26
3	2024-05-04
3	2024-10-02
3	2024-12-14
3	2025-01-15

▼ Expected Output

name	maturity_dates	earliest_maturity	latest_maturity	avg_days_to_maturity
------	----------------	-------------------	-----------------	----------------------

Beta Mortgage Bond	2	2024-07-25	2024-12-07	384
--------------------	---	------------	------------	-----

Question - 40

SQL: Bond Interest Rate Analysis

Analyze the coupon rates associated with variable-rate mortgage-backed bonds for a finance team.

The result should include these columns:

- name* - name of the bond
- interest_rates* - total number of interest rate records for a specific bond
- lowest_rate* - lowest interest rate for a specific bond with one place after the decimal
- highest_rate* - highest interest rate for a specific bond with one place after the decimal
- avg_rate* - average interest rate for a specific bond, rounded to two decimal places

The results should be sorted in ascending order by *name*.

Note:

- Only include bonds with an average interest rate greater than 3%
- Retain trailing zeros after the decimal as required (e.g., 5.0 or 3.10)

▼ Schema

bonds			
name	type	constraint	description
id	INT	PRIMARY KEY	Bond ID
name	VARCHAR(255)		Name of the bond

interest_rates			
name	type	constraint	description
bond_id	INT	FOREIGN KEY(bond_id => bonds.id)	Reference to the bond
rate	DECIMAL(2,1)		Interest rate of the bond (in percent)

▼ Sample Data Tables

bonds	
id	name
1	Alpha Mortgage Bond
2	Beta Mortgage Bond
3	Gamma Mortgage Bond

interest_rates	
bond_id	rate
1	1.4
1	1.8
1	2.0
1	2.4
1	3.4
1	4.6
1	4.7
1	4.9
2	2.0
2	2.1
2	3.0
2	3.2
2	4.0
3	1.2
3	1.3
3	1.4
3	2.1
3	2.5
3	3.5
3	4.0

▼ Expected Output

name	interest_rates	lowest_rate	highest_rate	avg_rate
Alpha Mortgage Bond	8	1.4	4.9	3.15

Monitor cash flows from various bond mutual funds over time for a financial advisory team. The analysis focuses on the predictability and amounts of cash flows.

The result should include these columns:

- *name* - name of the fund
- *total_payments* - total number of payment amounts for a specific bond
- *min_payment* - smallest payment amount for a specific bond
- *max_payment* - largest payment amount for a specific bond
- *avg_payment* - average payment amount for a specific bond, rounded to two decimal places

The result should be sorted in ascending order by *name*.

Note:

- Only include funds with an average payment greater than 500.00
- Retain trailing zeros after the decimal (e.g., 518.30)

▼ Schema

funds			
name	type	constraint	description
id	INT	PRIMARY KEY	Fund ID
name	VARCHAR(255)		Name of the fund

coupon_payments			
name	type	constraint	description
fund_id	INT	FOREIGN KEY(fund_id => funds.id)	Reference to the fund
payment	DECIMAL(5,2)		Payment amount for the fund

▼ Sample Data Tables

funds	
id	name
1	ABC Growth Fund
2	XYZ Balanced Fund
3	123 Income Fund

coupon_payments	
fund_id	payment
1	111.54

1	285.20
1	310.82
1	445.74
1	835.56
1	904.96
1	948.93
2	106.52
2	458.95
2	467.75
2	513.84
2	690.27
2	734.65
2	736.84
2	770.69
3	175.08
3	241.42
3	361.97
3	414.86
3	873.43

▼ Expected Output

name	total_payments	min_payment	max_payment	avg_payment
ABC Growth Fund	7	111.54	948.93	548.96
XYZ Balanced Fund	8	106.52	770.69	559.94

Question - 42

SQL: Bond Cash Flow Analysis for Bondholders

Calculate the total cash flow expected from bond investments for each bondholder based on coupon payments of the bonds they possess.

The result should include these columns:

- name* - name of the bondholder
- total_cash_flow* - total expected cash flow from all bonds the bondholder possesses with 2 places after the decimal, e.g., 10 shows as 10.00

The result should be sorted in descending order by *total_cash_flow*.

- Note:
- Only include bondholders with a total expected cash flow greater than 10,000.00
 - For each bond, the *total_cash_flow* = *annual_coupon* × *coupons_remaining*

▼ Schema

bondholders			
name	type	constraint	description
id	INT	PRIMARY KEY	Bondholder ID
name	VARCHAR(255)		Name of the bondholder

bonds			
name	type	constraint	description
id	INT	PRIMARY KEY	Bond ID
name	VARCHAR(255)		Name of the bond
annual_coupon	DECIMAL(5,2)		Annual coupon payment
coupons_remaining	INT		Number of remaining coupon payments

bondholders_bonds			
name	type	constraint	description
bondholder_id	INT	FOREIGN KEY(bondholder_id => bondholders.id)	Reference to the bondholder
bond_id	INT	FOREIGN KEY(bond_id => bonds.id)	Reference to the bond

▼ Sample Data Tables

bondholders	
id	name
1	Alex Smith
2	Taylor Johnson
3	Jordan Davis

bonds			
id	name	annual_coupon	coupons_remaining

1	Golden Bonds	150.00	4
2	Silver Lining	200.00	2
3	Diamond Trust	100.00	4
4	Emerald Wealth	350.00	5
5	Ruby Returns	150.00	8
6	Sapphire Security	450.00	5
7	Amber Assurance	100.00	8
8	Topaz Treasury	100.00	2
9	Opal Opportunities	150.00	5
10	Pearl Prosperity	450.00	5
11	Platinum Promise	450.00	9
12	Jade Investments	350.00	1
13	Garnet Growth	150.00	4
14	Onyx Returns	350.00	2
15	Quartz Capital	100.00	2
16	Citrine Securities	250.00	2
17	Aquamarine Assets	250.00	2
18	Peridot Portfolio	300.00	8
19	Tourmaline Trust	100.00	6
20	Moonstone Money	150.00	9

bondholders_bonds	
bondholder_id	bond_id
1	1
1	2
1	6
1	8
1	9
1	13
1	14
1	16

1	17
2	4
2	5
2	7
2	11
2	15
2	18
3	3
3	10
3	12
3	19
3	20

▼ Expected Output

name	total_cash_flow
Taylor Johnson	10400.00

Question - 43

SQL: Rental Property Cash Flow Analysis

Analyze the cash flows for various rental properties held by a property management company. Create a query that provides statistics for each property.

The result should include these columns:

- *name* - name of the property
- *cash_flows* - total number of cash flow records for the property
- *min_cash_flow* - minimum cash flow for the property
- *max_cash_flow* - maximum cash flow for the property
- *avg_cash_flow* - average cash flow for the property, rounded to two decimal places

The result should be sorted in ascending order by *name*.

Note:

- Only include properties with an average cash flow greater than 10,000.
- Retain trailing zeros after the decimal (e.g., 10279.20).

▼ Schema

properties

name	type	constraint	description
id	INT	PRIMARY KEY	Property ID
name	VARCHAR(255)		Name of the property

cash_flows			
name	type	constraint	description
property_id	INT	FOREIGN KEY(property_id => properties.id)	Reference to the property
amount	INT		Cash flow amount for the property

▼ Sample Data Tables

properties	
id	name
1	Cozy Cottage
2	Sunset Villa
3	Harmony Haven

cash_flows	
property_id	amount
1	3762
1	4513
1	5829
1	7340
1	8014
1	15213
2	2318
2	5917
2	6343
2	8158
3	1065
3	1575
3	6417

3	7669
3	11604
3	11644
3	11739
3	12686
3	18495
3	19898

▼ Expected Output

name	cash_flows	min_cash_flow	max_cash_flow	avg_cash_flow
Harmony Haven	10	1065	19898	10279.20

Question - 44

SQL: Sum of the Cash Flows Analysis

Analyze the cash flows from bond investments for a financial analysis firm. Create a query to extract insights on cash flows per investor.

The result should include these columns:

- *email* - investor email
- *investments* - total number of investments by a specific investor
- *min_cash_flow* - minimum cash flow from investments for a specific investor
- *max_cash_flow* - maximum cash flow from investments for a specific investor
- *avg_cash_flow* - average cash flow from investments for a specific investor, rounded to two decimal places

The result should be sorted in ascending order by *email*.

Note:

- Only include investors who have a total cash flow greater than 1,000,000
- Include trailing zeros after the decimal (e.g., 10.00)

▼ Schema

investors			
name	type	constraint	description
id	INT	PRIMARY KEY	Investor ID
email	VARCHAR(255)	UNIQUE	Email address

cash_flows

name	type	constraint	description
investor_id	INT	FOREIGN KEY(investor_id => investors.id)	Reference to the investor
cash_flow	DECIMAL(8,2)		Cash flow amount

▼ Sample Data Tables

investors	
id	email
1	ematson0@ebay.co.uk
2	lsalvadore1@msn.com
3	aclowser2@patch.com

cash_flows	
investor_id	cash_flow
1	184040.12
1	179280.08
1	179374.42
1	79302.21
1	87466.20
1	194588.36
1	153563.92
1	56377.92
2	59039.14
2	167247.23
2	59311.03
2	183883.00
2	118851.21
3	58868.62
3	96909.26
3	103735.73
3	171261.97

3	86463.11
3	56931.73
3	194699.58

▼ Expected Output

email	investments	min_cash_flow	max_cash_flow	avg_cash_flow
ematson0@ebay.co.uk	8	56377.92	194588.36	139249.15

Question - 45

SQL: Sum of Interest Cash Flows for Bond Portfolios

Analyze the interest cash flows generated from bond portfolios held by various investors. Create a query that provides insights into interest cash flows per investor.

The result should include these columns:

- *email* - investor email
- *bonds* - total number of bonds portfolios held by a specific investor, each record is a different bond
- *min_interest_flow* - minimum interest cash flow from the portfolios for a specific investor
- *max_interest_flow* - maximum interest cash flow from the portfolios for a specific investor
- *avg_interest_flow* - average interest cash flow from the portfolios for a specific investor, rounded to two decimal places

The results should be sorted in ascending order by *email*.

Note:

- Only include investors with a total interest cash flow exceeding 500,000
- Include trailing zeros after the decimal (e.g., 10.00)

▼ Schema

investors			
name	type	constraint	description
id	INT	PRIMARY KEY	Investor ID
email	VARCHAR(255)	UNIQUE	Email address

portfolios			
name	type	constraint	description
investor_id	INT	FOREIGN KEY(investor_id => investors.id)	Reference to the investor
interest_flow	DECIMAL(8,2)		Interest cash flow amount

investors	
id	email
1	lmolohan0@thetimes.co.uk
2	fhenighan1@trellian.com
3	rcomelli2@163.com

portfolios	
investor_id	interest_flow
1	54186.29
1	44161.23
1	35996.29
1	60679.23
1	84005.06
1	89919.90
1	18196.64
1	82522.77
1	85218.41
2	85992.30
2	97415.68
2	56353.27
2	21220.40
2	57609.31
3	66257.43
3	40742.78
3	11887.39
3	32262.87
3	37662.91
3	13725.84

▼ Expected Output

email	bonds	min_interest_flow	max_interest_flow	avg_interest_flow
lmolohan0@thetimes.co.uk	9	18196.64	89919.90	61653.98

Question - 46

SQL: Expected Cash Flow Analysis

Create a query for a financial analytics platform that analyzes expected cash flows from bond investments. The query should return:

- *email* - Investor email
- *investment_count* - Total number of investments
- *total_expected_flow* - Sum of all expected cash flows
- *range_expected_flow* - Range of expected cash flows (maximum minus minimum)

Only investors with *total_expected_flow* greater than 100,000 should be included. Results should be sorted in ascending order by *email*.

▼ Schema

investors			
name	type	constraint	description
id	INT	PRIMARY KEY	Investor ID
email	VARCHAR(255)	UNIQUE	Email address

cash_flows			
name	type	constraint	description
investor_id	INT	FOREIGN KEY(investor_id => investors.id)	Reference to the investor
expected_flow	DECIMAL(8,2)		Expected cash flow amount

▼ Sample Data Tables

investors	
id	email
1	tdowner0@timesonline.co.uk
2	cgarza1@opera.com
3	nbarwise2@si.edu

cash_flows	
investor_id	expected_flow
1	24923.83
1	30212.10
1	87126.50
1	56018.65
1	93357.47
1	55073.54
1	27095.07
2	44165.12
2	43658.84
2	35835.34
2	12660.46
2	58676.60
2	95929.25
2	47161.23
2	80283.91
2	54427.20
2	93223.98
3	19741.35
3	12243.25
3	50470.06

▼ Expected Output

email	investment_count	total_expected_flow	range_expected_flow
cgarza1@opera.com	10	566021.93	83268.79
tdowner0@timesonline.co.uk	7	373807.16	68433.64

Question - 47

SQL: Present Value of Future Cash Flows

Create a query for a financial services company that calculates the present value of zero-coupon bonds using a 5% discount rate. The query should return:

- *email* - Investor email
- *investment_count* - Total number of investments
- *total_present_value* - Sum of present values of expected cash flows, rounded to two decimal places, e.g., 1000.00
- *avg_present_value* - Average present value of expected cash flows, rounded to two decimal places, e.g., 1000.00

The results should be sorted in ascending order by *email*.

Note:

- To calculate the present value, use the formula: $present_value = \left(\frac{expected_flow}{(1 + discount_rate)^{periods}} \right)$. For example, the expected cash flow is 105 after 1 period. The present value is $present_value = \left(\frac{105}{(1 + 0.05)^1} \right) = 100$.
- Use a *discount_rate* of 0.05 and assume each cash flow has a different period ranging from 1 to the number of cash flows per investor.
- Trailing zeros after the decimal should be included.
- Only investors who have a *total_present_value* greater than 1,000,000 should be included in the result.

▼ Schema

investors			
name	type	constraint	description
id	INT	PRIMARY KEY	Investor ID
email	VARCHAR(255)	UNIQUE	Email address

cash_flows			
name	type	constraint	description
investor_id	INT	FOREIGN KEY(investor_id => investors.id)	Reference to the investor
expected_flow	DECIMAL(8,2)		Expected cash flow amount
cash_flow_period	INT		The period of the cash flow

▼ Sample Data Tables

investors	
id	email
1	bdunkerley0@ustream.tv
2	amaclaine1@oracle.com
3	mhenstone2@csmonitor.com

cash_flows

investor_id	expected_flow	cash_flow_period
1	467424.93	4
1	473197.72	2
1	325032.12	3
1	384188.89	1
1	89190.85	3
1	306760.94	2
1	96429.87	4
1	67530.35	5
1	173542.87	4
1	230858.63	2
2	225836.56	5
2	492465.14	3
2	242523.52	2
2	75880.55	2
2	436683.99	4
3	120612.96	4
3	249090.59	3
3	163647.03	1
3	69478.04	5
3	419617.32	2

▼ Expected Output

email	investment_count	total_present_value	avg_present_value
amaclaine1@oracle.com	5	1250421.63	250084.33
bdunkerley0@ustream.tv	10	2300127.32	230012.73

Question - 48

SQL: Bond Valuation Analysis

Create a query for a financial services company that analyzes customer bond holdings. The query should return:

- *email* - Investor email
- *investments* - Total number of investments
- *min_valuation* - Minimum bond valuation
- *max_valuation* - Maximum bond valuation
- *avg_valuation* - Average bond valuation, rounded to two decimal places, e.g., 5.00

Only investors with an average bond valuation greater than 100,000 should be included. All valuation amounts should have two decimal places. Results should be sorted in ascending order by *email*.

▼ Schema

investors			
name	type	constraint	description
id	INT	PRIMARY KEY	Investor ID
email	VARCHAR(255)	UNIQUE	Email address

investments			
name	type	constraint	description
investor_id	INT	FOREIGN KEY(investor_id => investors.id)	Reference to the investor
valuation	DECIMAL(8,2)		Bond's market value

▼ Sample Data Tables

investors	
id	email
1	gcordel0@t.co
3	mpolycote2@is.gd
2	tdye1@edublogs.org

investments	
investor_id	valuation
1	136397.66
1	100088.28
1	71534.17
1	117418.73
1	30466.29

1	164102.40
2	112209.07
2	57327.83
2	89859.84
2	45081.15
3	66453.81
3	64627.86
3	156862.58
3	175030.76
3	61761.07
3	31505.41
3	150005.29
3	126345.36
3	151254.58
3	153121.53

▼ Expected Output

email	investments	min_valuation	max_valuation	avg_valuation
gcordel0@t.co	6	30466.29	164102.40	103334.59
mpolycote2@is.gd	10	31505.41	175030.76	113696.83

Question - 49

SQL: Online Store Coupon Codes Report

Create a query for an online store that analyzes coupon code usage. The query should return:

- *coupon_code*
- *description*
- *total_uses* - total number of times the coupon was used
- *min_discount* - minimum discount amount given with the coupon
- *max_discount* - maximum discount amount given with the coupon
- *avg_discount* - average discount amount, rounded to two decimal places, e.eg., 5.00

Only active coupons should be included. Results should be sorted in ascending order by *coupon_code*.

▼ Schema

coupons			
name	type	constraint	description
id	INT	PRIMARY KEY	Coupon Code ID
coupon_code	VARCHAR(255)	UNIQUE	Unique code of the coupon
description	VARCHAR(255)		Description of the coupon code
is_enabled	SMALLINT		Coupon enable flag

coupon_uses			
name	type	constraint	description
coupon_id	INT	FOREIGN KEY(coupon_id => coupons.id)	Reference to the coupon
amount	DECIMAL(4,2)		Discount amount given using the coupon

▼ Sample Data Tables

coupons			
id	coupon_code	description	is_enabled
1	COUPON123	nisi nam ultrices libero non	0
2	SAVE20	ac est lacinia	1
3	DISCOUNT50	quis odio consequat	1

coupon_uses	
coupon_id	amount
1	36.68
1	3.56
1	2.10
1	39.58
2	39.81
2	24.07
2	28.42
2	31.03
2	3.24
2	36.33

3	8.89
3	30.44
3	36.94
3	42.65
3	33.61
3	41.92
3	1.78
3	20.26
3	27.92
3	0.23

▼ Expected Output

coupon_code	description	total_uses	min_discount	max_discount	avg_discount
DISCOUNT50	quis odio consequat	10	0.23	42.65	24.46
SAVE20	ac est lacinia	6	3.24	39.81	27.15

Question - 50

SQL: Game Streaming Service Top Streamers Report

Create a query for a game streaming platform that identifies top streamers.

The query should return: *username* / *total_streams* / *avg_viewers*.

- username* - the streamer username
- total_streams* - number of streams viewed
- avg_viewers* - average number of viewers per stream, rounded to the nearest integer

The result should be sorted in descending order by *avg_viewers*.

- Note:**
- Only streamers who have streamed more than 5 times should be included in the report.
 - Only streams with an average viewer count above 5,000 should be included in the report.

▼ Schema

streamers			
name	type	constraint	description
id	INT	PRIMARY KEY	Streamer ID

username	VARCHAR(255)	UNIQUE	Streamer username
----------	--------------	--------	-------------------

completed_streams			
name	type	constraint	description
id	INT	PRIMARY KEY	Stream ID
streamer_id	INT	FOREIGN KEY(streamer_id => streamers.id)	Reference to the streamer
viewers	INT		Number of viewers during the stream

▼ Sample Data Tables

streamers	
id	username
1	Gamer123
2	StreamMaster
3	ProGaming
4	NinjaPlayer

completed_streams		
id	streamer_id	viewers
3	4	9428
17	4	8643
15	4	6978
19	4	5762
18	4	5493
5	4	4225
13	4	2242
7	3	3937
14	3	3575
2	3	930
6	2	9275
9	2	6611

20	2	6576
1	2	5958
12	2	4380
16	2	1931
8	2	1213
4	2	489
10	1	4497
11	1	3221

▼ Expected Output

username	total_streams	avg_viewers
NinjaPlayer	7	6110

Question - 51

SQL: Freelancer Platform Yearly Income Report

Create a query for a freelancer platform that analyzes income by profession.

The query should return: *title / total_projects / total_income / total_freelancers / average_income_per_freelancer*.

- title* - the profession title (e.g., Web Development, Graphic Design)
- total_projects* - the number of projects in each profession
- total_income* - the income the platform earned from each profession
- total_freelancers* - the number of freelancers in each profession
- average_income_per_freelancer* - Average income per freelancer, rounded to two decimal places, e.g., 5.00

Only completed projects should be included. Results should be sorted in descending order by total income.

▼ Schema

professions			
name	type	constraint	description
id	INT	PRIMARY KEY	Profession ID
title	VARCHAR(255)	UNIQUE	Title of profession (e.g., Web Development)

freelancers			
name	type	constraint	description

id	INT	PRIMARY KEY	Freelancer ID
profession_id	INT	FOREIGN KEY(profession_id => professions.id)	Reference to the profession
email	VARCHAR(255)	UNIQUE	Email of the freelancer

projects			
name	type	constraint	description
id	INT	PRIMARY KEY	Project ID
freelancer_id	INT	FOREIGN KEY(freelancer_id => freelancers.id)	Reference to the freelancer
status	ENUM('Completed','Ongoing','Cancelled')		Status of the project
income	DECIMAL(6,2)		Income earned by the platform for the project

▼ Sample Data Tables

professions	
id	title
1	Artificial Intelligence Engineer
3	Game Developer
2	Network Administrator

freelancers		
id	profession_id	email
1	1	lfernandez0@microsoft.com
3	2	mbrydone2@delicious.com
4	2	jhamp3@4shared.com
5	3	cparfett4@twitter.com

projects			
id	freelancer_id	status	income
5	1	Completed	8562.13
11	1	Completed	6727.56
10	3	Completed	3753.46
20	3	Completed	6659.39

6	4	Completed	8459.28
13	4	Completed	5899.31
16	4	Completed	2709.63
4	5	Completed	5029.44
7	5	Completed	1763.94
9	5	Completed	6988.36
8	3	Cancelled	8699.67
1	5	Cancelled	5403.21
19	3	Ongoing	72.51
3	4	Ongoing	8561.14
15	4	Ongoing	9235.78
17	4	Ongoing	4307.76

▼ Expected Output

title	total_projects	total_income	total_freelancers	average_income_per_freelancer
Network Administrator	5	27481.07	2	5496.21
Artificial Intelligence Engineer	2	15289.69	1	7644.85
Game Developer	3	13781.74	1	4593.91

Question - 52

SQL: Ecommerce Warehouse Stock Report

Create a query for an e-commerce website admin panel that displays product stock status.

The query should return: *category* / *title* / *total_stock*.

- category* - the name of the product category (e.g., Electronics, Clothing)
- title* - the title of the product
- total_stock* - total number of items available in stock for that product

Only products with more than 10 units in stock should be included. Results should be sorted in ascending order by *category*, then by *title*, and finally in descending order by *total_stock*.

▼ Schema

categories

name	type	constraint	description
id	INT	PRIMARY KEY	Category ID
title	VARCHAR(255)	UNIQUE	Title of the product category

products			
name	type	constraint	description
id	INT	PRIMARY KEY	Product ID
category_id	INT	FOREIGN KEY(category_id => categories.id)	Category ID reference
title	VARCHAR(255)		Title of the product
sku	VARCHAR(255)	UNIQUE	SKU of the product
stock_number	INT		Number of items available in stock

▼ Sample Data Tables

categories	
id	title
1	Electronics
2	Clothing
3	Home & Kitchen

products				
id	category_id	title	sku	stock_number
11	1	Elegant Gadget	EG-11	4
3	1	Luxury Gizmo	LG-3	10
19	1	Sleek Widget	SW-19	8
8	1	Sleek Widget	SW-8	8
14	2	Elegant Gadget	EG-14	2
16	2	Elegant Gadget	EG-16	6
10	2	Elegant Gadget	EG-10	10
7	2	Luxury Gizmo	LG-7	3
2	2	Luxury Gizmo	LG-2	8
18	2	Luxury Gizmo	LG-18	9

1	2	Sleek Widget	SW-1	3
6	2	Sleek Widget	SW-6	7
20	3	Elegant Gadget	EG-20	10
9	3	Luxury Gizmo	LG-9	4
12	3	Luxury Gizmo	LG-12	5
13	3	Luxury Gizmo	LG-13	5
5	3	Luxury Gizmo	LG-5	9
4	3	Sleek Widget	SW-4	8
15	3	Sleek Widget	SW-15	9
17	3	Sleek Widget	SW-17	9

▼ Expected Output

category	title	total_stock
Clothing	Elegant Gadget	18
Clothing	Luxury Gizmo	20
Electronics	Sleek Widget	16
Home & Kitchen	Luxury Gizmo	23
Home & Kitchen	Sleek Widget	26

Question - 53

SQL: Antivirus Database Quarantine Report

Create a query for a cyber-security company that analyzes quarantined URLs. The query should return:

- domain_name*
- threat_identified* - Type of threat identified
- total_occurrences* - Total number of times the domain was quarantined for that threat
- total_users_affected* - Total number of users affected by the threat from that domain

Only URLs with status "Quarantined" should be included. Results should be sorted in descending order by users affected, then in ascending order by domain name.

▼ Schema

threat_types			
name	type	constraint	description
id	INT	PRIMARY KEY	Threat type ID

threat_type	VARCHAR(255)		Type of threat (e.g., Malware, Phishing)
-------------	--------------	--	--

quarantine_urls			
name	type	constraint	description
id	INT	PRIMARY KEY	URL ID
threat_id	INT	FOREIGN KEY(threat_id => threat_types.id)	Reference to the type of threat
domain_name	VARCHAR(255)		Domain name of the quarantined URL
status	ENUM('Quarantined','Safe','Deleted')		URL status in the system
users_affected	INT		Number of users affected by the quarantined URL

▼ Sample Data Tables

threat_types	
id	threat_type
1	Phishing
2	Rootkit
3	Malware

quarantine_urls				
id	threat_id	domain_name	status	users_affected
17	1	amazon.com	Quarantined	862
16	1	google.com	Quarantined	63
9	1	amazon.com	Quarantined	41
18	2	amazon.com	Quarantined	149
12	2	yahoo.com	Quarantined	967
4	3	amazon.com	Quarantined	377
10	3	yahoo.com	Quarantined	721
11	1	yahoo.com	Deleted	551
20	1	amazon.com	Safe	407
19	1	amazon.com	Deleted	665
15	1	facebook.com	Safe	52
2	1	google.com	Safe	309

1	2	twitter.com	Safe	562
13	2	facebook.com	Safe	208
14	2	google.com	Deleted	731
8	2	twitter.com	Safe	924
7	2	twitter.com	Safe	982
6	2	google.com	Deleted	864
3	2	facebook.com	Safe	136
5	3	yahoo.com	Safe	949

▼ Expected Output

domain_name	threat_type	total_occurrences	total_users_affected
yahoo.com	Rootkit	1	967
amazon.com	Phishing	2	903
yahoo.com	Malware	1	721
amazon.com	Malware	1	377
amazon.com	Rootkit	1	149
google.com	Phishing	1	63

Question - 54

SQL: Online Streaming Service Traffic Report

Create a query for an online streaming service that analyzes client streaming data. The query should return:

- mac* - Client MAC address
- streams* - Total number of streams
- total_traffic* - Traffic amount across all streams

Only streams with quality "720p" or higher should be included. Results should be sorted in descending order by total traffic.

▼ Schema

clients			
name	type	constraint	description
id	INT	PRIMARY KEY	Client ID
mac_address	VARCHAR(255)		MAC address

streams			
name	type	constraint	description
client_id	INT	FOREIGN KEY (client_id => clients.id)	Client ID
title	VARCHAR(255)		Title
quality	ENUM('240p','360p','480p','720p','1080p','1440p','2160p')		Quality
traffic	INT		Traffic

▼ Sample Data Tables

clients	
id	mac_address
1	2F-80-8E-F2-0E-4C
2	A1-F7-D4-48-B9-E6
3	9F-72-DB-7C-73-FC

streams			
client_id	title	quality	traffic
1	Monte Carlo	360p	71928308
1	Separation, The (Sparation, La)	480p	35221785
1	Felidae	480p	54617023
1	Dirty Dancing	1440p	56419563
1	Ragtime	1440p	12404457
1	Oscar	1440p	49717246
1	Barb Wire	2160p	83761463
1	Jason and the Argonauts	2160p	27364051
2	Carry on Cruising	240p	33226462
2	Best of the Best	240p	62793858
2	Ecstasy (xtasis)	240p	73079415
2	Go Go Tales	480p	48836837
2	Nights and Weekends	1440p	32708277
3	Coneheads	480p	92308213

3	Silences of the Palace, The (Saimt el Qusur)	480p	52917945
3	Good Pick	720p	71890218
3	Wuthering Heights	720p	19813053
3	Big Kahuna, The	1080p	28786846
3	Work of Director Michel Gondry, The	2160p	18789351
3	My Best Friends	2160p	44347338

▼ Expected Output

mac_address	streams	total_traffic
2F-80-8E-F2-0E-4C	5	229666780
9F-72-DB-7C-73-FC	5	183626806
A1-F7-D4-48-B9-E6	1	32708277

Question - 55

SQL: Cloud Hosting Instances Performance Statistics

Create a query for a cloud hosting provider dashboard that identifies networks with high CPU usage instances. The query should return:

- cidr* -Network CIDR
- instances* - Total number of instances in the network
- avg_cpu_usage* - Average CPU usage across all instances (formatted as a percent)
- avg_memory_usage* - Average memory usage across all instances (formatted as a percent)
- avg_network_usage* - Average network usage across all instances (formatted as a percent)

Only networks that have at least one instance with CPU usage of 80% or higher should be included. All averages should be rounded up to the nearest integer (ceiling). Results should be sorted in ascending order by CIDR.

▼ Schema

networks			
name	type	constraint	description
id	INT	PRIMARY KEY	Network ID
cidr	VARCHAR(255)		IP Address v4 CIDR

instances			
name	type	constraint	description
network_id	INT	FOREIGN KEY (network_id => networks.id)	Network ID

cpu_usage	VARCHAR(255)	CPU usage
memory_usage	VARCHAR(255)	Memory usage
network_usage	VARCHAR(255)	Network usage

▼ Sample Data Tables

networks	
id	cidr
1	24.77.36.156/9
2	74.213.138.70/7
3	167.244.163.58/29

instances			
network_id	cpu_usage	memory_usage	network_usage
1	20%	74%	74%
3	26%	9%	99%
3	2%	21%	97%
1	51%	19%	89%
2	2%	27%	79%
3	92%	35%	41%
2	27%	5%	44%
3	67%	47%	79%
1	14%	28%	43%
3	47%	0%	53%
1	38%	3%	46%
2	71%	51%	6%
3	77%	74%	53%
3	31%	48%	80%
2	31%	42%	24%
1	77%	65%	46%
2	51%	94%	41%

3	8%	3%	57%
1	1%	56%	62%
2	15%	66%	65%

▼ Expected Output

cidr	instances	avg_cpu_usage	avg_memory_usage	avg_network_usage
167.244.163.58/29	8	44%	30%	70%

Question - 56

SQL: AI Video Processing Service Usage Time Calculation

Create a query for a video processing service that measures the total execution time for task processes.

The query should return: *hash / usage_time*

- hash* - Task hash
- usage_time* - Total execution time in seconds across all processes for each task

Results should be sorted in descending order by total *usage_time*.

▼ Schema

tasks			
name	type	constraint	description
id	INT	PRIMARY KEY	Task ID
hash	VARCHAR(255)		Hash

processes			
name	type	constraint	description
task_id	INT	FOREIGN KEY (task_id => tasks.id)	Task ID
start_dt	VARCHAR(19)		Start timestamp
end_dt	VARCHAR(19)		End timestamp

▼ Sample Data Tables

tasks	
id	hash

1	208f95e0fcff792f617ade3cebf33ad9
2	0f44a9ffead2f18a7f25425c1260fc74
3	dbcf54e94395c32e01ec09a5db731912

processes		
task_id	start_dt	end_dt
1	2023-04-20 02:01:16	2023-04-20 02:11:35
1	2023-04-09 15:11:10	2023-04-09 15:26:43
1	2023-04-07 23:41:49	2023-04-08 00:34:10
2	2023-04-07 23:05:47	2023-04-08 00:00:05
2	2023-04-19 18:39:33	2023-04-19 18:54:57
2	2023-04-28 13:17:11	2023-04-28 13:24:37
2	2023-04-16 00:13:06	2023-04-16 01:02:39
2	2023-04-16 15:02:26	2023-04-16 15:58:14
2	2023-04-27 02:23:07	2023-04-27 02:59:13
2	2023-04-10 23:33:47	2023-04-11 00:09:35
2	2023-04-16 17:29:51	2023-04-16 18:10:22
2	2023-04-23 12:16:01	2023-04-23 12:48:07
3	2023-04-01 02:25:12	2023-04-01 02:49:26
3	2023-04-04 03:02:43	2023-04-04 03:42:03
3	2023-04-10 22:42:26	2023-04-10 23:14:42
3	2023-04-09 17:46:12	2023-04-09 18:10:19
3	2023-04-25 15:09:36	2023-04-25 15:19:54
3	2023-04-19 14:39:52	2023-04-19 15:21:23
3	2023-04-12 04:22:29	2023-04-12 04:25:10
3	2023-04-25 07:40:26	2023-04-25 08:01:30

▼ Expected Output

hash	usage_time
0f44a9ffead2f18a7f25425c1260fc74	19620

dbcF54e94395c32e01ec09a5db731912	11731
208f95e0fcff792f617ade3cebf33ad9	4693

Question - 57

SQL: AI Text Rewrite Service Output Stats

Create a query for an AI text rewriting service that calculates statistics on output results.

The query should return: *email* / *outputs* / *min_words* / *max_words* / *avg_words*

- email* - account email
- outputs* - number of outputs generated
- min_words* - minimum word count across all outputs
- max_words* - maximum word count across all outputs
- avg_words* - average word count across all outputs, rounded to two decimal places, e.g., 5.00

Only accounts with an average output of more than 200 words should be included. Results should be sorted in ascending order by *email*.

▼ Schema

accounts			
name	type	constraint	description
id	INT	PRIMARY KEY	Account ID
email	VARCHAR(255)		Email address

results			
name	type	constraint	description
account_id	INT	FOREIGN KEY (account_id => accounts.id)	Account ID
words	INT		Number of words

▼ Sample Data Tables

accounts	
id	email
1	bmacalpyne0@scribd.com
2	adarkins1@hatena.ne.jp
3	zphin2@theguardian.com

results	
account_id	words
1	354
1	120
1	60
1	91
1	95
2	283
2	353
2	316
2	193
2	341
2	111
2	208
2	347
2	136
3	280
3	176
3	355
3	136
3	385
3	210

▼ Expected Output

email	outputs	min_words	max_words	avg_words
adarkins1@hatena.ne.jp	9	111	353	254.22
zphin2@theguardian.com	6	136	385	257.00

Question - 58
SQL: Credentials Manager Salt Validation

Create a query for a credential management platform security audit that identifies accounts using short salt strings in their active encryptions. The query should return:

- Account MAC address
- Number of encryption salts that are too short (length less than 8)

Only active encryptions should be considered. Results should be sorted in ascending order by MAC address.

▼ Schema

accounts			
name	type	constraint	description
id	INT	PRIMARY KEY	Account ID
mac	VARCHAR(255)		MAC address

encryptions			
name	type	constraint	description
account_id	INT	FOREIGN KEY (account_id => accounts.id)	Account ID
salt	VARCHAR(255)		Salt
is_active	BOOLEAN		Activity flag

▼ Sample Data Tables

accounts	
id	mac
1	0C-6B-27-2E-E0-49
2	01-51-06-EC-7C-FB
3	7F-43-FD-22-2E-94

encryptions		
account_id	salt	is_active
1	cc20ad47815	0
1	eb4a0bb0	0
1	3be6c4d2a1644b	0
1	339c8ee8856c28	0
1	27d2075	0

1	960c872e5dc	0
1	531ca7f4d	1
1	4101965	1
1	f2707	1
1	7a1c7adbe686e	1
2	5d98604bbfb	0
2	f617f	0
2	013c49b42bee9	1
3	599f71b43c9	0
3	ea7b576a4b	0
3	99a57e	1
3	be2d70bb850	1
3	d1b48f27ecdba	1
3	0fd088f68	1
3	1bc22ee	1

▼ Expected Output

mac	salts
0C-6B-27-2E-E0-49	2
7F-43-FD-22-2E-94	2

Question - 59

SQL: Payment System Commission Calculation

Create a query for a payment system that calculates the total commission for customer transactions.

The query should return: *email / country / transactions / total_amount / total_commission*.

- email* - customer email address
- country* - customer country
- transactions* - total number of all transactions for a customer
- total_amount* - total amount of all transactions for a customer
- total_commission* - total commission for all transactions for a customer, rounded to two decimal places, e.g., 5.00

The commission is calculated based on the customer's country, where the total commission equals the sum of each transaction amount multiplied by the country's commission rate. Results should be sorted in ascending order by customer *email*.

customers			
name	type	constraint	description
id	INT	PRIMARY KEY	Customer ID
email	VARCHAR(255)		Customer email address
country	VARCHAR(255)	FOREIGN KEY (country => countries.name)	Customer country

countries			
name	type	constraint	description
name	VARCHAR(255)	PRIMARY KEY	Country name
commission	DECIMAL(3,2)		Country commission

transactions			
name	type	constraint	description
customer_id	INT	FOREIGN KEY (customer_id => customers.id)	Customer ID
amount	DECIMAL(5,2)		Transaction amount

customers		
id	email	country
1	lspellward0@ustream.tv	Germany
2	egauvain1@zimbio.com	Belgium
3	bnisius2@miitbeian.gov.cn	France
4	esnare3@netscape.com	Belgium
5	etricker4@google.cn	Other

countries	
name	commission
Austria	0.10
Belgium	0.25

France	0.15
Germany	0.15
Other	0.30
United Kingdom	0.20
United States	0.25

transactions	
customer_id	amount
4	19.98
1	28.51
1	64.46
4	82.06
4	13.75
3	16.99
4	58.54
4	46.50
2	13.96
4	13.31
1	19.66
2	64.51
5	20.60
3	56.76
4	67.45
3	72.56
4	61.12
1	41.41
1	91.28
5	3.93

▼ Expected Output

email	country	transactions	total_amount	total_commission
bnisius2@miitbeian.gov.cn	France	3	146.31	21.95
egauvain1@zimbio.com	Belgium	2	78.47	19.62
esnare3@netscape.com	Belgium	8	362.71	90.68
etricker4@google.cn	Other	2	24.53	7.36
lspellward0@ustream.tv	Germany	5	245.32	36.80

Question - 60

SQL: Shipping Company Packages Report

Create a query for a shipping company control panel that lists customer packages currently in the delivery process. The query should return:

- Customer email address
- Total number of packages for each customer
- Total weight of all packages for each customer

Only packages with status "created", "shipped", or "on hold" should be included. Results should be sorted in descending order by total weight.

▼ Schema

customers			
name	type	constraint	description
id	INT	PRIMARY KEY	Customer ID
email	VARCHAR(255)		Customer email address

packages			
name	type		description
id	INT		Package ID
status	ENUM('created','shipped','delivered','on hold','cancelled')		Package status
weight	DECIMAL(5,2)		Package weight

customers_packages			
name	type	constraint	description
customer_id	INT	FOREIGN KEY (customer_id => customers.id)	Customer ID
package_id	INT	FOREIGN KEY (package_id => packages.id)	Package ID

▼ Sample Data Tables

customers	
id	email
1	nhalifax0@auda.org.au
2	nbrashier1@reference.com
3	aflowerdew2@columbia.edu

packages		
id	status	weight
1	delivered	93.37
2	created	86.16
3	delivered	66.24
4	delivered	78.63
5	on hold	53.33
6	cancelled	53.34
7	created	13.25
8	delivered	17.54
9	shipped	42.80
10	on hold	72.64
11	cancelled	93.14
12	delivered	64.94
13	delivered	58.34
14	shipped	6.92
15	cancelled	58.50
16	delivered	99.20
17	delivered	11.26
18	created	94.78
19	shipped	12.28
20	created	5.16

customers_packages	
customer_id	package_id

1	4
1	6
1	13
2	18
2	7
2	8
2	9
2	15
3	11
3	20
3	19
3	17
3	16
3	14
3	12
3	1
3	10
3	5
3	3
3	2

▼ Expected Output

email	total_packages	total_weight
aflowerdew2@columbia.edu	6	236.49
nbrashier1@reference.com	3	150.83

Question - 61

SQL: Billing Software Report

Create a query for a billing application that retrieves customers with a positive balance. The query should return:

- Customer IBAN number
- Balance amount formatted with 2 decimal places, all are less than or equal to 100

Only customers with a balance greater than zero should be included.
Results should be sorted in descending order by balance amount.

▼ Schema

customers		
name	type	description
id	SMALLINT	ID
iban	VARCHAR(255)	IBAN number

balances		
name	type	description
customer_id	SMALLINT	Customer ID
amount	DECIMAL(5,2)	Amount

▼ Sample Data Tables

customers	
id	iban
1	FR03 8308 7233 40BK 6LHY HTTP 296
2	KZ55 828R LAKG RRTR MGEH
3	DE93 3002 5100 5869 7389 68
4	IL16 2899 3640 4350 6827 897
5	FO73 7625 1325 8044 80
6	PS22 XKTA RRGY CCDN S68S G5YE HRUP E
7	CZ07 8963 5878 7630 6909 2236
8	SE05 1865 4389 1326 4897 5396
9	SA21 31QH BPI3 K6IB WHGV GJTD
10	KZ87 4246 EWPW SLEO 4CP5
11	BH89 IWIF WKDM SQA4 F4SN Z0
12	CR74 4202 2113 4767 9001 2
13	BE23 7638 2407 6334
14	CH69 1970 7OAU 5LSO OJ4T L

15	FR10 4201 1613 36JQ TWTP 76NO 751
16	FR65 6231 0678 121K DPZV WE7M N05
17	MD78 ECGU XQ3T I8LF DR1K 4JRH
18	PT19 6035 6448 2662 1157 2219 8
19	CR95 5363 9756 8790 5129 9
20	HU48 3940 8443 3966 8531 7180 7191

balances	
customer_id	amount
18	97.04
13	85.78
4	79.53
5	67.06
17	36.77
1	14.86
19	5.65
10	4.81
2	-22.86
7	-24.60
15	-29.37
3	-30.95
16	-51.83
20	-61.38
14	-62.09
8	-63.43
12	-63.86
11	-65.31
9	-98.04
6	-98.74

iban	amount
PT19 6035 6448 2662 1157 2219 8	97.04
BE23 7638 2407 6334	85.78
IL16 2899 3640 4350 6827 897	79.53
FO73 7625 1325 8044 80	67.06
MD78 ECGU XQ3T I8LF DR1K 4JRH	36.77
FR03 8308 7233 40BK 6LHY HTTI 296	14.86
CR95 5363 9756 8790 5129 9	5.65
KZ87 4246 EWPW SLEO 4CP5	4.81

Question - 62

SQL: Benchmarking Tool Report

Create a query for a benchmarking tool that evaluates device performance. The query should return a formatted result for each device showing:

- Device ID and its performance class in the format: "Device [ID] has class: [CLASS]"

The performance class is determined by the device score:

- 'A': score ≥ 80
- 'B': $60 \leq \text{score} < 80$
- 'C': $40 \leq \text{score} < 60$
- 'D': $20 \leq \text{score} < 40$
- 'F': score < 20

Results should be sorted in ascending order by device ID.

▼ Schema

devices			
name	type	constraint	description
id	INT	PRIMARY KEY	Device ID
score	INT		Score

▼ Sample Data Tables

devices	
id	score
1	20
2	50

3	50
4	68
5	95

▼ Expected Output

device
Device 1 has class: D
Device 2 has class: C
Device 3 has class: C
Device 4 has class: B
Device 5 has class: A

Question - 63

SQL: Internet Gateway Firewall Rules Report

Create a query for a data center software that identifies Internet gateways meeting specific criteria:

- Gateway has more than one firewall assigned
- The sum of traffic rules across all assigned firewalls is greater than the maximum allowed (100)

Each Internet gateway can have multiple firewalls, but each firewall belongs to only one gateway. The query should return the gateway ID and the total count of rules across all its assigned firewalls.

▼ Schema

gateways			
name	type	constraint	description
id	INT	PRIMARY KEY	Internet gateway ID
firewall_id	VARCHAR(32)	FOREIGN KEY (firewall_id => firewalls.id)	Firewall ID

firewalls			
name	type	constraint	description
id	VARCHAR(32)		Firewall ID
rules_count	INT		Rules count

▼ Sample Data Tables

gateways	
id	firewall_id
1	abc123
2	def456
3	abc456
1	def123
2	def789

firewalls	
id	rules_count
abc123	60
def456	20
abc456	120
abc123	41
def789	70

Expected Output

gateway_id	total_rules_count
1	101

Question - 64

SQL: Fitness Tracker Activity Report

Create a query for a fitness tracker that returns all activities from the current month (September) with a summary of their segments.

The query should include: *name* / *dt* / *segments* / *average_segment_steps* / *total_steps* / *total_calories*.

- name* - activity name
- dt* - activity date and time
- segments* - total number of activity segments
- average_segment_steps* - average number of activity steps, rounded up to the nearest integer.
- total_steps* - total number of activity steps
- total_calories* - total calories burned during activity

The result should be sorted in ascending order by *dt*.

▼ Schema

activities			
name	type	constraint	description
id	INT	PRIMARY KEY	Activity ID
name	VARCHAR(255)		Activity name
dt	DATETIME		Activity date and time

segments			
name	type	constraint	description
activity_id	INT	FOREIGN KEY (activity_id => activities.id)	Activity ID
steps	SMALLINT		Segment steps
calories	SMALLINT		Segment calories

▼ Sample Data Tables

activities		
id	name	dt
1	Running	2022-08-28 00:24:13
2	Hiking	2022-09-14 06:15:50
3	Walking	2022-09-01 15:47:08

segments		
activity_id	steps	calories
1	1308	115
1	1931	98
1	522	112
1	1460	64
1	1598	58
1	1031	63
1	1480	22
1	2243	107
2	1230	35
2	733	25

2	2108	92
2	1831	54
2	1651	79
2	757	66
2	634	94
3	1184	111
3	1968	74
3	1048	104
3	1203	119
3	1441	58

▼ Expected Output

name	dt	segments	average_segment_steps	steps	calories
Walking	2022-09-01 15:47:08	5	1369	6844	466
Hiking	2022-09-14 06:15:50	7	1278	8944	445

Question - 65

SQL: Smart Home Application Customer Report

Create a query for a smart home application that calculates electricity consumption data for invoicing. The query should provide the following information for each account:

- Username
- Email address
- Name of the most expensive tariff consumed
- Total electricity consumption
- Total cost of all consumed electricity, rounded to two decimal places, e.g., 5.00

The total cost is calculated as the sum of all meter readings multiplied by their respective tariff costs. Results should be sorted in ascending order by *username*.

▼ Schema

accounts			
name	type	constraint	description
id	INT	PRIMARY KEY	Account ID
username	VARCHAR(255)		Account username

email	VARCHAR(255)		Account email address
-------	--------------	--	-----------------------

tariffs			
name	type	constraint	description
id	INT	PRIMARY KEY	Tariff ID
name	ENUM('A','B','C','D','E')		Tariff name
cost	DECIMAL(4,3)		Tariff cost/unit

readings			
name	type	constraint	description
account_id	INT	FOREIGN KEY (account_id => accounts.id)	Account ID
tariff_id	INT	FOREIGN KEY (tariff_id => tariffs.id)	Tariff ID
amount	SMALLINT		Units consumed

▼ Sample Data Tables

accounts		
id	username	email
1	hshillabeare0	rcalkin0@sourceforge.net
2	sdandy1	agaule1@businessweek.com
3	sgreiswood2	toppy2@lulu.com

tariffs		
id	name	cost
1	A	0.010
2	B	0.020
3	C	0.050
4	D	0.075
5	E	0.100

readings		
account_id	tariff_id	amount

1	2	54
1	3	19
1	3	37
1	3	89
1	3	119
2	1	12
2	1	44
2	1	81
2	2	60
2	2	164
2	2	199
2	3	79
2	5	186
3	1	31
3	1	59
3	1	77
3	1	95
3	1	110
3	1	125
3	2	31

▼ Expected Output

username	email	highest_tariff	consumption	total_cost
hshillabeare0	rcalkin0@sourceforge.net	C	318	14.28
sdandy1	agaule1@businessweek.com	E	825	32.38
sgreiswood2	toppy2@lulu.com	B	528	5.59

Question - 66

SQL: MMORPG Game Inventory Overload Notification

Generate a report of game accounts with overloaded inventories.

Your result should include:

- Account username
- Email address
- Number of items in inventory
- Total weight of all items

Requirements:

- Only include accounts where the total item weight exceeds 20 (overload threshold).
- Sort results in descending order by total weight, then ascending by *username*.

▼ Schema

accounts			
name	type	constraint	description
id	INT	PRIMARY KEY	Account ID
username	VARCHAR(255)		Account username
email	VARCHAR(255)		Account email address

items			
name	type	constraint	description
id	INT	PRIMARY KEY	Item ID
type	ENUM('sword','shield','armor')		Item type
name	VARCHAR(255)		Item name
weight	SMALLINT		Item weight

accounts_items			
name	type	constraint	description
account_id	INT	FOREIGN KEY (account_id => accounts.id)	Account ID
item_id	INT	FOREIGN KEY (item_id => items.id)	Item ID

▼ Sample Data Tables

accounts		
id	username	email
1	esoane0	alefwich0@nytimes.com
2	jrafter1	bmcniff1@census.gov

3	rcawston2	fnickoll2@flickr.com
---	-----------	--

items			
id	type	name	weight
1	shield	Shield of Asteraceae	3
2	sword	Sword of Cyperaceae	3
3	shield	Shield of Apiaceae	3
4	sword	Sword of Onagraceae	3
5	sword	Sword of Campanulaceae	3

accounts_items	
account_id	item_id
1	2
1	3
1	3
1	4
1	4
1	5
1	5
1	5
2	1
2	1
2	2
2	2
2	2
2	2
2	3
2	3
2	5
3	3

3	4
---	---

▼ Expected Output

username	email	items	total_weight
jrafter1	bmcniff1@census.gov	10	30
esoane0	alefwich0@nytimes.com	8	24

Question - 67
SQL: Outdoor Banner Digital Marketplace Placement Report

In creating a digital marketplace for outdoor banners, a query is required to retrieve details about the banners in each city.

The output should contain the following columns: *city* / *banners* / *min_area* / *avg_area* / *max_area* / *total_area*.

- city* - name of the city
- banners* - total number of banners in a specific city
- min_area* - smallest banner area available in a specific city
- avg_area* - average banner area in a specific city, rounded up to the nearest integer (for instance, ceiling(1.1) = 2)
- max_area* - largest banner area available in a specific city
- total_area* - combined banner area available in a specific city

The results should be ordered in ascending order by city.

Note:

- Calculate banner area as *width* × *height*

▼ Schema

cities			
name	type	constraint	description
id	INT	PRIMARY KEY	City ID
name	VARCHAR(255)		City name

banners			
name	type	constraint	description
city_id	INT	FOREIGN KEY (city_id => cities.id)	City ID
width	SMALLINT		Banner width
height	SMALLINT		Banner height

cities	
id	name
1	Kayu Agung
2	Yangkou
3	Marseille

banners		
city_id	width	height
3	6	20
1	20	14
1	6	17
1	15	6
2	16	8
2	6	7
3	6	9
1	20	16
3	19	14
2	9	17
2	8	12
1	12	16
3	15	14
3	11	7
3	6	14
2	12	7
3	7	20
1	13	6
3	10	13
2	19	15

▼ Expected Output

city	banners	min_area	avg_area	max_area	total_area
Kayu Agung	6	78	177	320	1062
Marseille	8	54	136	266	1081
Yangkou	6	42	132	285	788

Question - 68

SQL: Auction Web Service Lot Statistics

Generate a report of available auction lots and their offer statistics.

Your result should include:

- Lot name
- Number of offers
- Minimum offer amount
- Average offer amount (rounded to two decimal places)
- Maximum offer amount

Requirements:

- Include lots with no offers (showing NULL for min/avg/max).
- Format decimal values to show two places after the decimal, e.g., 5.00.
- Sort results in descending order by number of offers, then ascending by lot name.

▼ Schema

lots			
name	type	constraint	description
id	INT	PRIMARY KEY	Lot ID
name	VARCHAR(255)		Lot name

offers			
name	type	constraint	description
lot_id	INT	FOREIGN KEY (lot_id => lots.id)	Lot ID
amount	DECIMAL(6,2)		Offer amount

▼ Sample Data Tables

lots

id	name
1	Merremia quinquefolia (L.) Hallier f.
2	Plantago maritima L.
3	Hohenbergia antillana Mez
4	Penstemon eriantherus Pursh var. argillosus M.E. Jones

offers	
lot_id	amount
1	510.51
2	703.80
2	181.80
1	38.06
2	368.78
3	91.40
2	413.80
3	157.99
3	885.82
2	863.99
1	307.61
2	120.39
1	771.96
2	801.42
3	871.59
1	541.61
3	477.62
2	303.29
2	612.83
3	464.98

▼ Expected Output

name	offers	min_offer	avg_offer	max_offer
Plantago maritima L.	9	120.39	485.57	863.99
Hohenbergia antillana Mez	6	91.40	491.57	885.82
Merremia quinquefolia (L.) Hallier f.	5	38.06	433.95	771.96
Penstemon eriantherus Pursh var. argillosus M.E. Jones	0	NULL	NULL	NULL

Question - 69

SQL: Tax Calculator Web Service Simple Report

Create a tax calculation report for all accounts.

Your result should include:

- Full name (formatted as "*last_name first_name*")
- Account IBAN
- Total taxable income (income from all quarters Q1 - Q4)
- Individual tax rate (always 10%)
- Tax amount (taxable income multiplied by rate, rounded to two decimal places, e.g., 5.004 → 5.00)

Requirements:

- Sort results in ascending order by full name.

▼ Schema

accounts			
name	type	constraint	description
id	INT	PRIMARY KEY	Account ID
first_name	VARCHAR(255)		Account first name
last_name	VARCHAR(255)		Account last name
iban	VARCHAR(255)		Account IBAN

declarations			
name	type	constraint	description
account_id	INT	FOREIGN KEY (account_id => accounts.id)	Account ID
quarter	ENUM('Q1','Q2','Q3','Q4')		Declaration quarter
income	DECIMAL(7,2)		Declaration taxable income

▼ Sample Data Tables

accounts

id	first_name	last_name	iban
1	Alex	Cantua	IL29 9590 1551 0560 0553 712
2	Chris	Lashmore	AZ54 CNUI 01DR XEXZ ASKY QM4W F8JI
3	Taylor	Blum	HR20 2041 7741 5014 9873 9
4	Robin	Neachell	NL87 PPCD 0429 1849 92
5	Drew	Barbier	FR72 7843 3990 42WM QC8P GNVN F78

declarations		
account_id	quarter	income
1	Q1	49235.67
1	Q2	46653.11
1	Q3	63739.99
1	Q4	43222.54
2	Q1	69743.50
2	Q2	29641.01
2	Q3	97725.49
2	Q4	91481.98
3	Q1	68402.43
3	Q2	12660.12
3	Q3	59601.65
3	Q4	54701.74
4	Q1	55220.27
4	Q2	87752.41
4	Q3	44447.06
4	Q4	45876.26
5	Q1	42511.74
5	Q2	22022.78
5	Q3	88396.81
5	Q4	67252.54

▼ Expected Output

full_name	iban	income	rate	tax
Barbier Drew	FR72 7843 3990 42WM QC8P GNVV F78	220183.87	10%	22018.39
Blum Taylor	HR20 2041 7741 5014 9873 9	195365.94	10%	19536.59
Cantua Alex	IL29 9590 1551 0560 0553 712	202851.31	10%	20285.13
Lashmore Chris	AZ54 CNUI 01DR XEXZ ASKY QM4W F8JI	288591.98	10%	28859.20
Neachell Robin	NL87 PPCD 0429 1849 92	233296.00	10%	23329.60

Question - 70

SQL: Social Network Relationship Statistics

Generate a report of social network profiles and their relationship statistics.

Your result should include:

- Full name (formatted as "*last_name first_name*")
- Email address
- Total number of relationships
- Number of approved relationships
- Number of pending relationships

Requirements:

- A relationship is "approved" when *is_approved* is true.
- A relationship is "pending" when *is_approved* is false.
- Sort results in ascending order by full name.

▼ Schema

profiles			
name	type	constraint	description
id	INT	PRIMARY KEY	Profile ID
first_name	VARCHAR(255)		Profile first name
last_name	VARCHAR(255)		Profile last name
email	VARCHAR(255)		Profile email address

relations			
name	type	constraint	description
profile_id	INT	FOREIGN KEY (profile_id => profiles.id)	Profile ID
related_to	VARCHAR(255)		Relation to
is_approved	BOOLEAN		Relation approval flag

profiles			
id	first_name	last_name	email
1	Shayne	Shilito	sshilito0@ftc.gov
2	Shell	Shade	sshade1@paginegialle.it
3	Nobie	Splain	nsplain2@npr.org

relations		
profile_id	related_to	is_approved
1	cbasinigazzii	1
1	ldevered	1
1	edeniskeb	1
1	cstirland4	1
1	ngooddiea	1
1	alockney7	1
1	jsorrillj	0
1	bnodin3	0
1	dwall2	0
1	folivas1	0
2	ksharland6	0
2	pbarosch8	0
2	smacieja9	0
2	bbrasonf	0
2	dabrahartg	0
3	gaymer5	1
3	rwoolcockse	1
3	egilyott0	1
3	agillionc	0
3	fgribbinh	0

▼ Expected Output

full_name	email	total_relations	approved_relations	pending_relations
Shade Shell	sshade1@paginegialle.it	5	0	5
Shilito Shayne	sshilito0@ftc.gov	10	6	4
Splain Nobie	nsplain2@npr.org	5	3	2

Question - 71
SQL: Online Banking Transactions

Create a transaction report showing account activity for the month of September.

Your result should include:

- Account IBAN
- Number of transactions
- Total transaction amount

Requirements:

- Sort results in descending order by total amount.

▼ Schema

accounts			
name	type	constraint	description
id	INT	PRIMARY KEY	Account ID
iban	VARCHAR(255)		Account IBAN

transactions			
name	type	constraint	description
account_id	INT	FOREIGN KEY (account_id => accounts.id)	Account ID
dt	DATETIME		Transaction date and time
amount	DECIMAL(5,2)		Transaction amount

▼ Sample Data Tables

accounts	
id	iban

1	SE48 2961 2087 8112 2835 6438
2	BE89 2286 5514 4847
3	MU84 HRGV 2047 2584 5774 3195 856J PZ

transactions		
account_id	dt	amount
2	2022-09-25 19:24:50	75.06
2	2022-09-24 03:09:17	41.10
1	2022-09-19 04:13:17	65.85
3	2022-09-30 07:18:29	44.57
1	2022-09-26 01:51:44	98.93
1	2022-08-28 02:51:04	60.42
1	2022-08-25 23:25:54	45.34
2	2022-09-09 11:00:48	11.05
3	2022-08-25 19:37:02	53.61
2	2022-09-23 09:44:05	89.18
1	2022-08-28 19:48:40	47.60
3	2022-09-12 10:28:10	96.40
3	2022-10-03 16:49:51	45.41
2	2022-09-05 16:20:41	46.78
3	2022-10-03 04:51:29	50.81
1	2022-09-10 17:31:44	78.72
2	2022-08-31 21:59:56	61.09
2	2022-09-14 12:52:13	20.36
1	2022-09-28 11:05:21	70.52
3	2022-09-30 09:21:12	48.00

▼ Expected Output

iban	transactions	total
SE48 2961 2087 8112 2835 6438	4	314.02

BE89 2286 5514 4847	6	283.53
MU84 HRGV 2047 2584 5774 3195 856J PZ	3	188.97

Question - 72

SQL: Dialer Application Monthly Call History

Generate a call history report showing contacts and their call counts for the current month (September).

Your result should include:

- Full name (formatted as "*last_name first_name*")
- Phone number
- Total number of calls

Requirements:

- Only include calls from September (current month)
- Sort results in ascending order by the full name.

▼ Schema

contacts			
name	type	constraint	description
id	INT	PRIMARY KEY	Contact ID
first_name	VARCHAR(255)		Contact first name
last_name	VARCHAR(255)		Contact last name
phone	VARCHAR(255)		Contact phone number

calls			
name	type	constraint	description
contact_id	INT	FOREIGN KEY (contact_id => contacts.id)	Contact ID
dt	DATETIME		Call date and time

▼ Sample Data Tables

contacts			
id	first_name	last_name	phone
1	Alex	Kalinsky	+254 (590) 396-8341
2	Sam	Cruz	+93 (476) 737-7603
3	Chris	Opie	+33 (715) 652-9815

calls	
contact_id	dt
2	2022-09-18 05:21:38
2	2022-08-30 06:23:06
1	2022-09-22 22:36:27
3	2022-09-03 06:56:16
3	2022-09-19 19:05:09
2	2022-09-13 09:15:30
1	2022-09-18 22:31:21
3	2022-09-07 04:37:31
3	2022-09-27 12:15:50
1	2022-09-30 09:40:00
3	2022-09-27 23:43:14
1	2022-08-28 07:49:54
2	2022-09-04 02:08:54
1	2022-08-31 04:41:40
3	2022-09-21 00:09:05
3	2022-09-12 11:07:25
2	2022-09-07 14:36:39
1	2022-09-16 12:39:51
2	2022-09-28 07:39:39
1	2022-08-29 16:37:37

▼ Expected Output

full_name	phone	calls
Cruz Sam	+93 (476) 737-7603	5
Kalinsky Alex	+254 (590) 396-8341	4
Opie Chris	+33 (715) 652-9815	7

Create a report showing the total duration of user sessions for each city.

Your result should include:

- City name
- Total session duration (sum of all user session durations in that city)

Requirements:

- Sort results in ascending order by duration.

▼ Schema

There are 3 tables: *CITIES*, *CLIENTS*, and *SESSIONS*.

CITIES		
Name	Type	Description
id	String	The assigned ID to the city presented as 32 character UUID.
name	String	The name of the city.

CLIENTS		
Name	Type	Description
id	String	The assigned ID to the user presented as 32 character UUID.
city_id	String	The id of the city in which this user resides.
name	String	The name of the user.
email	String	The email of the user.

SESSIONS		
Name	Type	Description
id	String	The assigned ID to the session presented as 32 character UUID.
client_id	String	The id of the client who owns this session.
actions	Integer	The number of actions in this session.
duration	Integer	The duration of this session.

▼ Sample Tables

CITIES	
id	name
1	Cooktown
2	South Suzanne

CLIENTS			
id	city_id	name	email

1	2	Robert Delgado	robertdelgado@hotmail.com
2	2	Thomas Williams	thomaswilliams@bradley.org
3	1	Michele Peterson	michelepeterson@hotmail.com
4	1	Bill Wheeler	billwheeler@gmail.com
5	1	David Lloyd	davidlloyd@gmail.com
6	1	Morgan Powers	morganpowers@hansen.biz

SESSIONS			
id	client_id	actions	duration
1	1	21	200
2	3	6	55
3	2	30	230
4	2	16	125
5	2	11	110
6	6	30	285
7	3	18	170
8	1	6	50
9	2	4	40
10	1	10	90
11	6	11	95
12	5	16	140
13	3	24	220
14	6	17	160
15	2	23	205
16	3	11	90
17	6	5	50
18	3	19	180
19	5	22	205
20	4	6	60

South Suzanne 1050
 Cooktown 1710

Explanation

1. In the city *South Suzanne*, there are 2 clients, and the total duration of the sessions of these clients is 1050.
2. In the city *Cooktown*, there are 4 clients, and the total duration of the sessions of these clients is 1710.

Question - 74

SQL: Warehouse Accounting System Customer Report

Generate a warehouse accounting report showing customer companies and their active warehouse statistics.

Your result should include:

- Customer name
- Number of active warehouses
- Volume of smallest active warehouse
- Volume of largest active warehouse
- Total volume of all active warehouses

Requirements:

- Only include active warehouses.
- Sort results in ascending order by customer name.

▼ Schema

customers		
name	type	description
id	SMALLINT	unique ID, primary key
name	VARCHAR(255)	

warehouses		
name	type	description
customer_id	SMALLINT	foreign key customers.id
volume	DECIMAL(5,2)	
is_active	SMALLINT	1 = Active, 0 = Inactive

▼ Sample Data Tables

customers	
id	name
1	Grady Group
2	Yundt, Ritchie, and Mann
3	Feest Inc

warehouses

customer_id	volume	is_active
1	68.41	0
1	95.96	1
1	63.77	1
1	89.41	1
1	17.22	1
1	89.77	1
1	60.25	1
2	46.30	0
2	76.22	0
2	89.33	1
2	50.07	1
2	67.48	1
2	62.28	1
2	40.23	1
2	16.60	1
3	29.29	0
3	30.51	1
3	34.35	1
3	43.64	1
3	45.71	1

▼ Expected Output

name	warehouses	min_volume	max_volume	total_volume
Feest Inc	4	30.51	45.71	154.21
Grady Group	6	17.22	95.96	416.38
Yundt, Ritchie, and Mann	6	16.60	89.33	325.99

Create a billing report for an ISP showing client traffic and costs for May.

Your result should include:

- MAC address
- Total traffic amount
- Total cost (traffic multiplied by client-specific tariff, rounded to two decimal places, e.g., 5.004 → 5.00)

Requirements:

- Only include traffic from May.
- Sort results in descending order by cost.

▼ Schema

clients		
name	type	description
id	SMALLINT	ID
mac	VARCHAR(17)	MAC address
tariff	DECIMAL(6,5)	Tariff per MB

traffic		
name	type	description
client_id	SMALLINT	Client ID
dt	VARCHAR(19)	Reporting date
amount	INT	Amount

▼ Sample Data Tables

clients		
id	mac	tariff
1	A2-53-FC-0C-3E-B4	0.00007
2	DC-80-42-E9-AE-FC	0.00003
3	3F-9B-A9-2A-B1-7B	0.00007
4	D4-6F-E4-AF-47-D5	0.00004
5	B9-65-FC-8E-F0-15	0.00007

traffic		
client_id	dt	amount

1	2022-05-22	9127
1	2022-06-07	62203
1	2022-06-10	88227
2	2022-05-31	99874
2	2022-06-01	78400
2	2022-06-03	61106
2	2022-06-12	20963
2	2022-06-29	98304
2	2022-07-04	6626
3	2022-05-22	8386
3	2022-06-08	22959
3	2022-07-05	52096
3	2022-07-14	70777
4	2022-05-22	93743
5	2022-05-16	84660
5	2022-05-28	63267
5	2022-06-10	80681
5	2022-06-21	55460
5	2022-07-04	91365
5	2022-07-09	23296

▼ Expected Output

mac	traffic	cost
B9-65-FC-8E-F0-15	147927	10.35
D4-6F-E4-AF-47-D5	93743	3.75
DC-80-42-E9-AE-FC	99874	3.00
A2-53-FC-0C-3E-B4	9127	0.64
3F-9B-A9-2A-B1-7B	8386	0.59

Generate a report of Ethereum wallets and their balances based on transaction amounts.

Your result should include:

- Wallet address
- Number of confirmed transactions
- Balance (sum of all confirmed transaction amounts)

Requirements:

- Only include transactions with 10 or more confirmations.
- Only include wallets with positive balances.
- Sort results in descending order by balance.

▼ Schema

wallets		
name	type	description
id	SMALLINT	unique ID, primary key
address	VARCHAR(255)	

transactions		
name	type	description
wallet_id	SMALLINT	foreign key, wallets.id
amount	DECIMAL(6,3)	
confirmations	SMALLINT	number of confirmations

▼ Sample Data Tables

wallets	
id	address
1	0xce7a99fb564f654c1f48e9b687375b868553d7ec
2	0x7a70b14c1f883e5acca29d75413e803e4dab16d4
3	0x5d386db0077b732d273e2c00e5168d1da86b8239
4	0x3b14e880a15bbdbeaacc38c3a1dca182a209e953
5	0xc45a4f90ddb00df4eb7df2cfe6a01dcafd96fbcc

transactions		
wallet_id	amount	confirmations

1	92.254	40
1	91.815	45
2	74.581	11
2	99.270	19
2	-34.418	38
2	-63.751	42
2	-44.135	46
3	7.081	7
3	-3.689	20
3	80.335	27
4	94.665	14
4	86.743	16
4	-22.050	43
5	-30.202	8
5	-52.135	13
5	-87.167	31
5	45.879	32
5	-70.605	37
5	-67.584	37
5	8.219	49

▼ Expected Output

address	transactions	balance
0xce7a99fb564f654c1f48e9b687375b868553d7ec	2	184.069
0x3b14e880a15bbdbeaacc38c3a1dca182a209e953	3	159.358
0x5d386db0077b732d273e2c00e5168d1da86b8239	2	76.646
0x7a70b14c1f883e5acca29d75413e803e4dab16d4	5	31.547

Create a report of companies and their overall ratings based on categories in which they are listed.

Your result should include:

- Company name
- Address
- Phone number
- Overall review rating in format: "X.X (Y categories)" where X.X is the average rating rounded to one decimal place and Y is the number of categories

Requirements:

- Calculate the average review rating across all categories for each company.
- Sort results in descending order by average rating (before rounding), then ascending by company name.

▼ Schema

companies		
name	type	description
id	SMALLINT	unique id, primary key
name	VARCHAR(255)	
address	VARCHAR(255)	
phone	VARCHAR(255)	

categories		
name	type	description
company_id	SMALLINT	foreign key, companies.id
name	VARCHAR(255)	
review_rating	SMALLINT	

▼ Sample Data Tables

companies			
id	name	address	phone
1	Casper, Oberbrunner and Williamson	53 Di Loreto Hill	+420 (569) 566-3689
2	Tromp, Kozey and Abbott	84 Mcguire Plaza	+62 (145) 722-2330
3	Gerlach, Hayes and Stamm	80 Service Point	+86 (731) 234-4119
4	Wolff-Fadel	06 Fair Oaks Trail	+7 (894) 233-0976
5	Kihn-Cronin	483 Nobel Road	+1 (396) 693-1661

categories		
company_id	name	review_rating
1	HVAC	2
2	HVAC	2
2	Retaining Wall and Brick Pavers	1
2	Rebar & Wire Mesh Install	2
3	Prefabricated Aluminum Metal Canopies	2
3	Prefabricated Aluminum Metal Canopies	0
3	RF Shielding	2
3	Overhead Doors	0
3	Rebar & Wire Mesh Install	5
3	Termite Control	0
4	Sitework & Site Utilities	0
4	Electrical and Fire Alarm	2
4	Masonry	2
4	Temp Fencing, Decorative Fencing and Gates	0
4	Elevator	1
4	Drywall & Acoustical (FED)	5
5	Asphalt Paving	0
5	Glass & Glazing	1
5	Framing (Steel)	3
5	Structural & Misc Steel Erection	1

▼ Expected Output

name	address	phone	overall_review_rating
Casper, Oberbrunner and Williamson	53 Di Loreto Hill	+420 (569) 566-3689	2.0 (1 categories)
Tromp, Kozey and Abbott	84 Mcguire Plaza	+62 (145) 722-2330	1.7 (3 categories)
Wolff-Fadel	06 Fair Oaks Trail	+7 (894) 233-0976	1.7 (6 categories)
Gerlach, Hayes and Stamm	80 Service Point	+86 (731) 234-4119	1.5 (6 categories)
Kihn-Cronin	483 Nobel Road	+1 (396) 693-1661	1.3 (4 categories)

Question - 78

SQL: Domain Name Registrar Accounts Report

Generate a report of active accounts with unexpired domain names and their nearest expiration dates.

Your result should include:

- Account username
- Number of domains for each account
- Nearest expiration date after July 15, 2022

Requirements:

- Only include active accounts.
- Only include domain names with expiration dates after July 15, 2022.
- Sort results in ascending order by *username*.

▼ Schema

accounts		
name	type	description
id	SMALLINT	unique id, primary key
username	VARCHAR(255)	
is_active	SMALLINT	Account status: 1 = Active, 0 = Not active

domains		
name	type	description
account_id	SMALLINT	foreign key, accounts.id
name	VARCHAR(255)	
expiration_date	VARCHAR(19)	

▼ Sample Data Tables

accounts		
id	username	is_active
1	obeedie0	0
2	stopham1	1
3	ndolder2	1
4	jyanshinov3	1
5	ewilflinger4	0

domains		
account_id	name	expiration_date
1	imgur.com	2022-05-14
1	domainmarket.com	2022-07-02
1	comsenz.com	2022-07-28
1	gizmodo.com	2022-08-09
1	toplist.cz	2022-08-15
1	scientificamerican.com	2022-09-03
1	examiner.com	2022-12-18
1	photobucket.com	2023-01-22
2	merriam-webster.com	2022-02-20
2	tripod.com	2022-08-08
3	ca.gov	2022-04-24
3	ehow.com	2022-06-28
3	purevolume.com	2022-07-01
3	squidoo.com	2022-10-27
3	eepurl.com	2022-12-21
4	digg.com	2022-05-14
4	jugem.jp	2022-08-05
4	artisteer.com	2022-10-21
5	behance.net	2022-03-24
5	cnn.com	2022-05-11

▼ Expected Output

username	domains	nearest_expiration
jyanshinov3	2	2022-08-05
ndolder2	2	2022-10-27
stopham1	1	2022-08-08

Create a report of advertising campaigns with high average event values as of July 15, 2022.

Your result should include:

- Campaign name
- Total number of events for each campaign
- Average event value (rounded to five decimal places, e.g., 0.000004 → 0.00000)

Requirements:

- Only include events from July 15, 2022.
- Only include campaigns with average event values > 0.7.
- Sort results in descending order by average value.

▼ Schema

campaigns		
name	type	description
id	SMALLINT	unique id, primary key
name	VARCHAR(255)	

events		
name	type	description
campaign_id	SMALLINT	foreign key campaign.id
dt	VARCHAR(19)	Event datetime
value	DECIMAL(6,5)	Event value

▼ Sample Data Tables

campaigns	
id	name
1	11-080 - Registration Equipment
2	12-700 - Systems Furniture
3	9-900 - Paints and Coatings

events		
campaign_id	dt	value
1	2022-07-14 13:11:38	0.59275

1	2022-07-14 14:55:43	0.12928
1	2022-07-14 18:16:11	0.82350
1	2022-07-15 01:19:44	0.97144
1	2022-07-15 22:52:02	0.60728
1	2022-07-16 08:55:38	0.71158
1	2022-07-16 10:22:44	0.29627
2	2022-07-14 02:36:31	0.42323
2	2022-07-14 04:45:32	0.91077
2	2022-07-14 07:24:11	0.35956
2	2022-07-15 06:43:08	0.16662
2	2022-07-16 08:21:27	0.02559
2	2022-07-16 11:59:41	0.34606
2	2022-07-16 23:26:12	0.62697
3	2022-07-14 00:21:56	0.97297
3	2022-07-14 10:22:11	0.93894
3	2022-07-14 12:29:59	0.44633
3	2022-07-15 01:17:41	0.37531
3	2022-07-15 14:20:48	0.24872
3	2022-07-16 23:02:51	0.80594

▼ Expected Output

campaign	events	average_value
11-080 - Registration Equipment	2	0.78936

Question - 80

SQL: Ecommerce Deal Report

Generate a report of the top three seller profiles with the highest total deal amounts in June.

Your result should include:

- First name
- Last name
- Email

- Total deal amount
- Requirements:
- Only include deals from June.
 - Sort results in descending order by total amount.
 - Limit results to the top three records.

▼ Schema

profiles		
name	type	description
id	SMALLINT	unique id, primary key
first_name	VARCHAR(255)	
last_name	VARCHAR(255)	
email	VARCHAR(255)	

deals		
name	type	description
profile_id	SMALLINT	foreign key into profile.id
dt	VARCHAR(19)	Deal datetime
amount	DECIMAL(5,2)	Deal amount

▼ Sample Data Tables

profiles			
id	first_name	last_name	email
1	Wallis	Treadway	wtreadway0@senate.gov
2	Franklin	Blackston	fblackston1@parallels.com
3	Honorio	Constant	hconstant2@umich.edu
4	Bertine	Hillaby	bhillaby3@artisteer.com
5	Constance	Knutsen	cknutsen4@google.ca

deals		
profile_id	dt	amount
5	2022-05-21 02:44:24	49.10
2	2022-05-22 23:26:59	46.21

1	2022-05-23 09:56:25	58.57
5	2022-05-28 02:38:08	27.81
4	2022-06-04 07:16:27	22.31
4	2022-06-04 14:15:03	36.33
5	2022-06-04 15:03:10	21.41
1	2022-06-07 02:58:06	92.84
4	2022-06-08 05:09:52	24.41
3	2022-06-13 03:28:52	61.55
4	2022-06-16 15:09:39	77.70
5	2022-06-18 16:51:32	58.79
4	2022-06-20 02:55:20	43.61
3	2022-06-22 06:52:10	10.41
1	2022-06-23 04:59:05	6.59
1	2022-06-30 16:11:02	43.07
4	2022-07-05 06:05:28	36.45
5	2022-07-12 07:49:51	14.76
4	2022-07-12 18:58:11	91.61
5	2022-07-14 00:50:45	69.61

▼ Expected Output

first_name	last_name	email	total
Bertine	Hillaby	bhillaby3@artisteer.com	204.36
Wallis	Treadway	wtreadway0@senate.gov	142.50
Constance	Knutsen	cknutsen4@google.ca	80.20

Question - 81

SQL: Freelance Platform Candidate Review

Create a report of qualified freelancer profiles based on job success and verification status.

Your result should include:

- First name
- Last name

- Email
- Job success score

Requirements:

- Only include profiles with Job Success Score (JSS) greater than 90.
- Only include verified profiles (*is_verified* = 1).
- Sort results in descending order by *job_success_score*, then ascending by *first_name* and *last_name*.
- Limit results to the first ten records.

▼ Schema

profiles		
name	type	description
id	SMALLINT	unique id, primary key
first_name	VARCHAR(255)	
last_name	VARCHAR(255)	
email	VARCHAR(255)	
is_verified	SMALLINT	1 = True, 0 = False

stats		
name	type	description
profile_id	SMALLINT	foreign key into profiles.id
job_success_score	SMALLINT	

▼ Sample Data Tables

profiles				
id	first_name	last_name	email	is_verified
1	Junia	Sehorsch	jsehorsch0@oracle.com	0
2	Dave	Halliburton	dhalliburton1@pbs.org	1
3	Agneta	Dutch	adutch2@thetimes.co.uk	1
4	Kendell	Sylvester	ksylvester3@canalblog.com	1
5	Koralle	Ragsdale	kragdsdale4@buzzfeed.com	1
6	Roma	Kenelin	rkenelin5@prnewswire.com	1
7	Harold	Molloy	hmolloy6@ycombinator.com	1
8	Berri	Hartzogs	bhartzogs7@ox.ac.uk	0

9	Garrik	Preddle	gpreddle8@topsy.com	1
10	Sophie	Messenger	smessenger9@myspace.com	1
11	Ashby	Philipsson	aphilipssona@typepad.com	1
12	Kayle	Jesteco	kjestecob@ocn.ne.jp	1
13	Munroe	Chevolleau	mchevolleauc@yandex.ru	1
14	Etheline	Choake	echoaked@hao123.com	1
15	Marten	Zamboniari	mzamboniarie@cbc.ca	1
16	Hersch	Blasdale	hblasdalef@wunderground.com	1
17	Jori	MacFaell	jmacfaellg@va.gov	1
18	Margo	Finnemore	mfinnemoreh@discovery.com	1
19	Felicle	Ramsdale	framsdalei@devhub.com	1
20	Demetris	Arnet	darnetj@livejournal.com	1

stats	
profile_id	job_success_score
4	100
1	100
18	95
5	95
8	95
10	95
12	95
13	95
14	95
20	90
17	90
7	90
15	90
11	85
16	85
9	85

6	85
2	85
3	75
19	75

▼ Expected Output

first_name	last_name	email	job_success_score
Kendell	Sylvester	ksylvester3@canalblog.com	100
Etheline	Choake	echoaked@hao123.com	95
Kayle	Jesteco	kjestecob@ocn.ne.jp	95
Koralle	Ragsdale	kragsdale4@buzzfeed.com	95
Margo	Finnemore	mfinnemoreh@discovery.com	95
Munroe	Chevolleau	mchevolleauc@yandex.ru	95
Sophie	Messenger	smessenger9@myspace.com	95
Demetris	Arnet	darnetj@livejournal.com	90
Harold	Molloy	hmolloy6@ycombinator.com	90
Jori	MacFaell	jmacfaellg@va.gov	90

Question - 82

SQL: Virtual Machine Deployment Report

Generate a report of CVM (Cloud Virtual Machine) configurations and their total deployments in 2021.

Your result should include:

- Configuration name
- Number of deployments for the configuration

Requirements:

- Only include deployments from 2021.
- Sort results in descending order by number of deployments.

▼ Schema

configurations		
name	type	description
id	VARCHAR(64)	unique id, primary key

name	VARCHAR(255)	
------	--------------	--

deployments		
name	type	description
configuration_id	VARCHAR(64)	foreign key configurations.id
dt	VARCHAR(19)	deployment date and time

▼ Sample Data Tables

configurations	
id	name
1vcpu_512mb_10gb_500gb	1 CPU / 512 MB RAM / 10 GB SSD Disk / 500 GB transfer
1vcpu_1gb_25gb_1tb	1 CPU / 1 GB RAM / 25 GB SSD Disk / 1000 GB transfer
1vcpu_2gb_50gb_2tb	1 CPU / 2 GB RAM / 50 GB SSD Disk / 2 TB transfer
2vcpu_2gb_60gb_3tb	2 CPUs / 2 GB RAM / 60 GB SSD Disk / 3 TB transfer
2vcpu_4gb_80gb_4tb	2 CPUs / 4 GB RAM / 80 GB SSD Disk / 4 TB transfer
4vcpu_8gb_160gb_5tb	4 CPUs / 8 GB RAM / 160 GB SSD Disk / 5 TB transfer
8vcpu_16gb_320gb_6tb	8 CPUs / 16 GB RAM / 320 GB SSD Disk / 6 TB transfer

deployments	
configuration_id	dt
1vcpu_512mb_10gb_500gb	2020-10-22 05:59:47
1vcpu_1gb_25gb_1tb	2020-11-09 06:07:57
1vcpu_2gb_50gb_2tb	2020-12-02 14:47:24
2vcpu_2gb_60gb_3tb	2021-01-24 15:41:42
4vcpu_8gb_160gb_5tb	2021-01-25 09:31:37
2vcpu_2gb_60gb_3tb	2021-02-08 02:43:14
1vcpu_512mb_10gb_500gb	2021-03-25 06:13:36
8vcpu_16gb_320gb_6tb	2021-03-26 22:23:42
2vcpu_2gb_60gb_3tb	2021-05-24 09:48:16
1vcpu_512mb_10gb_500gb	2021-05-31 05:03:28

1vcpu_2gb_50gb_2tb	2021-08-25 22:24:10
2vcpu_4gb_80gb_4tb	2021-09-05 22:12:17
2vcpu_4gb_80gb_4tb	2021-09-23 08:31:50
2vcpu_4gb_80gb_4tb	2021-09-28 05:15:24
2vcpu_2gb_60gb_3tb	2021-10-14 08:26:20
1vcpu_2gb_50gb_2tb	2021-11-01 19:00:30
1vcpu_512mb_10gb_500gb	2021-11-26 10:53:05
1vcpu_1gb_25gb_1tb	2021-12-27 07:07:23
2vcpu_2gb_60gb_3tb	2022-02-13 06:00:29
2vcpu_2gb_60gb_3tb	2022-03-03 09:10:30

▼ Expected Output

configuration	deployments
2 CPUs / 2 GB RAM / 60 GB SSD Disk / 3 TB transfer	4
2 CPUs / 4 GB RAM / 80 GB SSD Disk / 4 TB transfer	3
1 CPU / 512 MB RAM / 10 GB SSD Disk / 500 GB transfer	3
1 CPU / 2 GB RAM / 50 GB SSD Disk / 2 TB transfer	2
8 CPUs / 16 GB RAM / 320 GB SSD Disk / 6 TB transfer	1
4 CPUs / 8 GB RAM / 160 GB SSD Disk / 5 TB transfer	1
1 CPU / 1 GB RAM / 25 GB SSD Disk / 1000 GB transfer	1

Question - 83

SQL: Visitors Behavior Report

Calculate the total number of purchase events that occurred in May 2022.

Your result should include:

- The total count of purchase events

Requirements:

- Only count events with "buy" in the type field.
- Only include events from May 2022.

▼ Schema

events		
name	type	description
dt	VARCHAR(19)	Event timestamp
type	VARCHAR(64)	Event type

▼ Sample Data Tables

events	
dt	type
2022-04-04 03:36:00	buy
2022-04-21 07:05:09	buy
2022-04-02 11:34:24	sell
2022-05-27 16:12:50	buy
2022-05-20 09:09:07	buy
2022-05-22 09:06:37	buy
2022-05-31 07:49:36	buy
2022-05-14 22:29:10	buy
2022-05-13 15:00:54	sell
2022-05-24 15:40:54	sell
2022-05-13 01:20:05	sell
2022-05-16 07:07:44	sell
2022-05-01 16:57:00	sell
2022-06-02 09:42:02	buy
2022-06-01 06:34:59	buy
2022-06-06 17:14:47	buy
2022-06-05 13:37:23	buy
2022-06-17 19:10:13	buy
2022-06-15 21:40:13	sell
2022-06-11 12:26:43	sell

▼ Expected Output

purchases
5

Question - 84

SQL: Advertising Campaigns Report

Identify the top three companies with positive returns on their advertising budgets.

Your result should include:

- Company name
- Profit (calculated as campaign revenues minus expenses)

Requirements:

- Only include companies with positive returns on advertising budgets.
- Sort results in descending order by profit.
- Limit results to the top three records.

▼ Schema

companies		
name	type	description
id	SMALLINT	ID
name	VARCHAR(255)	Company name

campaigns		
name	type	description
company_id	SMALLINT	Company ID
expenses	DECIMAL(7,2)	Campaign expenses
revenue	DECIMAL(7,2)	Campaign revenue

▼ Sample Data Tables

companies	
id	name
1	Lion Biotechnologies, Inc.
2	Boston Private Financial Holdings, Inc.
3	Universal Corporation

4	Arbutus Biopharma Corporation
5	Royal Bank Of Canada
6	Penn West Petroleum Ltd
7	Public Storage
8	Halcon Resources Corporation
9	TTM Technologies, Inc.
10	Atwood Oceanics, Inc.
11	ACADIA Pharmaceuticals Inc.
12	Central European Media Enterprises Ltd.
13	Oxbridge Re Holdings Limited
14	Western Refining Logistics, LP
15	Vaalco Energy Inc
16	Xilinx, Inc.
17	Liberty Global plc
18	Honda Motor Company, Ltd.
19	Great Plains Energy Inc
20	Assurant, Inc.

campaigns		
company_id	expenses	revenue
1	7390.24	8652.18
2	5774.65	7955.47
3	2154.71	5920.23
4	9366.49	3397.85
5	2765.18	9158.63
6	7908.41	5018.85
7	2251.44	6654.52
8	3383.14	9354.79
9	8287.96	9522.53
10	4356.62	4658.52
11	9272.86	9161.77

12	4996.18	5903.57
13	8354.75	2259.26
14	6402.90	8146.16
15	1692.05	686.71
16	5988.48	9089.41
17	6192.33	7580.19
18	3016.37	7761.25
19	9838.05	1293.09
20	4386.52	9513.73

▼ Expected Output

company_name	profit
Royal Bank Of Canada	6393.45
Halcon Resources Corporation	5971.65
Assurant, Inc.	5127.21

Question - 85

SQL: Traffic Audit Report

Generate a list of customers whose hardware has a higher downstream rate than upstream rate, with no more than one downtime incident. Your result should include:

- MAC address
- Upstream rate
- Downstream rate
- Downtime rate

Requirements

- Only include clients where *downstream_rate* exceeds *upstream_rate*.
- Only include clients with *downtime_rate* of "never" or "once".
- Sort results in ascending order by MAC address.

▼ Schema

clients		
name	type	description
mac	VARCHAR(64)	MAC address
upstream_rate	INT	Upstream rate

downstream_rate	INT	Downstream rate
downtime_rate	VARCHAR(64)	Downtime rate

▼ Sample Data Tables

clients			
mac	upstream_rate	downstream_rate	downtime_rate
78-C1-E5-20-D5-61	925526	5195	never
78-E2-20-71-9C-30	582152	375829	never
0D-09-F7-77-03-E5	359529	710743	never
56-18-67-55-58-EA	78626	562544	once
C9-73-EC-1C-4C-B7	574927	669655	yearly
02-35-3F-7B-CC-76	430072	296196	seldom
3D-95-33-8A-65-F9	894176	489401	monthly
8D-33-3F-0E-04-D5	897666	297063	weekly
0E-0A-63-B9-79-3E	69133	984354	seldom
15-D3-2A-DD-02-A4	19203	995983	seldom
8D-42-B1-97-AB-87	476648	177677	monthly
6B-64-60-47-16-D3	700056	374321	monthly
FD-61-81-00-BF-EC	216401	498229	yearly
95-46-C6-C7-6F-E0	236331	341013	monthly
11-0E-62-32-62-5E	694746	451525	daily
D6-5B-72-D5-FF-4F	2931	992852	monthly
66-BF-AD-F2-E5-45	861075	44216	monthly
E5-C9-C6-74-2E-A8	639487	968494	daily
18-56-3A-93-8E-9F	494945	259910	weekly
51-EB-D7-22-45-99	219419	326479	often

▼ Expected Output

mac	upstream_rate	downstream_rate	downtime_rate
0D-09-F7-77-03-E5	359529	710743	never

56-18-67-55-58-EA	78626	562544	once
-------------------	-------	--------	------

Question - 86

SQL: Calendar Application Events Report

Write a query to retrieve the five earliest events from a calendar application, excluding events whose owners are on vacation.

The result should have the following columns: *dt / title / full_name / email_address*. It should be sorted in ascending order by *dt*, and limited to the first five records.

▼ Schema

owners		
name	type	description
id	SMALLINT	ID
full_name	VARCHAR(255)	Full name
email_address	VARCHAR(255)	Email address
on_vacation	SMALLINT	Vacation status, 1 = on vacation

events		
name	type	description
owner_id	SMALLINT	Owner ID
dt	VARCHAR(19)	Timestamp
title	VARCHAR(255)	Title

▼ Sample Data Tables

owners			
id	full_name	email_address	on_vacation
2	Aleksandr Fellows	afellows1@instagram.com	0
3	Collete Pack	cpack2@mit.edu	0
4	Lorelle Squibb	lsquibb3@huffingtonpost.com	0
1	Benjamin Sevier	bsevier0@discuz.net	1
5	Cointon Welberry	cwelberry4@theguardian.com	1

events		
owner_id	dt	title
1	2021-08-13 09:17:41	ac consequat metus sapien ut
1	2021-10-26 16:14:28	a libero nam
1	2022-05-18 16:09:36	vestibulum ante ipsum primis in faucibus
1	2022-06-04 20:37:36	nisi volutpat eleifend donec
2	2021-01-17 23:34:59	eu mi nulla ac
2	2021-01-26 15:59:04	nisl venenatis lacinia
2	2021-04-25 05:04:29	arcu adipiscing molestie hendrerit
2	2022-02-22 10:24:50	lectus vestibulum quam sapien varius
2	2022-05-22 03:04:33	pellentesque quisque porta volutpat erat
3	2021-03-11 08:25:21	felis sed interdum venenatis
3	2022-02-15 07:29:45	nulla sed accumsan felis ut
4	2021-02-28 16:10:55	cras mi pede malesuada in imperdiet et
4	2021-05-20 03:35:11	odio in hac habitasse platea dictumst
4	2021-09-01 19:25:41	sagittis nam congue risus semper
4	2022-02-04 03:11:53	luctus cum sociis natoque penatibus et magnis
4	2022-06-18 08:39:41	nec molestie sed justo pellentesque
5	2021-04-04 13:39:38	id massa id nisl venenatis lacinia aenean
5	2021-10-18 19:04:14	adipiscing lorem vitae mattis
5	2021-10-19 00:56:11	vestibulum eget vulputate ut ultrices vel augue
5	2022-04-26 18:55:04	at dolor quis

▼ Expected Output

dt	title	full_name	email_address
2021-01-17 23:34:59	eu mi nulla ac	Aleksandr Fellows	afellows1@instagram.com
2021-01-26 15:59:04	nisl venenatis lacinia	Aleksandr Fellows	afellows1@instagram.com
2021-02-28 16:10:55	cras mi pede malesuada in imperdiet et	Lorelle Squibb	lsquibb3@huffingtonpost.com
2021-03-11 08:25:21	felis sed interdum venenatis	Collete Pack	cpack2@mit.edu
2021-04-25 05:04:29	arcu adipiscing molestie hendrerit	Aleksandr Fellows	afellows1@instagram.com

Question - 87

Cryptocurrency Wallets Monitoring

Write a query to identify wallets that meet two specific criteria: they have more than one transaction (at least two records with the same ID in the transactions table) and their total transaction amount exceeds 10. The output should display the wallet ID and the total amount of all transactions for that wallet.

▼ Schema

There are 2 tables:

wallets		
name	type	description
id	SMALLINT	Wallet ID
transaction_id	SMALLINT	Transaction ID

transactions		
name	type	description
id	SMALLINT	Transaction ID
amount	DECIMAL(4,2)	Transaction amount

▼ Sample Data Tables

For the sample data in tables:

wallets	
id	transaction_id
1	19
1	20
2	14
2	4
3	17
3	8
4	1
4	7
6	15

6	16
6	5
7	11
7	12
7	13
7	3
7	9
8	6
9	18
10	10
10	2

transactions	
id	amount
1	3.38
2	3.38
3	3.41
4	9.43
5	0.68
6	14.91
7	2.85
8	6.21
9	9.87
10	2.89
11	4.85
12	3.12
13	3.27
14	6.89
15	0.28
16	8.70
17	3.22

18	1.63
19	5.02
20	3.99

the expected output is:

wallet_id	total_amount
2	16.32
7	24.52

Question - 88

Firewall Active Clients Tracking

Write a query to list all client MAC addresses that have generated network traffic. The output should display the MAC addresses in alphabetical order, including only those clients whose IDs appear in the traffic table.

▼ Schema

There are 2 tables:

clients		
name	type	description
id	SMALLINT	Client ID
mac	VARCHAR(64)	MAC address

traffic		
name	type	description
client_id	SMALLINT	Client ID
amount	INT	Traffic amount

▼ Sample Data Tables

For the sample data in tables:

clients	
id	mac

1	E5-A3-AC-8A-20-F9
2	3B-2F-83-25-A8-81
3	3A-4E-A6-43-1D-B1
4	B7-03-14-91-8F-58
5	63-1A-FD-9A-AF-6F
6	D6-B8-1F-1D-34-04
7	38-83-E5-F8-C8-DC
8	E9-F6-89-7F-8D-34
9	82-2E-B3-67-04-41
10	D1-9D-DE-37-A0-49
11	8A-46-F4-83-29-13
12	4C-1F-7B-C7-08-7E
13	72-57-E6-CA-2C-91
14	57-F7-E7-E7-45-36
15	05-8A-05-1D-2D-20
16	46-06-F1-B9-65-7C
17	0A-E0-26-9D-2A-27
18	F0-86-99-18-36-9B
19	DD-81-BF-53-BD-9B
20	50-53-64-8E-42-BE

traffic	
client_id	amount
3	6385047
8	6490817
14	9109219
16	5558512
17	1870152
17	8228920
18	326127
18	5429741

18	4063477
19	7411789
20	5832337
20	1426585
23	1097368
23	6769594
23	802387
24	5959513
24	1300408
24	4631624
28	1629306
29	2814818

the expected output is:

mac
0A-E0-26-9D-2A-27
3A-4E-A6-43-1D-B1
46-06-F1-B9-65-7C
50-53-64-8E-42-BE
57-F7-E7-E7-45-36
DD-81-BF-53-BD-9B
E9-F6-89-7F-8D-34
F0-86-99-18-36-9B

Explanation

Each of the addresses listed has at least one record in the traffic table.

Question - 89

Active Wallets

Write a query to list all wallet addresses that have one or more transactions recorded. The output should display the wallet addresses in alphabetical order, including only those wallets whose IDs appear in the transactions table.

There are 2 tables:

wallets		
name	type	description
id	SMALLINT	Wallet ID
address	VARCHAR(64)	Wallet address

transactions		
name	type	description
wallet_id	SMALLINT	Wallet ID
credit	DECIMAL(4,2)	Transaction amount

▼ Sample Data Tables

For the sample data in tables:

wallets	
id	address
1	0x1ecc4cefde6dfb773352a2dcd8b5f518ccd24ff4
2	0x0f4487168610dcae7f16b6c000a7ba284bb6703c
3	0x54c5c516b5c601a3e6fdea18db966186274879e6
4	0x50c404a6790d44849c3500d95f147d2102db5f77
5	0x41cc32037fdcc6cbf5cb04ada4e38032a84361ac
6	0xb38ca2076b2c3a0a1c0796ccab5dc3fcb645336
7	0x923c6ee4debce6c37c512eb90e9b26fc9bbfc3e1
8	0x76cfbf67e1765117a98fd2286bdbcb731b0d29ec
9	0xf4dff43df89365453440dbeb5e53445d3400d218
10	0x010a4b23f985eda6b397864878cb70bf0b233aa6
11	0x106c4c852394d5a6580c11ca0060687a798ec78c
12	0x541da74b9f4b2283524a65156263bcce7429ba61
13	0x86df8e62ad23a35ee2e64b3b4b128d8a3660116a
14	0x6b021b12b779aec27009f18bd6ad227685df5474
15	0x7073e9f48b7211d9940db4c8bad8e31d5d9d0577

16	0x4672e132b9627a7db76e5af431bb5febc93b1b2f
17	0x5cbc82a01d3b9df8f50ee09bf5a1c48afe0cb966
18	0x1c7d30f90081b45004e0d6549534ef8658795672
19	0xf830618ca9f027ce5f23bf270629a6a418ad355c
20	0x1a31ac1b923fcc17a42a340091424ea18192a3e7

transactions	
wallet_id	credit
3	5.21
6	24.04
8	29.66
10	1.67
10	3.32
10	4.27
14	3.32
14	3.35
15	14.34
16	11.94
20	13.86
20	16.54
21	11.64
25	12.86
26	8.77
26	12.36
27	29.65
28	3.00
28	13.26
28	2.85

the expected output is:

address

0x010a4b23f985eda6b397864878cb70bf0b233aa6
0x1a31ac1b923fcc17a42a340091424ea18192a3e7
0x4672e132b9627a7db76e5af431bb5febc93b1b2f
0x54c5c516b5c601a3e6fdea18db966186274879e6
0x6b021b12b779aec27009f18bd6ad227685df5474
0x7073e9f48b7211d9940db4c8bad8e31d5d9d0577
0x76cfbf67e1765117a98fd2286bdbcb731b0d29ec
0xb38ca2076b2c3a0a1c0796ccab5dc3fcbb645336

Explanation

Each of the wallets listed has at least one record in the transactions table.

Question - 90
Animal Tracking

Write a query to retrieve the names of all animals that have been tracked at least once. The output should list the animal names in alphabetical order, including only those animals whose IDs appear in the tracking log.

▼ Schema

There are 2 tables:

animals		
name	type	description
id	SMALLINT	Animal ID
name	VARCHAR(64)	Animal name

tracklog		
name	type	description
animal_id	SMALLINT	Animal ID
tracked_at	VARCHAR(19)	Track timestamp

▼ Sample Data Tables

For the sample data in tables:

animals

id	name
1	Blue and yellow macaw
2	Jungle kangaroo
3	Stork, woolly-necked
4	North American river otter
5	Square-lipped rhinoceros
6	Black-fronted bulbul
7	American beaver
8	Capybara
9	Black-backed jackal
10	Dragon, ornate rock
11	Wombat, southern hairy-nosed
12	Snake, carpet
13	Egyptian cobra
14	Green heron
15	Indian star tortoise
16	Roan antelope
17	Rhea, common
18	Fairy penguin
19	Black-eyed bulbul
20	Starling, cape

tracklog	
animal_id	tracked_at
2	2021-07-08 12:30:34
5	2021-09-15 03:00:04
8	2021-12-14 11:20:50
9	2021-05-15 14:54:15
11	2021-07-03 17:04:14
11	2021-02-23 09:02:11
15	2021-11-02 22:49:37

15	2021-07-31 04:51:25
15	2021-06-06 14:57:23
19	2021-03-25 02:00:31
19	2021-04-21 22:02:51
19	2021-07-17 17:12:35
21	2021-05-06 03:35:59
22	2021-09-21 22:49:36
22	2021-12-04 00:26:29
24	2021-12-27 08:19:52
26	2021-05-14 07:30:11
28	2021-06-08 00:47:00
28	2021-04-24 00:42:31
30	2021-05-06 21:03:03

the expected output is:

name
Black-backed jackal
Black-eyed bulbul
Capybara
Indian star tortoise
Jungle kangaroo
Square-lipped rhinoceros
Wombat, southern hairy-nosed

Explanation

Each of the animals listed has at least one record in the tracklog table.

Question - 91

SQL: Exchange Rates

"Write a query to calculate the fees paid by customers for their stock exchange transactions. The exchange charges 0.1% for buy orders and 0.15% for sell orders.

Display the customer name and their total fees paid, rounded to 2 decimal places, e.g., 5.004 is shown as 5.00. Order the results alphabetically ascending by customer name.

To round the data:
In MySQL, Oracle, or DB2, use ROUND(val, 2).
In MS SQL use FORMAT(val, 'N2').

▼ Schema

You are provided 2 tables: customers , orders .

CUSTOMERS		
Name	Type	Description
id	int	Unique id of the customer.
customer_name	varchar(30)	Name of the customer.

ORDERS		
Name	Type	Description
order_id	int	Unique id of the order.
customer_id	int	Id of the customer referring to the customers table.
order_type	varchar(5)	Type of order placed (Buy or Sell).
order_amount	decimal(18,2)	Amount of the order.

▼ Sample Data Tables

CUSTOMERS	
id	customer_name
401	Hubert Keesler
402	Devin Vert
403	Lashawna Bowerman
404	Brigid Wellborn
405	Josefine Perl

ORDERS			
order_id	customer_id	order_type	order_amount
4361	401	Sell	912.77
3478	405	Sell	741.69
7292	405	Sell	436.05
5833	405	Sell	231.30
3472	402	Buy	950.92

4472	401	Sell	367.70
2624	404	Buy	218.15
7198	405	Buy	797.29
7660	403	Buy	131.18
5192	401	Buy	362.44
5260	402	Buy	636.26
2726	403	Sell	138.15
6594	401	Buy	234.51
4657	404	Buy	427.30
9744	402	Sell	623.36

OUTPUT	
customer_name	total_fees
Brigid Wellborn	0.65
Devin Vert	2.52
Hubert Keesler	2.52
Josefine Perl	2.91
Lashawna Bowerman	0.34

Explanation

For Devin Vert, total amount of buy orders was 1587.18 and total amount of buy orders was 623.36. Fee applied on buy and sell orders will be 1.58718 and 0.93504 with rate of 0.1% and 0.15% respectively. Total fee would be 2.52222 after rounding final fee would be 2.52.

Question - 92

SQL: Credit Dues

Write a query to calculate the amount due for credit card customers whose interest rate exceeds 12%. The amount due includes the total transaction amount plus the interest calculated based on each customer's individual interest rate. Round the results to 2 decimal places, e.g., 5.004 → 5.00, and order them from highest to lowest amount due.

▼ Schema

There are 2 tables: `credit_holders`, `transactions`.

CREDIT_HOLDERS		
Name	Type	Description
id	int	Unique id of the credit holder (customer).
first_name	varchar(15)	First name of the customer.

last_name	varchar(15)	Last name of the customer.
interest_rate	int	Interest based on their credit score (in percentage).

TRANSACTIONS		
Name	Type	Description
transaction_id	int	Unique id of the transaction.
credit_holder_id	int	Id of the customer referring to the credit_holders table.
amount	decimal(18,2)	Amount of the transaction.

▼ Sample Data Tables

CREDIT_HOLDERS			
id	first_name	last_name	interest_rate
101	Clemencia	Hutsell	12
102	Susannah	Ismail	18
103	Sixta	Hagy	18
104	Otto	Izquierdo	18
105	Anita	Degroot	15

TRANSACTIONS		
transaction_id	credit_holder_id	amount
4361	101	65.22
3478	104	51.85
7292	104	64.60
5833	105	72.15
3472	102	96.28
4472	101	80.06
2624	101	85.27
7198	104	23.73
7660	103	81.86
5192	101	69.64
5260	101	71.72
2726	102	57.66
6594	103	23.23

4657	101	81.68
9744	104	99.57
2054	103	51.13
7156	105	12.78
3273	105	36.15
9756	101	45.41
9702	105	69.75

OUTPUT	
full_name	dues
Otto Izquierdo	282.91
Anita Degroot	219.45
Sixta Hagy	184.34
Susannah Ismail	181.65

Explanation

For Otto Izquierdo, there are 4 transactions of 51.85, 64.60, 23.73, 99.57 which sums to 239.75. After applying 18% interest, the due amount is 282.905 rounded to 282.91.

Question - 93

SQL: Interest Earned

Write a query to calculate the interest earned in one year for bank depositors at a 5% interest rate. Report the depositor names and their earned interest, ordered alphabetically by name. The interest amount should display the same currency symbol as the deposit and be rounded to 2 decimal places, e.g., \$5.004 shows as \$5.00.

Note:

The *amount* field is a string where the first character is a currency symbol, and the remainder is the deposit amount.

▼ Schema

There is 1 table: `accounts`.

ACCOUNTS		
Name	Type	Description
id	int	Unique id of the account.
account_holder	varchar(30)	Name of the account holder.
amount	varchar(10)	Amount deposited in the fixed deposit.

▼ Sample Data Tables

ACCOUNTS		
id	account_holder	amount
1	Ellis Beane	\$5582.03
2	Drew Nolf	\$2470.3
3	Jordan Chatmon	\$6211.52
4	Robin Hansard	\$8133.31
5	Spencer Days	\$5273.81
6	Morgan Criss	\$3741.85
7	Wesley Waugh	\$7056.14
8	Alex Canty	\$2590.45
9	Blake Hawbaker	\$3987.27
10	Taylor Blackston	\$8351.98

OUTPUT	
account_holder	interest
Taylor Blackston	\$417.60
Wesley Waugh	\$352.81
Jordan Chatmon	\$310.58
Robin Hansard	\$406.67
Alex Canty	\$129.52
Drew Nolf	\$123.52
Blake Hawbaker	\$199.36
Spencer Days	\$263.69
Ellis Beane	\$279.10
Morgan Criss	\$187.09

Explanation

For Taylor Blackston, after 1 year the interest is $\$8351.98 \times 0.05 = \147.599 which rounds to \$417.60.

Question - 94

SQL: Monthly Revenue

Write a query to fetch the year, month, and total transaction amounts for each month over a three-year period. The transaction ID contains the year and month of the transaction in the format YYMMMXXXX, e.g., "19JAN1234" is January, 2019. Order the results ascending by year and month to show the financial pattern over time.

▼ Schema

There is 1 table: `transactions`.

TRANSACTIONS		
Name	Type	Description
transaction_id	varchar(10)	Unique id of the transaction YYYYMM[the rest]
amount	decimal(18,2)	Amount of the transaction.

▼ Sample Data Tables

TRANSACTIONS	
transaction_id	amount
19SEP2187	785.72
19OCT4361	752.64
19APR3478	197.92
19SEP7292	910.26
21MAR5833	344.70
20MAY3472	939.61
20DEC4472	154.98
20DEC2624	935.44
21JUN7198	309.81
19APR7660	528.10
20MAR5192	995.22
19OCT5260	861.11
21JUN2726	611.94
19OCT6594	478.54
19APR4657	183.20

OUTPUT		
year	month	total_transactions
19	APR	909.22
19	OCT	2092.29
19	SEP	1695.98
20	DEC	1090.42
20	MAR	995.22

20	MAY	939.61
21	JUN	921.75
21	MAR	344.70

Explanation

For the month of April, 19 there are three transactions of 197.92, 528.10, and 183.20 which sum up to 909.22.

Question - 95
MySQL: Final Result

Write a query to fetch the full name of each student along with their average GPA across all four years of their B.Tech degree program. Round the average to one decimal place, e.g., 4.01 → 4.0.

Order the results from highest to lowest average GPA after rounding, then ascending by full name to identify top academic performers.

▼ Schema

There is 1 table: results .

RESULTS		
Name	Type	Description
id	int	Unique id of the student.
first_name	varchar(20)	First name of the student.
last_name	varchar(20)	Last name of the student.
cgpa_first_year	float	CGPA year 1
cgpa_second_year	float	CGPA year 2
cgpa_third_year	float	CGPA year 3
cgpa_fourth_year	float	CGPA year 4

▼ Sample Data Tables

RESULTS						
id	first_name	last_name	cgpa_first_year	cgpa_second_year	cgpa_third_year	cgpa_fourth_year
1	Pearlene	Beane	7	5.1	8.4	8.9
2	Franklin	Nolf	7.7	7.2	5.2	8.3
3	Bell	Chatmon	7.3	8.4	8.9	10
4	Belva	Hansard	6.2	9.2	5.8	6.7
5	Missy	Days	8.3	10	7.3	6.7
6	Vicenta	Criss	5.4	9.5	6.1	9
7	Annelle	Waugh	6.5	7.9	9.6	9.3

8	Darby	Canty	5.5	9	8.6	5.9
9	Ka	Hawbaker	5.7	6.4	5.2	6.8
10	Alease	Blackston	5.3	7.5	9.3	6

OUTPUT	
full_name	average_gpa
Bell Chatmon	8.6
Annelle Waugh	8.3
Missy Days	8.1
Vicenta Criss	7.5
Darby Canty	7.3
Pearlene Beane	7.3
Franklin Nolf	7.1
Alease Blackston	7.0
Belva Hansard	7.0
Ka Hawbaker	6.0

Explanation

For Bell Chatmon, the cgpa sum is 7.3 + 8.4 + 8.9 + 10 = 34.6. The average is 34.6/4 = 8.65. After rounding to 1 decimal place the final value is 8.6. All values should show 1 place after the decimal, even in a case like Belva Hansard where the final value should be 7.0.

Question - 96

SQL: Mutual Funds

Write a query to fetch the month, name of the fund, and the total amount invested that month for a bank's mutual funds. Order the report ascending by month and mutual fund name to provide a chronological view of investments across different fund types.

▼ Schema

There is 1 table: funds .

FUNDS		
Name	Type	Description
id	int	Unique id of the order
order_date	date	transaction date
fund_name	varchar(50)	Name of the fund
order_amount	int	Amount invested

▼ Sample Data Tables

FUNDS			
id	order_date	fund_name	order_amount
1	2021-09-05	Mid-Cap	151
2	2020-11-27	Small-Cap	784
3	2020-11-22	Multi-Cap	761
4	2020-02-26	Large-Cap	778
5	2020-01-04	Mid-Cap	949
6	2020-02-01	Large-Cap	392
7	2020-02-07	Mid-Cap	629
8	2020-06-01	Mid-Cap	529
9	2020-08-05	Large-Cap	258
10	2021-09-23	Mid-Cap	739
11	2020-02-14	Large-Cap	563
12	2021-09-29	Small-Cap	817
13	2020-05-11	Large-Cap	121
14	2021-09-18	Mid-Cap	341
15	2021-09-07	Large-Cap	260
16	2021-10-11	Small-Cap	102
17	2020-06-04	Mid-Cap	496
18	2021-07-26	Multi-Cap	321
19	2020-04-11	Small-Cap	323
20	2020-09-24	Multi-Cap	938
21	2020-06-16	Small-Cap	668
22	2021-03-08	Small-Cap	733
23	2021-04-27	Small-Cap	597
24	2020-01-11	Small-Cap	767
25	2021-11-14	Small-Cap	489

OUTPUT		
month	fund_name	total_investments
1	Mid-Cap	949
1	Small-Cap	767
2	Large-Cap	1733
2	Mid-Cap	629

3	Small-Cap	733
4	Small-Cap	920
5	Large-Cap	121
6	Mid-Cap	1025
6	Small-Cap	668
7	Multi-Cap	321
8	Large-Cap	258
9	Large-Cap	260
9	Mid-Cap	1231
9	Multi-Cap	938
9	Small-Cap	817
10	Small-Cap	102
11	Multi-Cap	761
11	Small-Cap	1273

Explanation

For the 1st month the total investment for the Mid-Cap fund is 949 and for Small-Cap is 767. Since there is no Large-Cap or Multi-Cap fund investments that month, they are not shown.

Question - 97
SQL: International Event

Query an event database to count participants by country with specific criteria:

- Include only participants aged 15-20 (inclusive)
- Include only participants from England, France, or Germany
- Group results by country
- Sort results by country name in descending order

▼ Schema

There is 1 table: event.

EVENT		
Name	Type	Description
id	int	Unique id of the participant.
name	varchar(15)	Name of the participant.
country	varchar(15)	Country the participant belongs to.
age	int	Age of the participant.

▼ Sample Data Tables

EVENT			
id	name	country	age
26	Taylor	France	15
27	Wesley	France	16
28	Jordan	England	18
29	Robin	Germany	22
30	Alex	Germany	16
31	Drew	France	20
32	Blake	Germany	16
33	Spencer	Belgium	22
34	Ellis	France	17
35	Morgan	Germany	16

OUTPUT	
country	total_participants
Germany	3
France	4
England	1

Explanation

In the sample data, there are 4 participants from Germany. The output shows only 3 because one is outside the age range. There is one participant from Belgium who is not from one of the countries requested.

Question - 98

DPI Software Protocols Report

Analyze network traffic data to identify protocols with imbalanced traffic flow. Your query should:

- Find all protocols where incoming traffic exceeds outgoing traffic.
- Display protocol name, incoming traffic amount, and outgoing traffic amount.
- Format output as: *protocol traffic_in traffic_out*
- Sort results alphabetically by protocol name.

▼ Schema

There is 1 table:

traffic

name	type	description
client	VARCHAR(17)	Client MAC address
protocol	VARCHAR(64)	Protocol name
traffic_in	INT	Traffic in
traffic_out	INT	Traffic out

▼ Sample Data Tables

For the sample data in table:

traffic			
client	protocol	traffic_in	traffic_out
02-E1-80-76-EC-4B	BGP	0	234737
43-15-AA-26-0F-A4	BGP	402860	606565
90-E7-B0-14-7E-8C	BGP	840772	988197
FB-60-23-C1-5E-D6	DNS	341155	356569
4D-6D-7F-62-F4-00	FTP	8346	413322
09-89-26-46-C4-21	FTP	210656	470568
B1-6A-35-2F-1A-C2	FTP	897097	161083
0C-CA-68-2D-4B-F5	HTTP	918793	550403
A4-C6-52-10-2E-9C	HTTPS	520856	185387
95-B8-7D-78-06-42	POP	150880	423073
B9-C1-1B-32-55-95	POP	862946	979544
14-FD-21-F6-5E-67	SMTP	139389	280646
70-E1-2D-B1-B2-9B	SMTP	163986	450401
C6-F1-59-FF-5D-BE	SMTP	271295	878246
62-01-CF-AD-32-A7	SMTP	388933	81625
41-80-FB-86-D1-93	SMTP	752842	253981
93-3F-01-57-5F-4A	SSH	496717	599280
52-F2-BF-45-84-74	SSH	632534	128765
87-66-B5-A5-2F-7B	SSH	835441	354950
CE-FC-80-F3-95-58	UDP	903443	120298

the expected output is:

protocol ▲	traffic_in	traffic_out
FTP	1116099	1044973
HTTP	918793	550403
HTTPS	520856	185387
SSH	1964692	1082995
UDP	903443	120298

Question - 99

Advertising System Failures Report

Analyze and advertising system's data to identify customers with high failure rates. Your query should:

- Find customers with more than 3 failure events in their campaigns (*status* = "failure").
- Format customer names as "first_name last_name" (space-separated).
- Report the customer name and the number of failures.
- Row order is unimportant.

▼ Schema

There are 3 tables:

customers		
name	type	description
id	SMALLINT	Customer ID
first_name	VARCHAR(64)	Customer first name
last_name	VARCHAR(64)	Customer last name

campaigns		
name	type	description
id	SMALLINT	Campaign ID
customer_id	SMALLINT	Customer ID
name	VARCHAR(64)	Campaign name

events		
name	type	description

dt	VARCHAR(19)	Event timestamp
campaign_id	SMALLINT	Campaign ID
status	VARCHAR(64)	Event status

▼ Sample Data Tables

For the sample data in tables:

customers		
id	first_name	last_name
1	Alex	Ferrero
2	Robin	Romera

campaigns		
id	customer_id	name
1	1	Upton Group
2	1	Roob, Hudson and Rippin
3	1	McCullough, Rempel and Larson
4	1	Lang and Sons
5	2	Ruecker, Hand and Haley

events		
dt	campaign_id	status
2021-12-02 13:52:00	1	failure
2021-12-02 08:17:48	2	failure
2021-12-02 08:18:17	2	failure
2021-12-01 11:55:32	3	failure
2021-12-01 06:53:16	4	failure
2021-12-02 04:51:09	4	failure
2021-12-01 06:34:04	5	failure
2021-12-02 03:21:18	5	failure
2021-12-01 03:18:24	5	failure

2021-12-02 15:32:37	1	success
2021-12-01 04:23:20	1	success
2021-12-02 06:53:24	1	success
2021-12-02 08:01:02	2	success
2021-12-01 15:57:19	2	success
2021-12-02 16:14:34	3	success
2021-12-02 21:56:38	3	success
2021-12-01 05:54:43	4	success
2021-12-02 17:56:45	4	success
2021-12-02 11:56:50	4	success
2021-12-02 06:08:20	5	success

The expected output is:

customer	failures ▼
Alex Ferrero	6

Question - 100

Election Exit Poll Report

Create a query to analyze election exit poll data. Your task is to:

- List each candidate with their total vote count
- Format candidate names as "first_name last_name" (space-separated)
- Count only votes that can be matched to valid candidates
- Format output as: candidate, votes
- Sort results by vote count (descending), then by candidate name (ascending)

▼ Schema

There are 2 tables:

candidates		
name	type	description
id	SMALLINT	Candidate ID
first_name	VARCHAR(64)	Candidate first name
last_name	VARCHAR(64)	Candidate last name

results		
name	type	description
candidate_id	SMALLINT	Candidate ID
vote_at	VARCHAR(19)	Vote timestamp

▼ Sample Data Tables

For the sample data in tables:

candidates		
id	first_name	last_name
1	Alex	Ping
2	Sam	Drewell
3	Chris	Scoble

results	
candidate_id	vote_at
0	2021-12-01 14:15:52
1	2021-12-01 03:55:23
1	2021-12-01 21:53:26
1	2021-12-02 07:57:40
1	2021-12-02 13:56:06
2	2021-12-01 11:46:40
2	2021-12-01 14:56:05
2	2021-12-01 21:54:50
2	2021-12-02 00:43:18
2	2021-12-02 06:59:33
2	2021-12-02 08:36:35
2	2021-12-02 10:20:33
2	2021-12-02 14:02:38
3	2021-12-01 05:18:34
3	2021-12-02 03:55:37
3	2021-12-02 05:30:24

3	2021-12-02 08:32:06
4	2021-12-02 05:05:55
5	2021-12-02 15:50:50
5	2021-12-02 20:45:08

the expected output is:

candidate	votes ▼
Sam Drewell	8
Alex Ping	4
Chris Scoble	4

Question - 101

Billing Analytics Customer Report

Identify Most Valued Customers (MVCs) from December transaction data. An MVC is defined as a customer with 3 or more transactions in the month. Your query should:

- List all MVCs in December
- Include each customer's name, transaction count, and total transaction amount
- Format output as: customer, transactions, total
- Sort results alphabetically by customer name

▼ Schema

There is 1 table:

events		
name	type	description
dt	VARCHAR(19)	Transaction timestamp
customer	VARCHAR(64)	Customer name
amount	DECIMAL(5,2)	Transaction amount

▼ Sample Data Tables

For the sample data in table:

events		
dt	customer	amount

2021-11-22 06:41:01	Donaugh Furneaux	0.89
2021-12-22 20:07:04	Donaugh Furneaux	10.51
2021-12-31 05:22:11	Donaugh Furneaux	55.92
2021-12-12 21:26:42	Harley Lyddiard	37.68
2021-11-22 21:24:30	Kippy Jelly	85.87
2021-11-25 07:00:29	Kippy Jelly	7.25
2021-12-16 16:48:32	Kippy Jelly	65.49
2021-11-22 23:30:55	Latrina Jackman	93.49
2021-11-24 19:38:52	Latrina Jackman	82.28
2021-11-30 22:59:33	Latrina Jackman	96.87
2021-12-30 13:05:34	Latrina Jackman	88.19
2021-11-22 02:08:02	Maribel Braim	20.19
2021-12-13 00:14:58	Maribel Braim	97.99
2021-12-26 13:22:20	Maribel Braim	57.06
2021-12-29 00:20:27	Maribel Braim	24.35
2021-11-25 14:29:29	Orrin Curley	6.69
2021-12-08 06:22:16	Orrin Curley	36.85
2021-12-09 15:32:16	Orrin Curley	11.04
2021-11-28 00:15:20	Rasla Venny	14.59
2021-12-25 09:58:23	Rasla Venny	6.41

the expected output is:

customer ▲	transactions	total
Maribel Braim	3	179.40

Question - 102

Accounting Software Balance Report

HackerFinance is developing an accounting software and needs to generate a list of customers and the change in their account balances during December.

The output should display each customer and their corresponding balance, formatted as "customer balance". The balance should show two places after the decimal, e.g., 0.00.

The change is calculated as the sum of debit transactions minus the sum of credit transactions during December. The results should be sorted in ascending order by customer name.

▼ Schema

There is 1 table:

transactions		
name	type	description
dt	VARCHAR(19)	Transaction timestamp
customer	VARCHAR(64)	Customer name
debit	DECIMAL(5,2)	Transaction debit
credit	DECIMAL(5,2)	Transaction credit

▼ Sample Data Tables

For the sample data in table:

transactions			
dt	customer	debit	credit
2021-11-30 12:48:22	Arney Cuff	6.43	16.12
2021-12-25 19:00:46	Arney Cuff	97.78	12.53
2021-11-27 21:34:24	Donaugh Furneaux	89.71	85.04
2021-11-25 07:31:37	Ferrell Brunn	63.58	28.58
2021-11-25 15:30:56	Gibbie Jurisic	25.81	13.75
2021-11-21 00:09:50	Harley Lyddiard	57.49	7.11
2021-12-01 07:37:42	Harley Lyddiard	48.33	82.35
2021-12-02 13:08:52	Harley Lyddiard	12.13	63.81
2021-11-24 03:51:13	Kippy Jelly	50.34	12.91
2021-12-04 10:11:40	Latrina Jackman	10.73	39.51
2021-12-12 13:02:50	Latrina Jackman	5.35	96.74
2021-12-20 17:31:44	Latrina Jackman	54.99	92.73
2021-12-30 13:40:43	Maribel Braim	57.06	21.37
2021-12-02 09:57:35	Orrin Curley	65.44	51.31
2021-12-14 19:57:25	Orrin Curley	40.04	96.44

2021-11-20 09:28:11	Rasla Venny	80.33	20.69
2021-12-08 03:31:31	Rasla Venny	55.43	99.04
2021-12-09 21:31:29	Rasla Venny	87.96	5.87
2021-12-11 13:02:54	Rasla Venny	45.42	55.81
2021-12-28 18:04:52	Rasla Venny	68.17	85.30

the expected output is:

customer ▲	balance
Arney Cuff	85.25
Harley Lyddiard	-85.70
Latrina Jackman	-157.91
Maribel Braim	35.69
Orrin Curley	-42.27
Rasla Venny	10.96

Question - 103

Least Earning Locations

- Query a ride-hailing company database to analyze earnings by city. Your query should:
- Calculate total earnings as the sum of all ride fares for each city
 - Display city names alongside their total earnings
 - Sort results first by earnings (ascending), then by city name (ascending)

▼ Schema

There are 3 tables: *CITIES*, *USERS*, and *RIDES*.

CITIES		
Name	Type	Description
id	String	The assigned ID to the city presented as 32 character UUID.
name	String	The name of the city.

USERS		
Name	Type	Description
id	String	The assigned ID to the user presented as 32 character UUID.
city_id	String	The id of the city in which this user resides.
name	String	The name of the user.

email	String	The email of the user.
-------	--------	------------------------

RIDES		
Name	Type	Description
id	String	The assigned ID to the ride presented as 32 character UUID.
user_id	String	The id of the user who took this ride.
distance	Integer	The travelled distance in this ride.
fare	Integer	The fare of this ride.

▼ Sample Data Tables

CITIES	
id	name
1	Cooktown
2	South Suzanne

USERS			
id	city_id	name	email
1	2	Robin Delgado	robindelgado@hotmail.com
2	2	Sam Williams	samwilliams@bradley.org
3	1	Pat Peterson	patpeterson@hotmail.com
4	1	Chris Wheeler	chrislwheeler@gmail.com
5	1	Jordan Lloyd	jordanlloyd@gmail.com
6	1	Morgan Powers	morganpowers@hansen.biz

RIDES			
id	user_id	distance	fare
1	1	21	200
2	3	6	55
3	2	30	230
4	2	16	125
5	2	11	110
6	6	30	285
7	3	18	170
8	1	6	50
9	2	4	40
10	1	10	90

11	6	11	95
12	5	16	140
13	3	24	220
14	6	17	160
15	2	23	205
16	3	11	90
17	6	5	50
18	3	19	180
19	5	22	205
20	4	6	60

South Suzanne 1050

Cooktown 1710

Explanation

1. In the city South Suzanne, there are 2 users, and the total fare of rides taken by those users is 1050.
2. In the city Cooktown, there are 4 users and the total fare of rides taken by those users is 1710.

Question - 104

The Superheroes Name

Query the *SUPERHERO* table to find superheroes whose names contain fewer than 7 characters. Your task is to:

- Select only the *NAME* field from superheroes with names shorter than 7 characters
- Sort the results by *ID* in ascending order

Schema

SUPERHERO		
Name	Type	Description
ID	Integer	A superhero ID in the inclusive range <i>[1, 1000]</i> . This field is the primary key.
NAME	String	A superhero name. This field contains between <i>1</i> and <i>100</i> characters (inclusive).
LATITUDE	Float	The latitude of the superhero.
LONGITUDE	Float	The longitude of the superhero.

Sample Input

SUPERHERO			
ID	NAME	LATITUDE	LONGITUDE
1	Batman	50.0	85.0
2	Spiderman	65.0	65.0
3	Thor	45.0	30.0
4	Deadpool	25.0	69.0
5	Hulk	30.0	20.0
6	Captain America	70.0	75.0
7	Superman	85.0	45.0

Sample Output

```
Batman
Thor
Hulk
```

Explanation

- For Batman, the id is 1, and the name has 6 characters.
 - For Spiderman, the id is 2, and the name 9 has characters.
 - For Thor, the id is 3, and the name 4 has characters.
 - Other records are analyzed similarly.
- Batman, Thor, and Hulk fewer than 7 characters in their names, so they are printed in increasing order of their ids: 1, 3, 5.

Question - 105

Examination Data

Write a query to find students who scored an even number of marks in their exams. The output should display:

- Student names in uppercase letters
 - Marks for each student
 - Results sorted alphabetically by first name and last name
- Your query should output the results in the format: *NAME MARKS*

▼ Schema

There is 1 table: *exam*

exam		
Name	Type	Description
NAME	STRING	space-separated first and last name, primary key.
MARKS	INTEGER	

▼ Sample Data Tables

exam

NAME	MARKS
Robin A	10
Sam B	6
Alex C	15

Sample Output

```
ROBIN A 10
SAM B 6
```

Explanation

- Robin A and Sam B have an even number of marks, so they are included in the query.

Question - 106

Bill Payment Query

Write a query to determine the maximum *VALUE* in the *INVOICES* table, then print the *NAMES* of all customers who have an invoice *VALUE* of that amount. Rows can be in any order.

▼ Schema

There are 2 tables: *CUSTOMERS* and *INVOICES*.

CUSTOMERS		
Name	Type	Description
NAME	STRING	Customer name
ID	STRING	relates to INVOICES.CUSTOMER_ID

INVOICES		
Name	Type	Description
CUSTOMER_ID	STRING	ID of the customer
VALUE	INTEGER	Total of outstanding invoices

▼ Sample Data Tables

CUSTOMERS	
NAME	ID
Chris	9aa8af3678334f0e8f0ebcb22ca46107
Robin	146434f56e564f0aa9d36333b3f20c70
Sam	17f6bc0447064a899e18635d737ea840

INVOICES	
CUSTOMER_ID	VALUE

17f6bc0447064a899e18635d737ea840	1000
9aa8af3678334f0e8f0ebcb22ca46107	500
146434f56e564f0aa9d36333b3f20c70	1000

SAMPLE OUTPUT

```
Robin
Sam
```

Question - 107

Range of Marks Scored

Write a query to print the maximum and minimum marks of students.

The result should be in the format: *MAX_MARKS MIN_MARKS*.

▼ Schema

There is 1 table: *marks*.

marks		
Name	Type	Description
ID	STRING	This is the student's ID. It is the primary key.
MARKS	INTEGER	These are the marks scored.

▼ Sample Data Tables

marks	
ID	MARKS
abc123	30
def456	70
def123	50

Sample Output

```
70 30
```

Explanation

Max marks = 70

Min marks = 30

Question - 108

Active Backlogs

Write a query to return a list of all students with at least one occurrence of a backlog item.

The result should include only the student names.

▼ Schema

There are 2 tables: `student` , `backlog` .

student		
Name	Type	Description
ID	INTEGER	unique id, the primary key.
NAME	STRING	

backlog		
Name	Type	Description
STUDENT_ID	INTEGER	foreign key, student.id
SUBJECT_ID	STRING	

▼ Sample Data Tables

student	
ID	NAME
1	Chris
2	Sam
3	Alex

backlog	
STUDENT_ID	SUBJECT_ID
1	abc123
3	def456

Sample Output

```
Alex
Chris
```

Explanation

Each student listed has at least one record in the backlog table.

Question - 109

Value of Properties Owned

Write a query to print the IDs of owners who have more than 100 million worth of houses and own more than 1 house.

The result should be in the format: *BUYER_ID TOTAL_WORTH*.

▼ Schema

There are 2 tables: *house* , *price* .

house		
Name	Type	Description
BUYER_ID	INTEGER	Unique buyer ID.
HOUSE_ID	STRING	Unique house ID.

price		
Name	Type	Description
HOUSE_ID	STRING	Unique house ID. The primary key.
PRICE	INTEGER	The price of the house.

▼ Sample Data Tables

house	
BUYER_ID	HOUSE_ID
1	abc123
2	def456
3	abc456
1	def123
2	def789

price	
HOUSE_ID	PRICE
abc123	60
def456	20
abc456	120
def123	40
def789	70

Sample Output

```
1 100
```

Question - 110

Aggregate Marks

Write a query to retrieve the records of students who have a sum of marks greater than or equal to 500.

The result should include *STUDENT_ID* and *SUM_OF_MARKS*, sorted in descending order by *STUDENT_ID*.

▼ Schema

There is 1 table: marks .

marks		
Name	Type	Description
STUDENT_ID	INTEGER	This is the student's unique ID.
MARKS	INTEGER	These are the marks obtained.

▼ Sample Data Table

marks	
STUDENT_ID	MARKS
1	450
2	200
3	260
2	300
3	250

Sample Output

3	510
2	500

Explanation

- 3 has a sum of 510, so it is printed in the query results.
- 1 has a sum of 450, so it does not get printed in the query results.
- 2 has a sum of 500, so it is printed in the query results.

Question - 111

Trip Query

Write a query to print the maximum number of discounted tours any single family in the *FAMILIES* table can choose from.

▼ Schema

There are 2 tables: FAMILIES , COUNTRIES .

FAMILIES		
Name	Type	Description
ID	STRING	Unique ID of the family.

NAME	STRING	Name of the primary contact.
FAMILY_SIZE	INTEGER	Size of the family.

COUNTRIES		
Name	Type	Description
ID	STRING	Unique ID of the country.
NAME	STRING	Name of the country.
MIN_SIZE	INTEGER	Minimum size group to get a discount.

▼ Sample Data Tables

FAMILIES		
ID	NAME	FAMILY_SIZE
c00dac11bde74750b4d207b9c182a85f	Alex Thomas	9
eb6f2d3426694667ae3e79d6274114a4	Chris Gray	2

COUNTRIES		
ID	NAME	MIN_SIZE
023fd23615bd4ff4b2ae0a13ed7efec9	Bolivia	2
be247f73de0f4b2d810367cb26941fb9	Cook Islands	4
3e85ab80a6f84ef3b9068b21dbcc54b3	Brazil	4

Sample Output

3

Explanation

The Thomas family can choose from any of the 3 tours and qualify for the discount. The Gray family only qualifies for 1.

Question - 112

City Revenue

Write a query to print the floor of the average revenue for each city. The average revenue is defined as the average of all agencies' estimates of revenue for a city. The output should be in the format: *CITY_NAME AVERAGE_REVENUE*. Row order is not important.

▼ Schema

There are 2 tables: CITIES , and REVENUE .

CITIES		
Name	Type	Description
CITY_CODE	INTEGER	The city's PINCODE. This is a primary key.

CITY_NAME	STRING	The city's name.
-----------	--------	------------------

REVENUE		
Name	Type	Description
CITY_CODE	INTEGER	The city's PINCODE.
REVENUE	INTEGER	The estimated revenue.

▼ Sample Data Tables

CITIES	
CITY_CODE	CITY_NAME
1	New York
2	London
3	Paris

REVENUE	
CITY_CODE	REVENUE
1	10
2	5
2	10
3	15

Sample Output

```
New York 10
London 7
Paris 15
```

Explanation

- New York has only one estimate, so it is the average.
- London has two estimates. The average of those is floor((5+9)/2) = 7.
- Paris has only one estimate, so it is the average.

Question - 113

Grade Assignment

Write a query to assign grades to students based on their marks using the following logic:

- Greater than 90: A+
- Greater than 70: A
- Greater than 50: B
- Greater than or equal to 40: C
- Less than 40: Fail

Return *ID*, *name*, *marks*, and the letter grade for each student.

▼ Schema

There is 1 table: *students*

students		
Name	Type	Description
id	INTEGER	This is the first column. It is the primary key.
name	VARCHAR	Name of the student.
marks	INTEGER	Total marks of the student

▼ Sample Data Tables

students		
id	name	marks
1	Matt	44
2	John	57
3	Lucas	78

Desired output

id	name	marks	grade
1	Matt	44	C
2	John	57	B
3	Lucas	78	A

Question - 114
Restaurant's Growth

Write a query to compute the moving average of how much customers spent over a window of 7 days (current day and 6 days before). The output should contain three columns: *VISITED_ON*, *AMOUNT*, and *AVG_AMOUNT* (average over 7 days). Row order does not matter.

▼ Schema

There is 1 table: customers

CUSTOMERS		
Name	Type	Description
id	INTEGER	The customer's id, Primary Key
name	VARCHAR	Name of the customer
phone	VARCHAR	Phone of the customer
visited_on	DATE	Date that the customer visited

amount	INTEGER	Amount he customer spent
--------	---------	--------------------------

▼ Sample Data Tables

CUSTOMERS				
id	name	phone	visited_on	amount
1	Julia	1234567890	2015-05-01	100
2	Samantha	1234567890	2015-05-02	200
3	Julia-Samantha	1234567890	2015-05-03	300

Sample Output

2015-05-01 100 100
2015-05-02 200 150
2015-05-03 300 200

Explanation

Row 1: On May 1, Julia visited and spent *100*. *avg_amount* = 100/1
Row 2: On May 2, Samantha visited and spent *200*. *avg_amount* = (100+200)/2 = 150
Row 3: On May 3, they both visited and spent 300. *avg_amount* = (100+200+300)/3 = 200

Question - 115
Examination Data Management

Write a query to display the student ID, the subject name, and the total number of times the student has taken exams in that particular subject. The order of the results is not important. The output should follow this format: *STUDENT.ID EXAMINATION.SUBJECT NUMBER_OF_TIMES*

▼ Schema

STUDENT		
Name	Type	Description
ID	Integer	The student's ID number. This is the primary key.
NAME	String	The student's name.

EXAMINATION		
Name	Type	Description
STUDENT_ID	Integer	The student's ID number.
SUBJECT	String	The subject's name.

▼ Sample Data Tables

STUDENT	
ID	NAME
1	Taylor

2	Wesley
3	Jordan
4	Robin
5	Alex

EXAMINATION	
STUDENT_ID	SUBJECT
1	Biology
1	Physics
3	History
4	Geography
4	Geography

Sample Output

```

1 Biology 1
1 Physics 1
3 History 1
4 Geography 2

```

Explanation

- *Taylor* appeared in *Biology* examination one time.
- *Taylor* appeared in the *Physics* examination one time.
- *Jordan* appeared in the *History* examination one time.
- *Robin* appeared in the *Geography* examination two times.

Question - 116

Botany Query

Write a query to print the *PLANT.NAME* and *WEATHER.TYPE* for each plant that blooms in only one type of weather. Row order is not important.

▼ Schema

There are 2 tables: `plant` and `weather`.

PLANT		
Name	Type	Description
NAME	String	primary key
SPECIES	String	relates to <i>weather.plant_species</i>
SEED_DATE	Date	

WEATHER		
Name	Type	Description

PLANT_SPECIES	String	
TYPE	String	

▼ Sample Data Tables

PLANT		
NAME	SPECIES	SEED_DATE
Marchantia1	Bryophyte	2012-06-14
Cycas1	Gymnosperm	2017-05-10
Marchantia2	Bryophyte	2016-09-27

WEATHER	
PLANT_SPECIES	TYPE
Bryophyte	Sunny
Gymnosperm	Rainy
Gymnosperm	Sunny
Gymnosperm	Dry

Sample Output

```
Marchantia1 Sunny
Marchantia2 Sunny
```

Explanation

1. The plant species Bryophyte blooms only in sunny weather. The result contains the plants with the names Marchantia1 and Marchantia2 whose species is Bryophyte.
2. The plant species Gymnosperm blooms in sunny, rainy, and dry weather.

Question - 117

The Perfect Arrangement

Write a query to print the *ID*, *FIRST_NAME*, and *LAST_NAME* of customers whose concatenated first and last names are less than 12 letters long.

Sort the results by:

1. Combined name length (ascending)
2. Combined name alphabetically (case insensitive, ascending)
3. ID (ascending)

Schema

CUSTOMER

Name	Type	Description
ID	Integer	unique id, primary key.
FIRST_NAME	String	
LAST_NAME	String	
COUNTRY	String	
CREDIT_LIMIT	Float	

Sample Input

CUSTOMER

ID	FIRST_NAME	LAST_NAME	COUNTRY	CREDIT_LIMIT
1	Alex	White	USA	200350.54
2	Tyler	Hanson	UK	15354.23
3	Jordan	Fernandez	France	359200.67
4	Drew	Bradley	Albania	1060.57
5	Blake	Fuller	USA	14789.00
6	Spencer	Johnston	China	100243.35
7	Ellis	Gutierrez	USA	998999.20
8	Morgan	Thomas	Canada	500500.23
9	Riley	Garza	UK	18782.44
10	Peyton	Harris	USA	158367.00

Sample Output

```
1      Alex White
9      Riley Garza
5      Blake Fuller
4      Drew Bradley
2      Tyler Hanson
```

Explanation

AlexWhite is 9 letters, so it is included in the results. JordanFernandez is 15 letters, so it is omitted. The last 3 names are the same length, so they are sorted alphabetically ascending.

Names that are excluded and their lengths

```
MorganThomas    12
PeytonHarris    12
EllisGutierrez  14
```

Question - 118
Relative Pay

Write a query to produce a list containing two columns:

1. The name of an employee who earns less than some other employee
2. The name of a higher-earning employee

Include all combinations of lesser and greater earning employees. Sort ascending, first by the lower earning employee's *ID*, then by the higher earning employee's *SALARY*.

▼ Schema

EMPLOYEE		
Name	Type	Description
ID	Integer	The ID of the employee. This is a primary key.
NAME	String	Employee name, 1 - 20 characters.
AGE	Integer	Employee age in years.
ADDRESS	String	Employee address, 1 - 25 characters.
SALARY	Integer	Employee salary.

▼ Sample Data Tables

EMPLOYEE				
ID	NAME	AGE	ADDRESS	SALARY
1	Chris	27	Paris	74635
2	Sam	30	Sydney	72167
3	Pat	29	Paris	75299

Sample Output

Chris Pat
Sam Chris
Sam Pat

Explanation

- Chris with ID 1 earns a salary of 74635.
- Sam with ID 2 earns a salary of 72167.
- Pat with ID 3 earns a salary of 75299.

Chris has the lowest ID and there is at least one other with a higher salary, so Chris is included in the list first. Only Pat earns more.
Sam has the next lowest ID and there is at least one other with a higher salary, so Sam is included next. Chris and Pat earn more than Sam. Chris earns less than Pat, so Chris is listed first.

Pat earns the most of all employees. Since there are no employees earning more, Pat is excluded from the listing.

Question - 119

Employees and UIN

A company maintains the data of its employees in two tables: *EMPLOYEE* and *EMPLOYEE_UIN*. Write a query to print the *NAME* and *UIN* of employees. If there is no unique identification number present corresponding to the employee, then print *NULL* instead as the *UIN*. The order of output does not matter.

Schemas

EMPLOYEE		
Name	Type	Description
ID	Integer	The ID of the employee. This is a primary key.
NAME	String	The name of the employee having <i>[1, 20]</i> characters.
AGE	Integer	The age of the employee.
ADDRESS	String	The address of the employee having <i>[1, 25]</i> characters.
SALARY	Integer	The salary of the employee.

EMPLOYEE_UIN		
Name	Type	Description
ID	Integer	The ID of the employee. This is a primary key.
UIN	Integer	The unique identification number of the employee.

Sample Input

EMPLOYEE				
ID	NAME	AGE	ADDRESS	SALARY
1	Ileana	27	Paris	74635
2	Bob	30	Sydney	72167
3	Julia	29	Paris	75299
4	Samantha	47	Sydney	46681
5	David	27	Texas	11843

EMPLOYEE_UIN	
ID	UIN
1	57520-0440
2	49638-001

3	63550-194
---	-----------

Sample Output

```
Samantha NULL
Ileana 57520-0440
Bob 49638-001
Julia 63550-194
David NULL
```

Explanation

- Ileana has UIN 57520-0440.
- Bob has UIN 49638-001.
- Julia has UIN 63550-194.
- Samantha does not have a UIN in the EMPLOYEE_UIN table, so NULL is printed.
- David does not have a UIN in the EMPLOYEE_UIN table, so NULL is printed.

Question - 120
Youngest Employees

Write a query to generate a list of all employees who are less than 25 years old.

Sort the results first by *NAME*, then by employee *ID*, both in ascending order. The result should include the *UIN* followed by the *NAME*.

▼ Schema

EMPLOYEE		
Name	Type	Description
ID	Integer	The ID of the employee. This is a primary key.
NAME	String	The name of the employee having <i>[1, 20]</i> characters.
AGE	Integer	The age of the employee.
ADDRESS	String	The address of the employee having <i>[1, 25]</i> characters.
SALARY	Integer	The salary of the employee.

EMPLOYEE_UIN		
Name	Type	Description
ID	Integer	The ID of the employee. This is a primary key.
UIN	String	The unique identification number of the employee.

▼ Sample Data Tables

Sample Input

EMPLOYEE				
ID	NAME	AGE	ADDRESS	SALARY
1	Sherrie	23	Paris	74635

2	Paul	30	Sydney	72167
3	Mary	24	Paris	75299
4	Sam	47	Sydney	46681
5	Dave	22	Texas	11843

EMPLOYEE_UIN	
ID	UIN
1	57520-0440
2	49638-001
3	63550-194
4	68599-6112
5	63868-453

Sample Output

```
63868-453 Dave
63550-194 Mary
57520-0440 Sherrie
```

Explanation

- *Sherrie* is 23 years old and has UIN *57520-0440*. This record is printed.
- *Paul* is 30 years old and has UIN *49638-001*. This record is not printed.
- A similar analysis is done on the remaining records.

None of the three names of people less than 25 years old is repeated, so print them in alphabetical order. There is no additional sorting by *ID* in this case.

Question - 121

Scoring System

Write a query to print the *ID* and *NAME* of each of the three highest scoring students.
 Print the records in descending order by *SCORE*. For matching scores, sort by *ID* in ascending order.

Schema

STUDENT		
Name	Type	Description
ID	Integer	unique ID, the primary key
NAME	String	student name
SCORE	Float	the Math score of the student

Sample Input

STUDENT		
ID	NAME	SCORE
1	Bob	50
2	John	65.5
3	Harry	45
4	Dick	85
5	Dev	25
6	Sid	98
7	Tom	90
8	Julia	70.5
9	Erica	81
10	Jerry	85

Sample Output

```
6 Sid
7 Tom
4 Dick
```

Explanation

The students are arranged in the descending order of their math scores, followed by the ascending order of their IDs, as shown below:

Sid > Tom > Dick > Jerry > Erica > Julia > John > Bob > Harry > Dev

Dick's and Jerry's scores were the same, so they are shown in ID order.

Question - 122
Students Score

The Math scores of students have been stored in the *STUDENT* table. Write a query to print the ids and the names of the students who have secured higher than the average score. The result should be sorted in ascending order of student ID.

Input Format

STUDENT		
Name	Type	Description
ID	Integer	unique ID, the primary key.
NAME	String	student name
SCORE	Float	the Math score of the student

Sample Input

STUDENT		
ID	NAME	SCORE
1	Bob	50
2	John	65.5
3	Harry	45
4	Dick	85
5	Dev	25
6	Sid	98
7	Tom	90
8	Julia	70.5
9	Erica	81
10	Jerry	85

Sample Output

```
4 Dick
6 Sid
7 Tom
8 Julia
9 Erica
10 Jerry
```

Explanation

The average score is $(50 + 65.5 + 45 + 85 + 25 + 98 + 90 + 70.5 + 81 + 85) / 10 = 69.5$.

- Bob scored 50, which is less than 69.5.
- John scored 65.5, which is less than 69.5.
- Harry scored 45, which is less than 69.5.
- Dick scored 85, which is more than 69.5.
- Dev scored 25, which is less than 69.5.
- Sid scored 98, which is more than 69.5
- Tom scored 90, which is more than 69.5.
- Julia scored 70.5, which is more than 69.5.
- Erica scored 81, which is more than 69.5.
- Jerry scored 85, which is more than 69.5.

Sid, Dick, Tom, Julia, Erica, and Jerry scored more than the average score.

Question - 123
Undelivered Orders

Write a query to print the number of orders that are not yet delivered, where *ORDERS.STATUS* is not equal to "DELIVERED".

Schema

ORDERS		
Name	Type	Description
ID	Integer	A number in the inclusive range <i>[1, 1000]</i> which uniquely identifies the order. This is the primary key.

ORDER_DATE	Date	The date when the order was placed.
STATUS	String	This is the order status. It can be PLACED, SHIPPED, IN TRANSIT, DELIVERED.
CUSTOMER_ID	Integer	A number in the inclusive range $[1, 1000]$ which uniquely identifies the customer who placed the order.

Sample Input

ORDERS			
ID	ORDER_DATE	STATUS	CUSTOMER_ID
1	2003-01-06	PLACED	363
2	2003-01-06	PLACED	128
3	2003-01-06	IN TRANSIT	181
4	2003-01-06	DELIVERED	121
5	2003-01-07	DELIVERED	114
6	2003-01-07	IN TRANSIT	278
7	2003-01-07	PLACED	114
8	2003-05-05	IN TRANSIT	350
9	2003-05-05	DELIVERED	103

Sample Output

6

Explanation

Out of all nine orders, the ones with *IDs* 4, 5, and 9 were delivered, while the other six have not been delivered.

Question - 124 Employee Salaries

Write a query to print the *NAME* and *SALARY* of all employees in the *EMPLOYEE* table who earn more than \$500.

Sort the results as follows:

- First by the last three characters of the employee name in ascending order
- If two or more employees have names ending with the same three characters, sort by salary in descending order
- If two or more employees have the same ending characters and salary, sort by ID in ascending order

Schema

EMPLOYEE		
Name	Type	Description
ID	Integer	A number which uniquely identifies the employee. This is the primary key.
NAME	String	The name of the employee.

SALARY	Integer	The salary of the employee.
--------	---------	-----------------------------

Sample Input

EMPLOYEE		
ID	NAME	SALARY
1	Candice	4685
2	Juliana	2253
3	Scarlet	2350
4	Ileana	1151

Sample Output

```

Juliana 2253
Ileana 1151
Candice 4685
Scarlet 2350

```

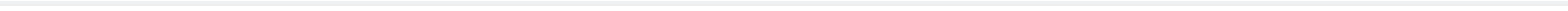
Explanation

- Juliana is an employee with ID 2 and earning a salary of \$2253.
- Ileana is an employee with ID 4 and earning a salary of \$1151.
- Candice is an employee with ID 1 and earning a salary of \$4685.
- Scarlet is an employee with ID 3 and earning a salary of \$2350.

Note that, Juliana's name is printed before Ileana's because they have the names ending at same last three characters and Juliana's salary is more than Ileana's.

Question - 125

Largest Number of Orders



Write a query to find the *CUSTOMER_ID* of the customer who has placed the largest number of orders.

Schema

ORDERS		
Name	Type	Description
ID	Integer	unique ID of the order, the primary key.
ORDER_DATE	Date	The date when the order was placed.
STATUS	String	This is the order status. It can be PLACED, SHIPPED, IN TRANSIT, DELIVERED.
CUSTOMER_ID	Integer	unique ID of the customer who placed the order.

Sample Input

ORDERS			
ID	ORDER_DATE	STATUS	CUSTOMER_ID
1	2003-01-06	PLACED	363
2	2003-01-06	PLACED	128

3	2003-01-06	IN TRANSIT	121
4	2003-01-06	DELIVERED	121

Sample Output

121

Explanation

The *CUSTOMER_ID* 121 has placed the largest number of orders, i.e, two orders.

Question - 126

Student Grades

Assign letter grades to students based on their scores.

Write a query to print each student's ID and letter grade using the following grading scale:

- F: Score < 20
- D: $20 \leq \text{Score} < 40$
- C: $40 \leq \text{Score} < 60$
- B: $60 \leq \text{Score} < 80$
- A: Score ≥ 80

Format the output as: "Student *STUDENT.ID* has grade: *letter grade*"

▼ Schema

STUDENT		
Name	Type	Description
ID	Integer	primary key
SCORE	Integer	

▼ Sample Data Tables

Sample Input

STUDENT	
ID	SCORE
1	20
2	50
3	50
4	68
5	95

Sample Output

```
Student 1 has grade: D
Student 2 has grade: C
Student 3 has grade: C
Student 4 has grade: B
Student 5 has grade: A
```

Explanation

- The student with ID 1 has a SCORE of 20 and the grade D.
- 50 translates to C
- 68 translates to B
- 95 translates to A

Question - 127
The First Orders

A company maintains information about its orders in the ORDERS table.

Print details of the earliest five undelivered orders sorted by order date (ascending). If there are more than five such orders, select those with the lowest order ID. Sort the output by increasing order ID.

▼ Schema

Table: Orders

column name	column type
id	int
order_date	date
status	varchar(50)
customer_id	int

▼ Sample Data Tables

Sample Input

ORDERS			
ID	ORDER_DATE	STATUS	CUSTOMER_ID
10100	2003-01-06	PLACED	363
10101	2003-01-06	PLACED	128
10102	2003-01-06	IN TRANSIT	181
10103	2003-01-06	DELIVERED	121
10104	2003-01-07	DELIVERED	114
10106	2003-01-07	IN TRANSIT	278

10120	2003-01-07	PLACED	114
10122	2003-05-05	IN TRANSIT	350
10123	2003-05-05	DELIVERED	103

Sample Output

```
10100 2003-01-06 PLACED 363
10101 2003-01-06 PLACED 128
10102 2003-01-06 IN TRANSIT 181
10106 2003-01-07 IN TRANSIT 278
10120 2003-01-07 PLACED 114
```

Explanation

The orders with order numbers 10100, 10101, 10102, 10106, and 10120 are the earliest placed orders and also have order status not equal to DELIVERED, so all their information in the increasing order of their order ID is printed.

Question - 128

Order Management System

A retail company maintains its customer data in the *CUSTOMER* table. List all customers sorted by their names in descending order. If multiple customers have the same *NAME*, sort them by *ID* in ascending order.

Input Format

CUSTOMER		
Name	Type	Description
ID	Integer	A customer ID in the inclusive range <i>[1, 1000]</i> . This is the primary key.
NAME	String	A customer name. This field contains between <i>1</i> and <i>100</i> characters (inclusive).
COUNTRY	String	The country of the customer.
CREDITS	Integer	The credit limit of the customer.

▼ Sample

CUSTOMER			
ID	NAME	COUNTRY	CREDITS
1	Frances White	USA	200350
2	Carolyn Bradley	UK	15354
3	Annie Fernandez	France	359200
4	Ruth Hanson	Albania	1060
5	Paula Fuller	USA	14789
6	Bonnie Johnston	China	100243
7	Ruth Gutierrez	USA	998999
8	Ernest Thomas	Canada	500500
9	Joe Garza	UK	18782
10	Anne Harris	USA	158367

Sample Output

```
4 Ruth Hanson
7 Ruth Gutierrez
5 Paula Fuller
9 Joe Garza
1 Frances White
8 Ernest Thomas
2 Carolyn Bradley
6 Bonnie Johnston
3 Annie Fernandez
10 Anne Harris
```

Explanation

According to the lexicographical arrangement,

Ruth Hanson > Ruth Gutierrez > Paula Fuller > Joe Garza > Frances White > Ernest Thomas > Carolyn Bradley > Bonnie Johnston > Annie Fernandez > Anne Harris

There are no duplicate names, so all records are in descending alphabetical *NAME* order.

Question - 129
Customers Credit Limit

A company maintains its customer data in the CUSTOMER table. Find customers from the USA with credit limits greater than 100000.

Print the IDs and names of these customers. Row order is unimportant.

Input Format

CUSTOMER		
Name	Type	Description
ID	Integer	A customer ID in the inclusive range $[1, 1000]$. This is the primary key.
NAME	String	A customer name. This field contains between 1 and 100 characters (inclusive).
COUNTRY	String	The country of the customer.
CREDITS	Integer	The credit limit of the customer.

Sample Input

CUSTOMER			
ID	NAME	COUNTRY	CREDITS
1	Frances White	USA	200350
2	Carolyn Bradley	UK	15354
3	Annie Fernandez	France	359200
4	Ruth Hanson	Albania	1060
5	Paula Fuller	USA	14789
6	Bonnie Johnston	China	100243
7	Ruth Gutierrez	USA	998999
8	Ernest Thomas	Canada	500500
9	Joe Garza	UK	18782
10	Anne Harris	USA	158367

Sample Output

```
1 Frances White
7 Ruth Gutierrez
10 Anne Harris
```

Explanation

Description of some of the customers is given below:

- Frances White is from the USA, and his credit limit is 200350, which is greater than 100000.
- Carolyn Bradley is from the UK, and her credit limit is 15354, which is less than 100000.
- Paula Fuller is from the USA, and her credit limit is 14789, which is less than 100000.
- Remaining records are analyzed similarly.

So, Frances White, Ruth Gutierrez, and Anne Harris are from USA and their credit is greater than 100,000.

Question - 130
Count the Employees

Find companies that have more than 10000 employees.

The data for the number of employees at several IT companies is maintained in the COMPANY table. Print the IDs of these companies. Order the results by ID.

211/235

Input Format

COMPANY		
Name	Type	Description
ID	Integer	A company ID in the inclusive range $[1, 1000]$. This is the primary key.
NAME	String	A company name. This field contains between 1 and 100 characters (inclusive).
EMPLOYEES	Integer	The total number of employees in the company.

Sample Input

COMPANY		
ID	NAME	EMPLOYEES
1	Adobe	28085
2	Flipkart	35543
3	Amazon	1089
4	Paytm	9982
5	BookMyShow	5589
6	Oracle	4003
7	NIIT	57782
8	Samsung	2000
9	TCS	10046
10	Wipro	3500

Sample Output

1
2
7
9

Explanation

Adobe, Flipkart, NIIT, and TCS have more than 10000 employees, so their IDs are printed.

Question - 131
The Superheroes Location

Find superheroes whose latitude and longitude coordinates are both less than 50. Print their IDs in any order.

Their locations are stored in the SUPERHERO table.

Input Format

SUPERHERO		
Name	Type	Description
ID	Integer	A superhero ID in the inclusive range $[1, 1000]$. This field is the primary key.
NAME	String	A superhero name. This field contains between 1 and 100 characters (inclusive).
LATITUDE	Float	The latitude of the superhero.
LONGITUDE	Float	The longitude of the superhero.

Sample Input

SUPERHERO			
ID	NAME	LATITUDE	LONGITUDE
1	Batman	50.23	85.45
2	Spiderman	65.43	65.66
3	Thor	45.34	30.89
4	Iron Man	85.34	80.98
5	Deadpool	25.12	69.21
6	Hulk	30.34	20.98
7	Doctor Strange	40.45	40.56
8	Captain America	70.00	75.32
9	Avengers	81.32	90.84
10	Superman	85.32	45.78

Sample Output

```

3
6
7

```

Explanation

- For Batman, *latitude* = 50.23 and *longitude* = 85.45. So, neither is less than 50.
- For Thor, *latitude* = 45.34 and *longitude* = 30.89. So, both are less than 50.
- For Deadpool, *latitude* = 25.12, and *longitude* = 69.21. So, the *latitude* is less than 50, but the *longitude* is greater than 50.
- The remaining records are analyzed similarly.

Thor, Hulk, and Doctor Strange have latitudes and longitudes less than 50, so their IDs are printed.

Question - 132
Employee Salaries and Departments

A company stores information in two tables: *Employee* and *Department*.

Write a query to print each employee's name, salary, department name, and department location. Sort the results by employee salary in descending order. If multiple employees have the same salary, sort them alphabetically by name.

Input Format

EMPLOYEE		
Name	Type	Description
ID	Integer	A unique ID, the primary key.
NAME	String	The name of the employee.
SALARY	Integer	The salary of the employee.
DEPT_ID	Integer	The employee department ID, a foreign key to DEPARTMENT.ID.

DEPARTMENT		
Name	Type	Description
ID	Integer	A unique ID, the primary key.
NAME	String	The name of the department.
LOCATION	String	The department location.

Sample Input

EMPLOYEE				DEPARTMENT		
ID	NAME	SALARY	DEPT_ID	ID	NAME	LOCATION
1	Candice	4685	1	1	Executive	Sydney
2	Julia	2559	2	2	Production	Sydney
3	Bob	4405	4	3	Resources	Cape Town
4	Scarlet	2350	1	4	Technical	Texas
5	Ileana	1151	4	5	Management	Paris

Sample Output

```
Candice 4685 Executive Sydney
Bob 4405 Technical Texas
Julia 2559 Production Sydney
Scarlet 2559 Executive Sydney
Ileana 1151 Technical Texas
```

Explanation

- Candice with id 1 and is in the department with dept_id 1 i.e., Executive which is located at Sydney and earns a salary of 4685.
- Bob with id 3 and is in the department with dept_id 4 i.e., Technical which is located at Texas and earns a salary of 4405.
- Julia with id 2 and is in the department with dept_id 2 i.e., Production which is located at Sydney and earns a salary of 2559.
- Scarlet with id 4 and is in the department with dept_id 1 i.e., Executive which is located at Sydney and earns a salary of 2559.
- Ileana with id 5 and is in the department with dept_id 4 i.e., Technical which is located at Texas and earns a salary of 1151.

The output contains the details of Candice followed by Bob, Julia, Scarlet, and Ileana.

Note that as Scarlet and Julia earn the same salary, they are displayed in alphabetical order of their names.

There are various collections of red, green, and blue balls. Categorize these collections based on their counts:

- GOOD: If the count of red, green, and blue balls are equal
- BAD: If the count of any two colored balls are equal
- WORSE: If all colored balls have different counts

Write a query to print the type represented by each row of the table. The output is case-sensitive, so print only GOOD, BAD, or WORSE.

Input Format

COLLECTION		
Name	Type	Description
RED	Integer	This describes the count of red balls. The count can be between 30 units and 100 units (inclusive).
GREEN	Integer	This describes the count of green balls. The count can be between 30 units and 100 units (inclusive).
BLUE	Integer	This describes the count of blue balls. The count can be between 30 units and 100 units (inclusive).

Output Format

Each row of the result should contain one of the types which are described above, in the following format. Note that the output is *case-sensitive*, so make sure to output only GOOD , BAD , or WORSE .

TYPE_OF_COLLECTION

Sample Input

COLLECTION		
RED	GREEN	BLUE
65	65	87
50	50	50
30	50	100
40	50	90
92	50	50

Sample Output

BAD
GOOD
WORSE
WORSE
BAD

Explanation

The type of collection represented by each row is explained below:

- The count of the red balls is equal to the count of the green balls. But the count of blue balls is not equal to the count of red or green balls. So, the type is BAD .
- The count of the red, green, and blue balls are equal. So, the type is GOOD .
- The count of the red, green, and blue balls are different from each other. So, the type is WORSE .
- The count of the red, green, and blue balls are different from each other. So, the type is WORSE .

- The count of the green balls is equal to the count of the blue balls. But the count of red balls is not equal to the count of green or blue balls. So, the type is BAD .

Question - 134

Counting Hackos

Find all hackers who have earned more than 100 hackos in less than 10 months.

Print the names in ascending order of their *ID*.

Input Format

HACKER		
Name	Type	Description
ID	Integer	A hacker ID, the primary key.
NAME	String	A hacker's name.
MONTHS	Integer	The number of months the hacker has been programming.
HACKOS	Integer	The number of points the hacker gained per month.

Sample Input

HACKER			
ID	NAME	MONTHS	HACKOS
1	Frances White	5	20
2	Carolyn Bradley	2	10
3	Annie Fernandez	10	5
4	Ruth Hanson	5	15
5	Paula Fuller	6	15
6	Bonnie Johnston	8	12
7	Ruth Gutierrez	7	10
8	Ernest Thomas	4	30
9	Joe Garza	5	25
10	Anne Harris	7	15

Sample Output

```
Ernest Thomas
Joe Garza
Anne Harris
```

Explanation

The respective earnings of hackers since they started coding are described below:

- Frances White started coding 5 months ago and earned 20 hackos per month. So the total earned hackos are $5 \times 20 = 100$.
- Carolyn Bradley started coding 2 months ago and earned 10 hackos per month. So the total earned hackos are $2 \times 10 = 20$.
- Annie Fernandez started coding 10 months ago and earned 5 hackos per month. So the total earned hackos are $10 \times 5 = 50$.
- Ruth Hanson started coding 5 months ago and earned 15 hackos per month. So the total earned hackos are $5 \times 15 = 75$.
- Paula Fuller started coding 6 months ago and earned 15 hackos per month. So the total earned hackos are $6 \times 15 = 90$.
- Bonnie Johnston started coding 8 months ago and earned 12 hackos per month. So the total earned hackos are $8 \times 12 = 96$.
- Ruth Gutierrez started coding 7 months ago and earned 10 hackos per month. So the total earned hackos are $7 \times 10 = 70$.
- Ernest Thomas started coding 4 months ago and earned 30 hackos per month. So the total earned hackos are $4 \times 30 = 120$.
- Joe Garza started coding 5 months ago and earned 25 hackos per month. So the total earned hackos are $5 \times 25 = 125$.
- Anne Harris started coding 7 months ago and earned 15 hackos per month. So the total earned hackos are $7 \times 15 = 105$.

So, Ernest Thomas, Joe Garza, and Anne Harris have earned more than 100 hackos in less than 10 months.

Question - 135

Maximum Cumulative Hackos

A coding platform maintains all the participating hackers' data in the *HACKER* table. Write a query to print the maximum total hackos, i.e., months $\times$ hackos. Also, print the number of hackers who have earned the maximum total hackos.

Input Format

HACKER		
Name	Type	Description
ID	Integer	A hacker ID in the inclusive range $[1, 1000]$. This is the primary key.
NAME	String	A hacker name. This field contains between <i>1</i> and <i>100</i> characters (inclusive).
MONTHS	Integer	The total number of months the hacker has been programming.
HACKOS	Integer	The total number of points the hacker gained per month.

Output Format

The result should contain the maximum total hackos earned (months $\times$ hackos). Also, print the number of hackers who have earned the maximum total hackos, in the following format:

MAXIMUM_HACKOS
NUMBER_OF_HACKERS

Sample Input

HACKER			
ID	NAME	MONTHS	HACKOS
1	Frances White	5	20
2	Carolyn Bradley	2	10
3	Annie Fernandez	10	5
4	Ruth Hanson	5	15
5	Paula Fuller	6	15
6	Bonnie Johnston	8	12
7	Ruth Gutierrez	7	10
8	Ernest Thomas	6	24
9	Joe Garza	8	18
10	Anne Harris	7	15

Sample Output

```
144 2
```

Explanation

The total number of hackos earned by each of the hackers is calculated as given below:

- *Frances White* has been coding for 5 months and earned 20 hackos per month. So the total earned hackos are $5 \times 20 = 100$.
- *Carolyn Bradley* has been coding for 2 months and earned 20 hackos per month. So the total earned hackos are $2 \times 20 = 40$.
- *Annie Fernandez* has been coding for 10 months and earned 5 hackos per month. So the total earned hackos are $10 \times 5 = 50$.
- *Ruth Hanson* has been coding for 5 months and earned 15 hackos per month. So the total earned hackos are $5 \times 15 = 75$.
- *Paula Fuller* has been coding for 6 months and earned 15 hackos per month. So the total earned hackos are $6 \times 15 = 90$.
- *Bonnie Johnston* has been coding for 8 months and earned 12 hackos per month. So the total earned hackos are $8 \times 12 = 96$.
- *Ruth Gutierrez* has been coding for 7 months and earned 10 hackos per month. So the total earned hackos are $7 \times 10 = 70$.
- *Ernest Thomas* has been coding for 6 months and earned 24 hackos per month. So the total earned hackos are $6 \times 24 = 144$.
- *Joe Garza* has been coding for 8 months and earned 18 hackos per month. So the total earned hackos are $8 \times 18 = 144$.
- *Anne Harris* has been coding for 7 months and earned 15 hackos per month. So the total earned hackos are $7 \times 15 = 105$.

So, the maximum earned hackos is 144 and there are two hackers who earned this maximum hackos: *Ernest Thomas* and *Joe Garza*.

Question - 136
Average Price in Category

A department store maintains data on customers, products, and purchase records in three tables: *CUSTOMER*, *PRODUCT*, and *PURCHASE*.

Write a query to print the category and the average of the price minus discount for the distinct products that are purchased in the category. The average price should be printed with four decimal places, e.g., 5.0000. Row order is not important.

▼ Schema

CUSTOMER		
Name	Type	Description
ID	Integer	A customer's ID in the inclusive range [1, 500]. This is a primary key.

NAME	String	A customer's name. This field contains between 1 and 100 characters (inclusive).
CITY	String	A city name. This field contains between 1 and 50 characters (inclusive).
STATE	String	A state name. This field contains between 1 and 50 characters (inclusive).

PRODUCT		
Name	Type	Description
ID	Integer	A product's ID in the inclusive range [1, 500]. This is a primary key.
NAME	String	A product's name. This field contains between 1 and 50 characters (inclusive).
CATEGORY	String	A category name of the product. This field contains between 1 and 50 characters (inclusive).
PRICE	Integer	The price of the product in the inclusive range [500, 1000].
DISCOUNT	Integer	The discount associated with the product in the inclusive range [5, 20].
AVAILABLE	Integer	The availability of a product. It is 1 if the product is available or it is 0 if the product is not available.

PURCHASE		
Name	Type	Description
ID	Integer	An id associated with a purchase that is done in the inclusive range [1, 1000]. This is a primary key.
CUSTOMER_ID	Integer	A customer ID. This is a foreign key to customer.customer_id.
PRODUCT_ID	Integer	A product ID. This is a foreign key to product.product_id.
PURCHASE_DATE	Date	The date associated with a purchase. The date falls in the range '2000-01-01' to '2000-12-31' (inclusive).

▼ Sample Data Tables

CUSTOMER			
ID	NAME	CITY	STATE
1	Pickett	Wilhelm Park	SD
2	Pickett	Ipswich	AZ
3	Poole	Farwell	KS
4	Pollard	Bent Pine	WV
5	Phelps	Momford Landing	VA

PRODUCT					
ID	NAME	CATEGORY	PRICE	DISCOUNT	AVAILABLE
1	P-1	C-5	720	10	1
2	P-2	C-1	935	17	1
3	P-3	C-2	588	19	1
4	P-4	C-4	619	5	0
5	P-5	C-1	803	16	1

PURCHASE			
ID	CUSTOMER_ID	PRODUCT_ID	PURCHASE_DATE
1	5	5	2000-06-04
2	1	5	2000-11-24
3	1	3	2000-10-02
4	5	1	2000-08-06
5	5	3	2000-06-22
6	3	1	2000-06-30
7	2	5	2000-01-10
8	2	1	2000-07-26
9	3	5	2000-09-23
10	4	5	2000-09-02
11	3	2	2000-05-25

Sample Output

```
C-1  852.5000
C-2  569.0000
C-5  710.0000
```

Explanation

By referring to the sample data above, we find that:

1. For category C-1, two distinct products are purchased:

- P-2 with price - discount equal to 918.
- P-5 with price - discount equal to 787.

The average for this category is $(918 + 787)/2 = 852.5000$

2. For category C-2, only one distinct product is purchased:

- P-3 with price - discount equal to 569.

The average for this category is 569.0000

3. For category C-5, only one distinct product is purchased:

- P-1 with price - discount equal to 710.

The average for this category is 710.0000

Question - 137

Connections in City

A blood bank maintains two tables: *DONOR*, which contains individuals willing to donate blood, and *ACCEPTOR*, which contains individuals in need of blood. The bank seeks to identify the number of donors and acceptors that can be matched within each city.

A donor and an acceptor can be matched if:

1. They have the same blood group.
2. They live in the same city.
3. The donor can donate at least the amount of blood that the acceptor requires.

Write a query returning two columns: the city and the count of donors matched to acceptors within that city, for cities with at least one match. The result should be ordered in ascending order by city.

▼ Schema

You are provided 2 tables: DONOR and ACCEPTOR.

DONOR		
Name	Type	Description
DID	Integer	It is the id of the donor.
NAME	String	It is the name of the donor.
GENDER	Character	It is the gender of the donor.
CITY	String	It is the city where the donor lives.
BG	String	It is the blood group of the donor.
AMOUNT	Integer	It is the amount of blood in <i>pints</i> , which the donator can donate.

ACCEPTOR		
Name	Type	Description
AID	Integer	It is the id of the acceptor.
NAME	String	It is the name of the acceptor.
GENDER	Character	It is the gender of the acceptor.
CITY	String	It is the city where the acceptor lives.
BG	String	It is the blood group of the acceptor.
AMOUNT	Integer	It is the amount of blood in <i>pints</i> , which the acceptor needs.

▼ Sample Data Tables

DONOR					
DID	NAME	GENDER	CITY	BG	AMOUNT
1	MARIA	F	Warne, NH	AB+	7
2	RUBY	F	East Natchitoché, PA	AB+	3
3	CHARLES	M	East Natchitoché, PA	A-	6
4	DOROTHY	F	East Natchitoché, PA	AB+	9
5	MICHAEL	M	Warne, NH	A+	1

ACCEPTOR					
AID	NAME	GENDER	CITY	BG	AMOUNT
1	LINDA	F	Warne, NH	A+	9
2	CHARLES	M	Warne, NH	AB+	8

3	RICHARD	M	East Natchitoché, PA	AB+	3
4	LINDA	F	East Natchitoché, PA	A+	1
5	PATRICIA	F	Warne, NH	A+	5

For "East Natchitoché, PA", both Ruby and Dorothy can donate blood to Richard. No other connections are possible, so the output is:

East Natchitoché, PA 2

Question - 138

Big Companies

An organization maintains employment data in three tables: *EMPLOYEE*, *COMPANY*, and *SALARY*.

Print the names of companies where the average employee salary is greater than 40000. Each row in the output must contain the name of a company in the *COMPANY.NAME* format.

▼ Schema

EMPLOYEE		
Name	Type	Description
ID	Integer	An employee ID in the inclusive range [1, 1000]. This is the primary key.
NAME	String	An employee name. This field contains between 1 and 100 characters (inclusive).

COMPANY		
Name	Type	Description
ID	Integer	A company ID in the inclusive range [1, 1000]. This is the primary key.
NAME	String	A company name. This field contains between 1 and 100 characters (inclusive).

SALARY		
Name	Type	Description
EMPLOYEE_ID	Integer	An employee ID in the inclusive range [1, 1000].
COMPANY_ID	Integer	A company ID in the inclusive range [1, 1000].
SALARY	Integer	The salary of the employee in the inclusive range [10000, 100000].

▼ Sample Data Tables

EMPLOYEE	
ID	NAME
1	Frances White
2	Carolyn Bradley

3	Annie Fernandez
4	Ruth Hanson
5	Paula Fuller
6	Bonnie Johnston
7	Ruth Gutierrez
8	Ernest Thomas
9	Joe Garza
10	Anne Harris

COMPANY	
ID	NAME
1	PeopleSoft Inc
2	Baker Hughes Incorporated
3	MDU Resources Group Inc.
4	DST Systems, Inc.
5	Williams Companies Inc
6	Fisher Scientific International Inc.
7	Emcor Group Inc.
8	Genuine Parts Company
9	MPS Group Inc.
10	Novellus Systems Inc

SALARY		
EMPLOYEE_ID	COMPANY_ID	SALARY
2	4	27779
2	9	36330
3	9	71466
3	10	22804
5	5	49892
6	4	31493
6	10	26888
7	3	87118
7	7	70767
7	9	39929

MDU Resources Group Inc.
Williams Companies Inc
Emcor Group Inc.
MPS Group Inc.

Explanation

The following companies have average employee salaries > 40,000.

- MDU Resources Group Inc. has one employee (ID 7) whose SALARY is 87118. The company's average employee salary is $87118 \div 1 = 87118$ and $87118 > 40000$.
- Williams Companies Inc has one employee (ID 5) whose SALARY is 49892. The company's average employee salary is $49892 \div 1 = 49892$.
- Emcor Group Inc. has one employee (ID 7) whose SALARY is 70767. The company's average employee salary is $70767 \div 1 = 70767$.
- MPS Group Inc. has three employees (IDs 2, 3, and 7) whose SALARY values are 36330, 71466, and 39929. The company's average employee salary $(36330 + 71466 + 39929) \div 3 = 49241.67$.

Question - 139
Scheduling Errors

Find all professors who teach courses outside of their departments. Return a list with professor names and their associated courses. No duplicate rows should appear in the results.

The output should contain two columns: *professor.name*, *course.name*.

▼ Schema

PROFESSOR		
Name	Type	Description
ID	Integer	unique id, primary key
NAME	String	
DEPARTMENT_ID	Integer	foreign key, <i>department.id</i>
SALARY	Integer	

DEPARTMENT		
Name	Type	Description
ID	Integer	unique id, primary key
NAME	String	

COURSE		
Name	Type	Description
ID	Integer	unique id, primary key
NAME	String	
DEPARTMENT_ID	Integer	foreign key, <i>department.id</i>
CREDITS	Integer	

SCHEDULE		
Name	Type	Description
PROFESSOR_ID	Integer	foreign key, <i>professor.id</i>
COURSE_ID	Integer	foreign key, <i>course.id</i>
SEMESTER	Integer	
YEAR	Integer	

▼ Sample Data Tables

PROFESSOR			
ID	NAME	DEPARTMENT_ID	SALARY
1	Alex Daniels	4	7169
2	Drew Knight	1	9793
3	Jordan Myers	4	25194
4	Tyler Rodriguez	3	9686
5	Blake Gome	2	30860
6	Spencer George	5	10487
7	Ellis Vasquez	4	6353
8	Morgan Flores	1	25796
9	Riley Gilbert	5	35678
10	Peyton Stevens	2	26648

DEPARTMENT	
ID	NAME
3	Biological Sciences
5	Technology
6	Humanities & Social Sciences
2	Clinical Medicine
4	Arts and Humanities
1	Physical Sciences

COURSE			
ID	NAME	DEPARTMENT_ID	CREDITS
9	Clinical Biochemistry	2	3
4	Astronomy	1	6
10	Clinical Neuroscience	2	5
1	Pure Mathematics and Mathematical Statistics	1	3
6	Geography	1	7
8	Chemistry	1	1
5	Physics	1	8
3	Earth Science	1	7
7	Materials Science and Metallurgy	1	5
2	Applied Mathematics and Theoretical Physics	1	5

SCHEDULE			
PROFESSOR_ID	COURSE_ID	SEMESTER	YEAR
4	4	3	2003
3	3	1	2011
1	7	5	2011
7	7	1	2010
4	6	1	2001
9	3	1	2012
10	2	4	2009
1	1	3	2014
1	2	3	2008
1	7	5	2007

Sample Output

```

Tyler Rodriguez Astronomy
Jordan Myers Earth Sciences
Alex Daniels Materials Science and Metallurgy
Ellis Vasquez Materials Science and Metallurgy
Tyler Rodriguez Geography
Riley Gilbert Earth Sciences
Peyton Stevens Applied Mathematics and Theoretical Physics
Alex Daniels Pure Mathematics and Mathematical Statistics
Alex Daniels Applied Mathematics and Theoretical Physics
Alex Daniels Materials Science and Metallurgy

```

Explanation

Example logic

- 1. Professor *Tyler Rodriguez's* *department_id* is 3, but the *Astronomy* course's *department_id* is 1.
- 2. Professor *Jordan Myers's* *department_id* is 4, but the *Earth Sciences* course's *department_id* is 1

Question - 140

Maximum Average Departmental Salary

A university maintains data on professors and their departments in two tables: *PROFESSOR* and *DEPARTMENT*.

Find the department name that has the highest average salary. Print the name of the department and its average salary rounded to four decimal places. If multiple departments have the highest average salary, list all of them in any order.

▼ Schema

PROFESSOR		
Name	Type	Description
ID	Integer	A professor's ID in the inclusive range [1, 1000]. This is a primary key.
NAME	String	A professor's name. This field contains between 1 and 100 characters (inclusive).
DEPARTMENT_ID	Integer	A professor's department ID. This is a foreign key to DEPARTMENT.ID.
SALARY	Float	A professor's salary in the inclusive range [5000, 40000].

DEPARTMENT		
Name	Type	Description
ID	Integer	A department ID in the inclusive range [1, 1000]. This is a primary key.
NAME	String	A department name. This field contains between 1 and 100 characters (inclusive).

▼ Sample Data Tables

PROFESSOR			
ID	NAME	DEPARTMENT_ID	SALARY
1	Shawn Rivera	1	22606
8	Ruth Price	3	9287
9	Julie Gonzalez	4	18870
2	Craig Elliott	5	27524
10	Scott Butler	1	26200
3	Nancy Russell	2	20067
4	Clarence Johnson	1	7249
7	Louis Schmidt	1	13437
5	Terry Thompson	3	28432

6	Keith Gilbert	5	12610
---	---------------	---	-------

DEPARTMENT	
ID	NAME
3	Biological Sciences
5	Technology
6	Humanities & Social Sciences
2	Clinical Medicine
4	Arts and Humanities
1	Physical Sciences

Sample Output

```
Technology 20067.0000
Clinical Medicine 20067.0000
```

Explanation

We find the following average departmental salaries:

- The average salary of department $ID = 1$ is $(22606 + 26200 + 7249 + 13437) / 4 = 17373.0000$.
- The average salary of department $ID = 2$ is $20067 / 1 = 20067.0000$.
- The average salary of department $ID = 3$ is $(9287 + 28432) / 2 = 18859.5000$.
- The average salary of department $ID = 4$ is $18870 / 1 = 18870.0000$.
- The average salary of department $ID = 5$ is $(27524 + 12610) / 2 = 20067.0000$.

The departments with $ID = 2$ and 5 have the highest average salary (20067.0000), and the department *names* are *Technology* and *Clinical Medicine* respective.

Resources

- [Cast](#) and [Round](#) in Oracle.
- [Cast](#) and [Round](#) in MS SQL Server.
- [Truncate](#) and [Round](#) in MySQL.
- [Cast](#) and [Round](#) in DB2

Question - 141

List the Course Names

Write a query to return a list of professor names and the courses they teach or have taught. The results can be in any order but must not contain duplicate rows.

▼ Schema

PROFESSOR		
Name	Type	Description
ID	Integer	unique id, primary key

NAME	String	
DEPARTMENT_ID	Integer	This is a foreign key to DEPARTMENT.ID.
SALARY	Integer	

DEPARTMENT		
Name	Type	Description
ID	Integer	unique id, primary key
NAME	String	

COURSE		
Name	Type	Description
ID	Integer	unique id, primary key
NAME	String	
DEPARTMENT_ID	Integer	foreign key to DEPARTMENT.ID
CREDITS	Integer	

SCHEDULE		
Name	Type	Description
PROFESSOR_ID	Integer	foreign key to PROFESSOR.ID
COURSE_ID	Integer	foreign key to COURSE.ID
SEMESTER	Integer	
YEAR	Integer	

▼ Sample Data Tables

PROFESSOR			
ID	NAME	DEPARTMENT_ID	SALARY
1	Alex Burton	5	7340
8	Jordan Diaz	1	17221
9	Drew Hicks	5	16613
2	Tyler Matthews	2	14521
10	Blake Foster	4	28526
3	Spencer Peters	1	10487
4	Ellis Marshall	3	6353
7	Morgan Lee	2	25796

5	Riley Peterson	1	35678
6	Peyton Fields	5	26648

DEPARTMENT	
ID	NAME
3	Biological Sciences
5	Technology
6	Humanities & Social Sciences
2	Clinical Medicine
4	Arts and Humanities
1	Physical Sciences

COURSE			
ID	NAME	DEPARTMENT_ID	CREDITS
9	Clinical Biochemistry	2	3
4	Astronomy	1	6
10	Clinical Neuroscience	2	5
1	Pure Mathematics and Mathematical Statistics	1	3
6	Geography	1	7
8	Chemistry	1	1
5	Physics	1	8
3	Earth Science	1	7
7	Materials Science and Metallurgy	1	5
2	Applied Mathematics and Theoretical Physics	1	5

SCHEDULE			
PROFESSOR_ID	COURSE_ID	SEMESTER	YEAR
5	3	6	2012
7	3	1	2013
5	7	6	2010
2	10	2	2004
5	1	1	2011
2	9	4	2005
7	10	6	2009
5	6	4	2007

7	9	1	2014
9	9	5	2011

Sample Output

```
Tyler Matthews Clinical Biochemistry
Tyler Matthews Clinical Neuroscience
Drew Hicks Clinical Biochemistry
Morgan Lee Clinical Biochemistry
Morgan Lee Clinical Neuroscience
Morgan Lee Earth Science
Riley Peterson Earth Science
Riley Peterson Geography
Riley Peterson Materials Science and Metallurgy
Riley Peterson Pure Mathematics and Mathematical Statistics
```

Question - 142

Professor Names and Salaries

A university maintains data on professors and departments in two tables: *PROFESSOR* and *DEPARTMENT*. Write a query to find the *NAME* and *SALARY* of each professor who meets both of these conditions:

- Does not work in the Arts and Humanities department
- Has a salary greater than the lowest salary of any professor in the Arts and Humanities department

Display the name before the salary in your results. The order of rows does not matter.

▼ Schema

DEPARTMENT		
Name	Type	Description
ID	Integer	A department ID in the inclusive range [1, 1000]. This is a primary key.
NAME	String	A department name. This field contains between 1 and 100 characters.

PROFESSOR		
Name	Type	Description
ID	Integer	A professor's ID in the inclusive range [1, 1000]. This is a primary key.
NAME	String	A professor's name. This field contains between 1 and 100 characters.
DEPARTMENT_ID	Integer	A professor's department ID. This is a foreign key to DEPARTMENT.ID.
SALARY	Integer	A professor's salary in the inclusive range [5000, 40000].

▼ Sample Data Tables

DEPARTMENT	
ID	NAME
3	Biological Sciences
5	Technology

6	Humanities & Social Sciences
2	Clinical Medicine
4	Arts and Humanities
1	Physical Sciences

PROFESSOR			
ID	NAME	DEPARTMENT_ID	SALARY
1	Shauna Rivera	1	22606
8	Ruth Price	3	9287
9	Julie Gonzalez	4	18870
2	Craig Elliott	5	27524
10	Scott Butler	1	26200
3	Nancy Russell	2	7076
4	Clarence Johnson	1	7249
7	Louis Schmidt	1	13437
5	Terri Thompson	3	28432
6	Keith Gilbert	5	12610

Sample Output

```
Shauna Rivera 22606
Craig Elliott 27524
Terri Thompson 28432
Scott Butler 26200
```

Explanation

Julie Gonzalez has a salary of 18870 , which is smaller than the salary of any other professor in the Arts and Humanities department. The following employees of other departments have salaries higher than Julie's:

- Shauna Rivera 's salary of 22606 is higher than Julie Gonzalez 's.
- Craig Elliott 's salary of 27524 is higher than Julie Gonzalez 's.
- Terri Thompson 's salary of 28432 is higher than Julie Gonzalez 's.
- Scott Butler 's salary of 26200 is higher than Julie Gonzalez 's.

Question - 143

Student's Major

A university maintains data on students and their majors in three tables: *STUDENTS*, *MAJORS*, and *REGISTER*.

Write a query to display *STUDENT_NAME* and *MAJOR_NAME*. Sort the results by *STUDENT_ID* and return the first 20 records.

▼ Table Schema

STUDENTS		
Name	Type	Description
STUDENT_ID	Integer	The ID of a student. This is the <i>primary key</i> .
STUDENT_NAME	String	The name of the student.
STUDENT_AGE	Integer	The age of the student.

MAJORS		
Name	Type	Description
MAJOR_ID	Integer	The ID of a major. This is the <i>primary key</i> .
MAJOR_NAME	String	The name of a major.

REGISTER		
Name	Type	Description
STUDENT_ID	Integer	The ID of a student. This is a <i>foreign key</i> .
MAJOR_ID	Integer	The ID of a major. This is a <i>foreign key</i> .

▼ Sample Case 0

Sample Input 0

STUDENTS			MAJORS		REGISTER	
STUDENT_ID	STUDENT_NAME	STUDENT_AGE	MAJOR_ID	MAJOR_NAME	STUDENT_ID	MAJOR_ID
1	Alex	20	1000	Computer Science	2	1000
2	Chris	21	2000	Biology	3	3000
3	Robin	21	3000	Physics	1	2000

Sample Output

Alex Biology
Chris Computer Science
Robin Physics

Question - 144

Clumsy Administrator

Your company maintains employee records in a database. Due to some administrative issues, employee records have been inserted multiple times into the employee table. A record is considered a duplicate only when all columns (fields) of that record are exactly the same.

Write a query to find the names of employees whose complete records appear more than once in the table. Each name should appear only once in your results. The order of results does not matter.

▼ Schema

You are provided 1 table: EMPLOYEE.

EMPLOYEE		
Name	Type	Description
NAME	STRING	The name of the employee.
PHONE	STRING	The telephone number of the employee.

AGE	INTEGER	The age of the employee.
-----	---------	--------------------------

▼ Sample Data Tables

EMPLOYEE		
NAME	PHONE	AGE
Sam	1000040000	30
Alex	1000020000	60
Alex	1000020012	65
Sam	1000040000	30
Chris	1000012000	34
Chris	1000012000	34

Chris
Sam

Here, the exact records of Sam and Chris occur more than once in the table. Hence, Sam and Chris are the employees in the resultant output.

Question - 145

Student Rank

A university stores students' standardized test scores in a table named *STUDENT*. Student X placed 213th on the test.

Write a query to find Student X's test score (the 213th highest score in the *STUDENT.SCORE* column).

▼ Schema

A table is provided: `STUDENT` .

STUDENT		
Name	Type	Description
ID	Integer	The student's unique ID. This is a <i>primary key</i> .
AGE	Integer	The student's age.
SCORE	Integer	The student's standardized test score.

▼ Sample Data Tables

STUDENT		
ID	AGE	SCORE
1	19	91
2	20	90

3	20	87
4	21	72
5	19	98
6	20	50

The table's scores (from highest to lowest) are {98, 91, 90, 87, 72, 50}. As an example, the fourth-highest score is 87. For the real query, find the 213th highest score.